

An Axiomatic Model of Causality in Behaviour-Oriented Concurrency



Luke
Cheeseman



Elias
Castegren



Sophia
Drossopoulou



Tobias
Wrigstad



Matt
Parkinson



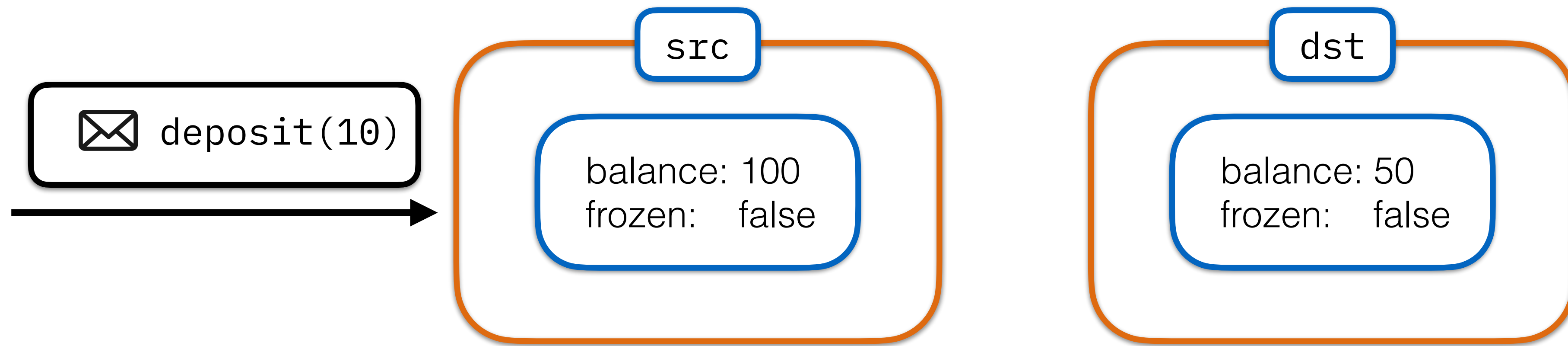
Why did we design Behaviour-oriented Concurrency (BoC)?

Why did we design Behaviour-oriented Concurrency (BoC)?



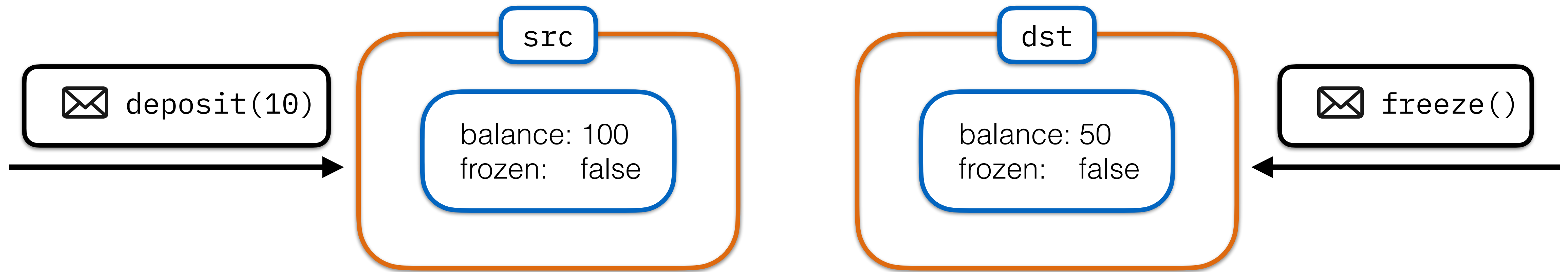
Inspired by the Actor model

Why did we design Behaviour-oriented Concurrency (BoC)?



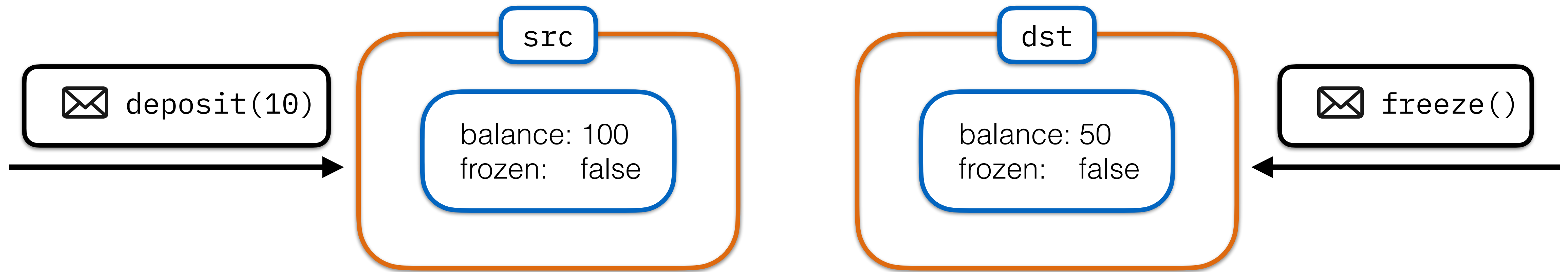
Inspired by the Actor model

Why did we design Behaviour-oriented Concurrency (BoC)?



Inspired by the Actor model

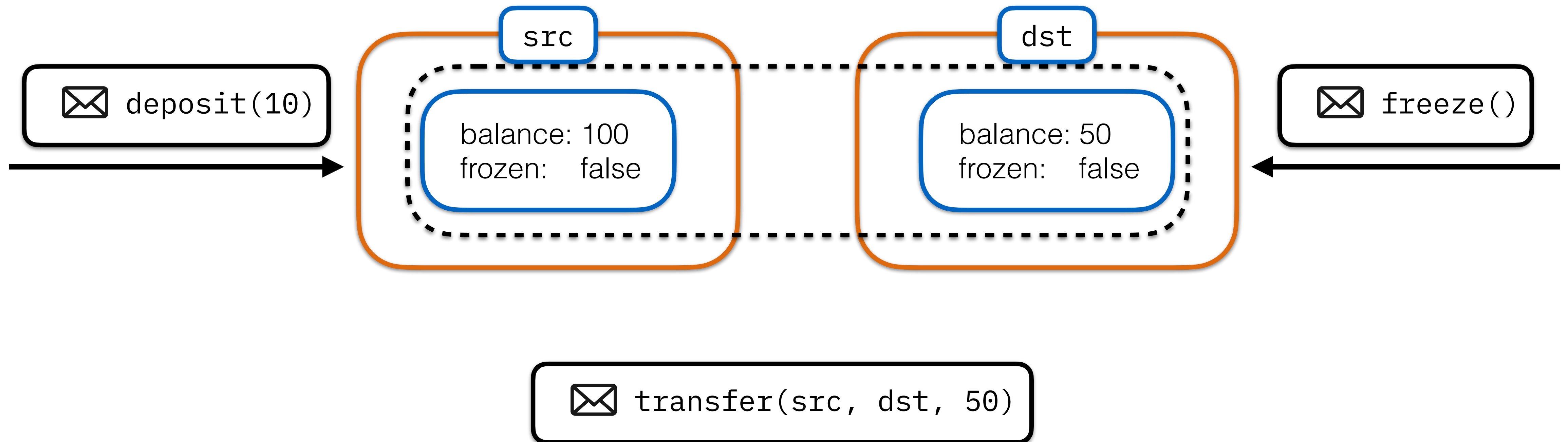
Why did we design Behaviour-oriented Concurrency (BoC)?



Inspired by the Actor model

Highly concurrent and data-race free

Why did we design Behaviour-oriented Concurrency (BoC)?

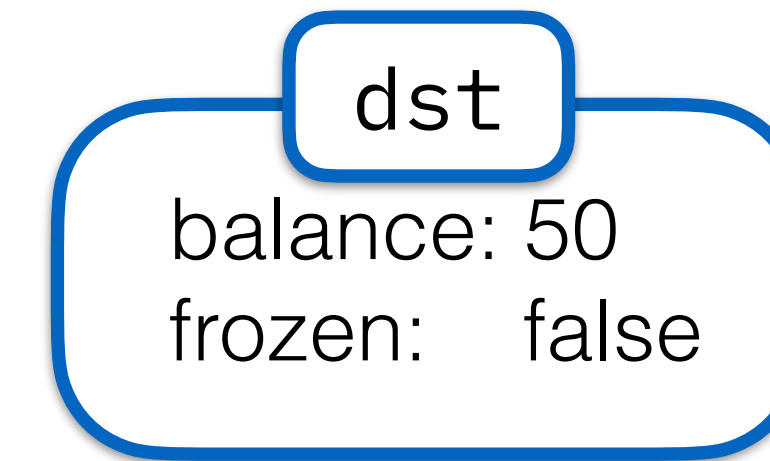
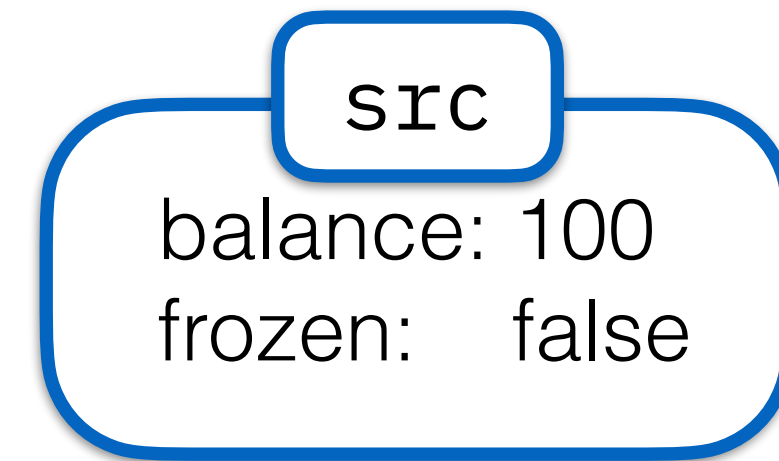


Inspired by the Actor model

Highly concurrent and data-race free

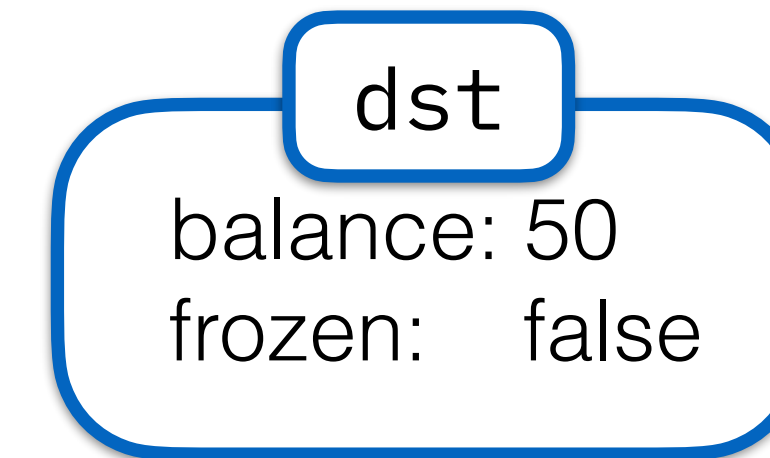
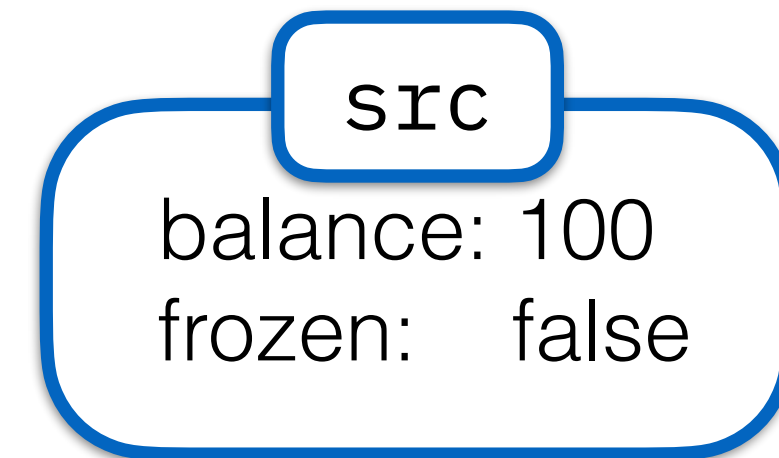
But, coupling isolation and concurrency makes multi-actor updates are hard

Why did we design Behaviour-oriented Concurrency (BoC)?



BoC decouples isolation and concurrency

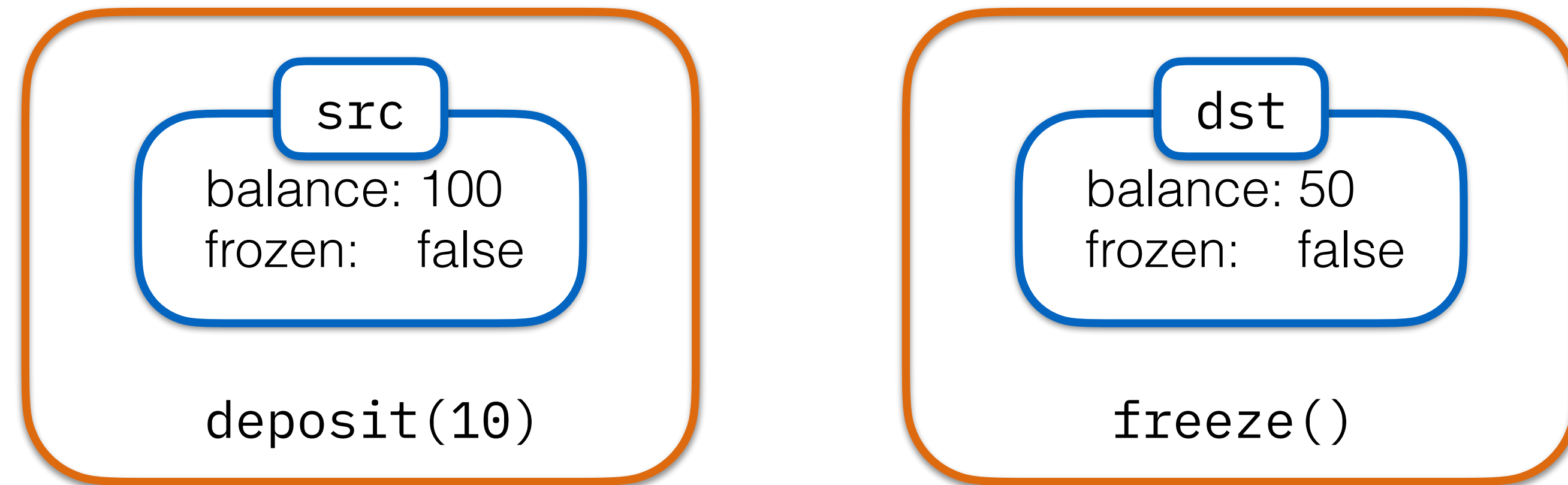
Why did we design Behaviour-oriented Concurrency (BoC)?



BoC decouples isolation and concurrency

Program state is isolated in *Cowns*

Why did we design Behaviour-oriented Concurrency (BoC)?

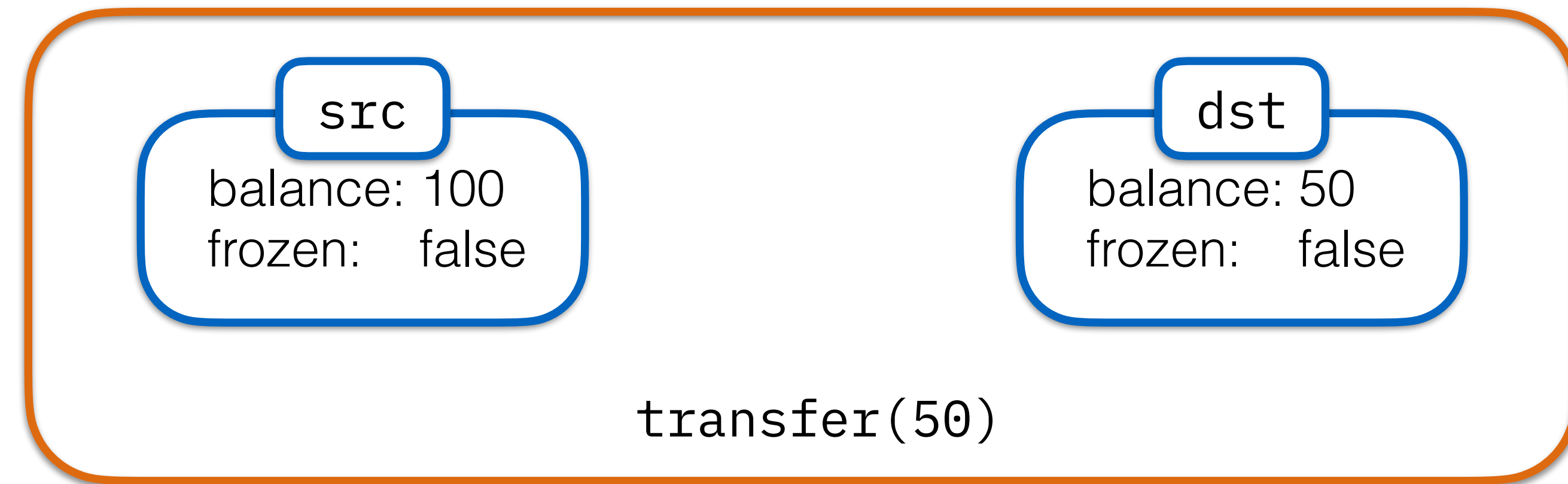


BoC decouples isolation and concurrency

Program state is isolated in *Cowns*

The unit of concurrency is still *Behaviours*

Why did we design Behaviour-oriented Concurrency (BoC)?



BoC decouples isolation and concurrency

Program state is isolated in *Cowns*

The unit of concurrency is still *Behaviours*, but they can access multiple cowns

The state of BoC today

OOPSLA 2023

When Concurrency Matters: Behaviour-Oriented Concurrency

LUKE CHEESEMANN, Imperial College London, UK
MATTHEW J. PARKINSON, Microsoft Azure Research, UK
SYLVAN CLEBSCH, Microsoft Azure Research, UK
MARIOS KOGIAS, Imperial College London, UK Microsoft Research, UK
SOPHIA DROSSOPOULOU, Imperial College London, UK
DAVID CHISNALL, Microsoft, UK
TOBIAS WRIGSTAD, Uppsala University, Sweden
PAUL LIÉTAR, Imperial College London, UK

Operational Semantics

$$\begin{array}{l} \text{STEP} \frac{E, h \hookrightarrow E', h'}{R \uplus (\bar{\kappa}, E), P, h \rightsquigarrow R \uplus (\bar{\kappa}, E'), P, h'} \quad \text{SPAWN} \frac{E \hookrightarrow_{\text{when } (\bar{\kappa}') \{E''\}} E'}{R \uplus (\bar{\kappa}, E), P, h \rightsquigarrow R \uplus (\bar{\kappa}, E'), P : (\bar{\kappa}', E''), h} \\ \\ \text{RUN} \frac{(\bigcup_{(\bar{\kappa}', _) \in (P' \cup R)} \bar{\kappa}') \cap \bar{\kappa} = \emptyset}{R, P' : (\bar{\kappa}, E) : P'', h \rightsquigarrow R \uplus (\bar{\kappa}, E), P' : P'', h} \quad \text{END} \frac{\text{finished}(E)}{R \uplus (\bar{\kappa}, E), P, h \rightsquigarrow R, P, h} \end{array}$$

The state of BoC today

OOPSLA 2023

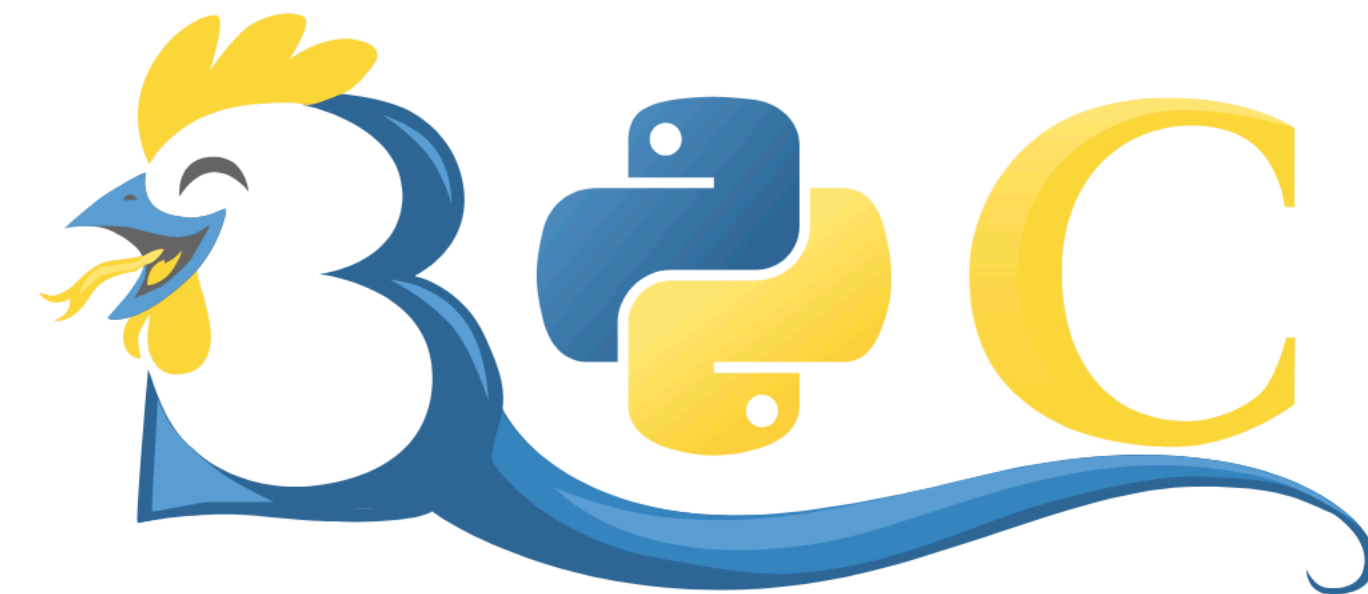
When Concurrency Matters: Behaviour-Oriented Concurrency

LUKE CHEESEMAN, Imperial College London, UK
MATTHEW J. PARKINSON, Microsoft Azure Research, UK
SYLVAN CLEBSCH, Microsoft Azure Research, UK
MARIOS KOGIAS, Imperial College London, UK Microsoft Research, UK
SOPHIA DROSSOPOULOU, Imperial College London, UK
DAVID CHISNALL, Microsoft, UK
TOBIAS WRIGSTAD, Uppsala University, Sweden
PAUL LIÉTAR, Imperial College London, UK

Operational Semantics

$$\begin{array}{c} \text{STEP} \frac{E, h \hookrightarrow E', h'}{R \uplus (\bar{\kappa}, E), P, h \rightsquigarrow R \uplus (\bar{\kappa}, E'), P, h'} \quad \text{SPAWN} \frac{E \hookrightarrow_{\text{when } (\bar{\kappa}') \{E''\}} E'}{R \uplus (\bar{\kappa}, E), P, h \rightsquigarrow R \uplus (\bar{\kappa}, E'), P : (\bar{\kappa}', E''), h} \\ \\ \text{RUN} \frac{(\bigcup_{(\bar{\kappa}', _) \in (P' \cup R)} \bar{\kappa}') \cap \bar{\kappa} = \emptyset}{R, P' : (\bar{\kappa}, E) : P'', h \rightsquigarrow R \uplus (\bar{\kappa}, E), P' : P'', h} \quad \text{END} \frac{\text{finished}(E)}{R \uplus (\bar{\kappa}, E), P, h \rightsquigarrow R, P, h} \end{array}$$

<https://microsoft.github.io/bocpy/>



Behavior-Oriented Concurrency in Python

Lock-less, Deadlock-free, Ownership-based

The state of BoC today

OOPSLA 2023

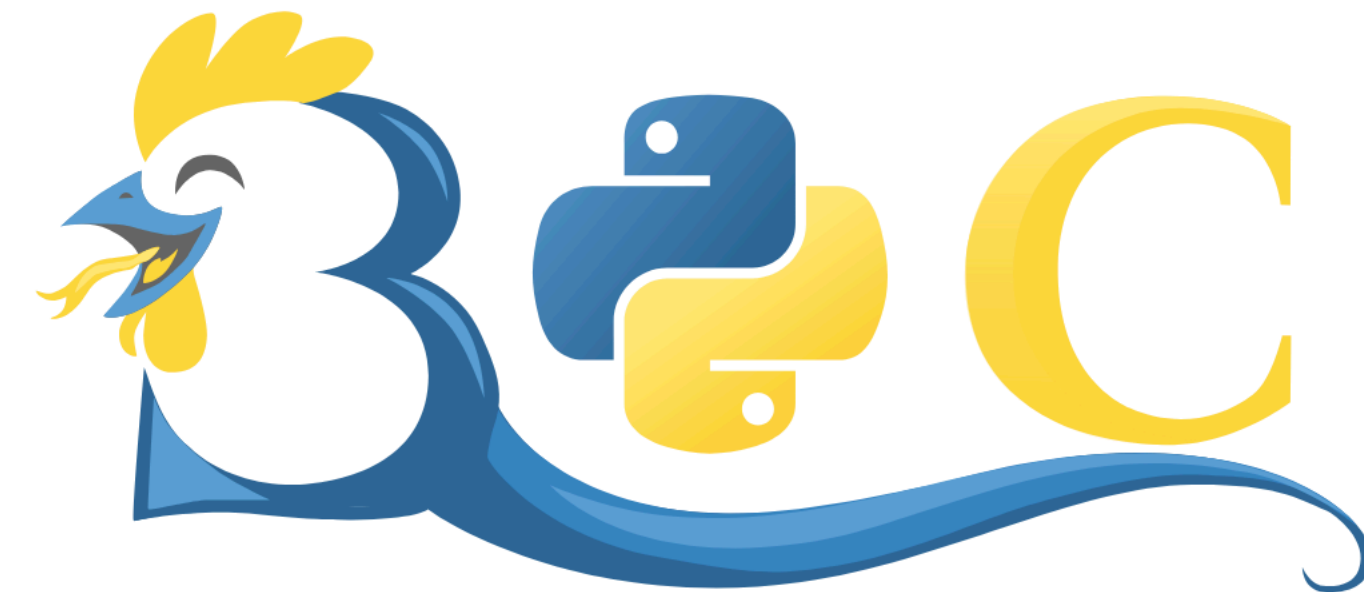
When Concurrency Matters: Behaviour-Oriented Concurrency

LUKE CHEESEMAN, Imperial College London, UK
MATTHEW J. PARKINSON, Microsoft Azure Research, UK
SYLVAN CLEBSCH, Microsoft Azure Research, UK
MARIOS KOGIAS, Imperial College London, UK Microsoft Research, UK
SOPHIA DROSSOPOULOU, Imperial College London, UK
DAVID CHISNALL, Microsoft, UK
TOBIAS WRIGSTAD, Uppsala University, Sweden
PAUL LIÉTAR, Imperial College London, UK

Operational Semantics

$$\begin{array}{c} \text{STEP} \frac{E, h \hookrightarrow E', h'}{R \uplus (\bar{\kappa}, E), P, h \rightsquigarrow R \uplus (\bar{\kappa}, E'), P, h'} \quad \text{SPAWN} \frac{E \hookrightarrow_{\text{when } (\bar{\kappa}') \{E''\}} E'}{R \uplus (\bar{\kappa}, E), P, h \rightsquigarrow R \uplus (\bar{\kappa}, E'), P : (\bar{\kappa}', E''), h} \\ \\ \text{RUN} \frac{(\bigcup_{(\bar{\kappa}', _) \in (P' \cup R)} \bar{\kappa}') \cap \bar{\kappa} = \emptyset}{R, P' : (\bar{\kappa}, E) : P'', h \rightsquigarrow R \uplus (\bar{\kappa}, E), P' : P'', h} \quad \text{END} \frac{\text{finished}(E)}{R \uplus (\bar{\kappa}, E), P, h \rightsquigarrow R, P, h} \end{array}$$

<https://microsoft.github.io/bocpy/>



Behavior-Oriented Concurrency in Python

Lock-less, Deadlock-free, Ownership-based

But, ...

We specified *how*, and not *what*

How a program runs?

Operational semantics and implementations

We specified *how*, and not *what*

How a program runs?

Operational semantics and implementations

What a program may do?

Implementation-independent design

?

We have the *how*, but not the *what*

We specified *how*, and not *what*

How a program runs?

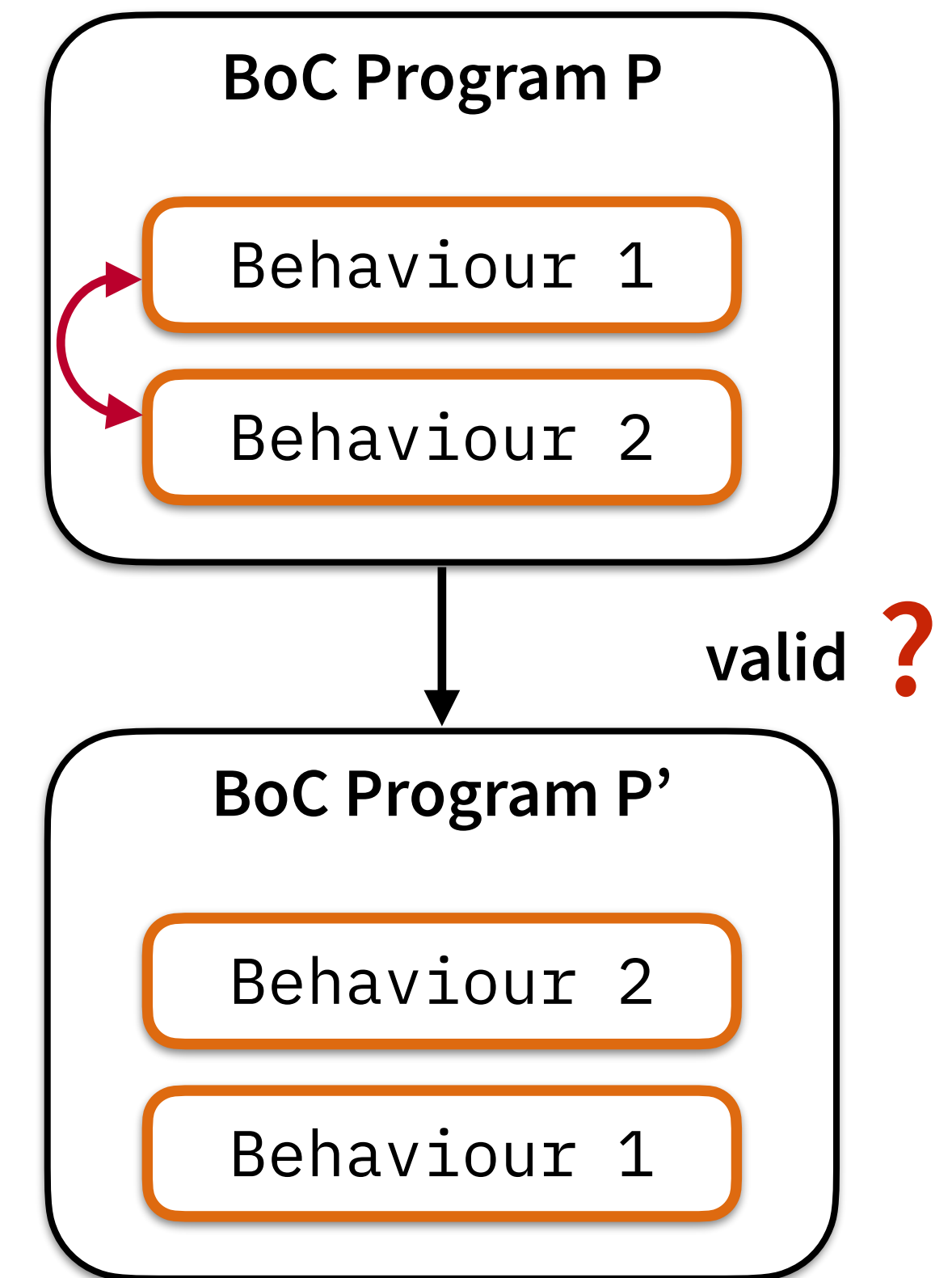
Operational semantics and implementations

What a program may do?

Implementation-independent design

?

We have the *how*, but not the *what*



We can't transform without the design

We specified *how*, and not *what*

What a program may do?

?

Implementation-independent design

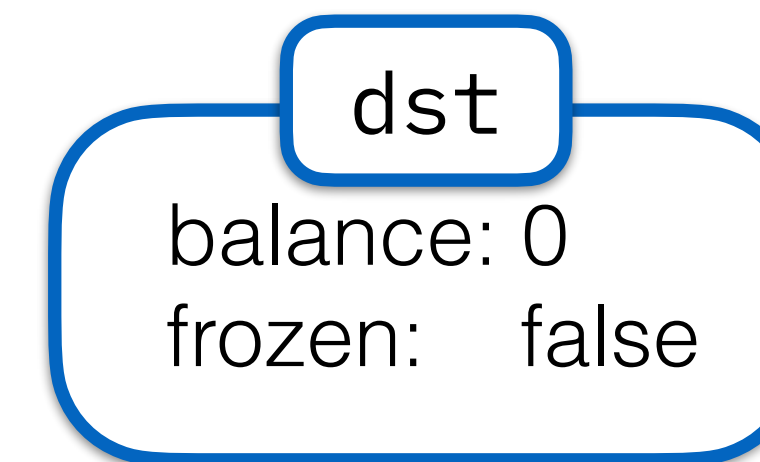
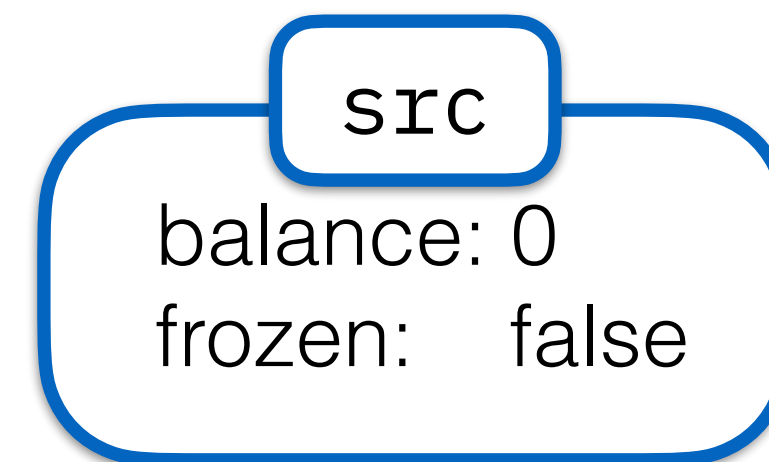
We are going to specify that design

BoC: Cowns and Behaviours

```
def main(src: cown[Account], dst: cown[Account]) {  
    when(src) { A src.balance += 10; }  
    when(dst) { B dst.frozen = true; }  
}
```

BoC: Cowns and Behaviours

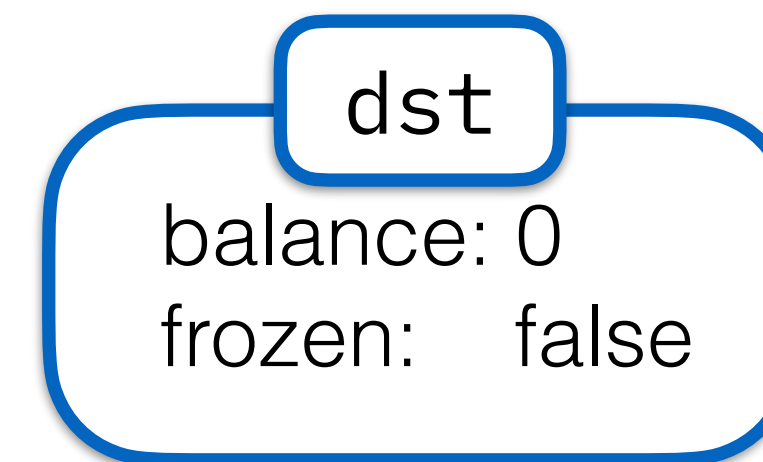
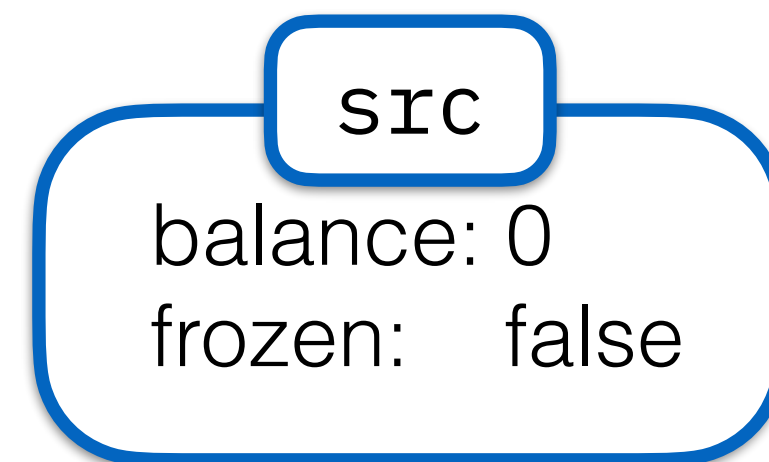
```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A src.balance += 10; }  
  when(dst) { B dst.frozen = true; }  
}
```



Cowns are isolated resources

BoC: Cowns and Behaviours

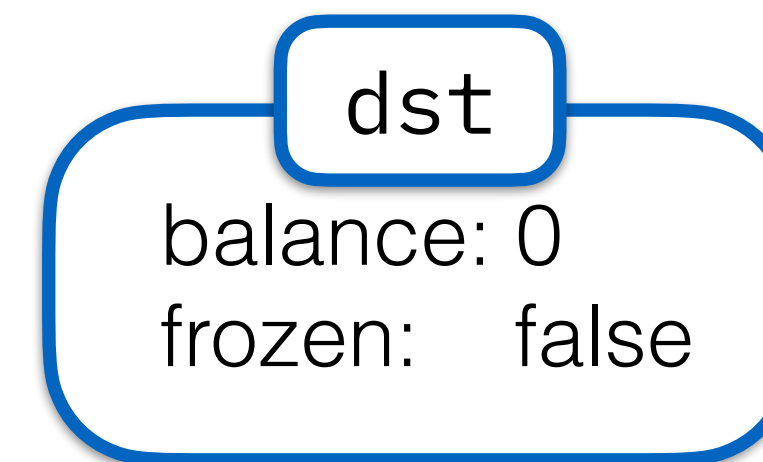
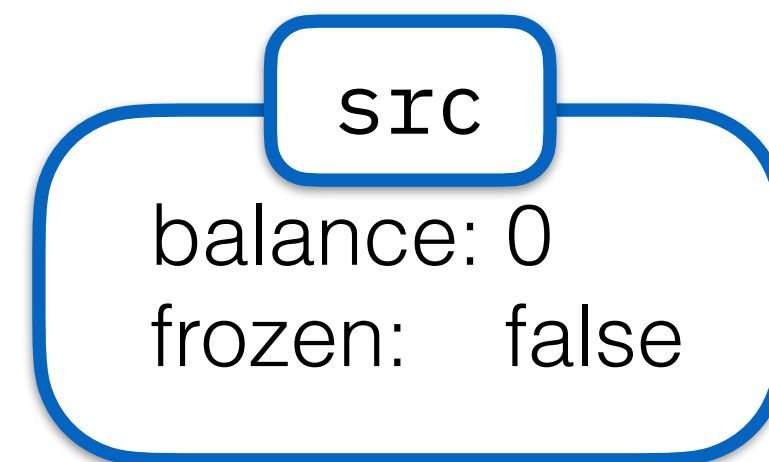
```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A src.balance += 10; }  
  when(dst) { B dst.frozen = true; }  
}
```



Behaviours are concurrent tasks which access cowns

BoC: Cowns and Behaviours

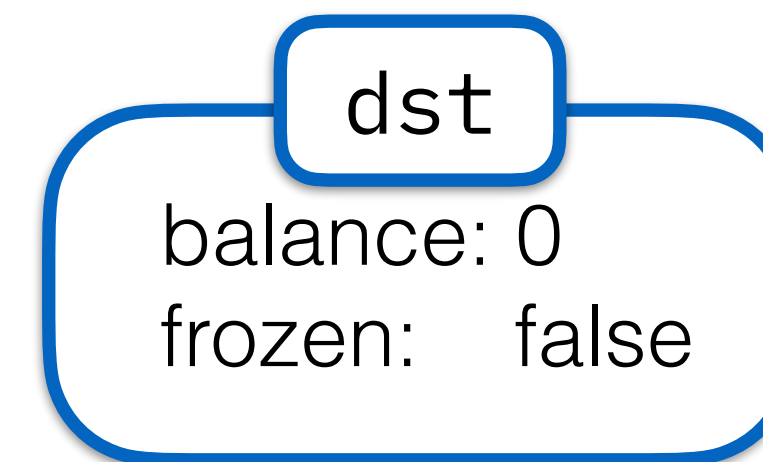
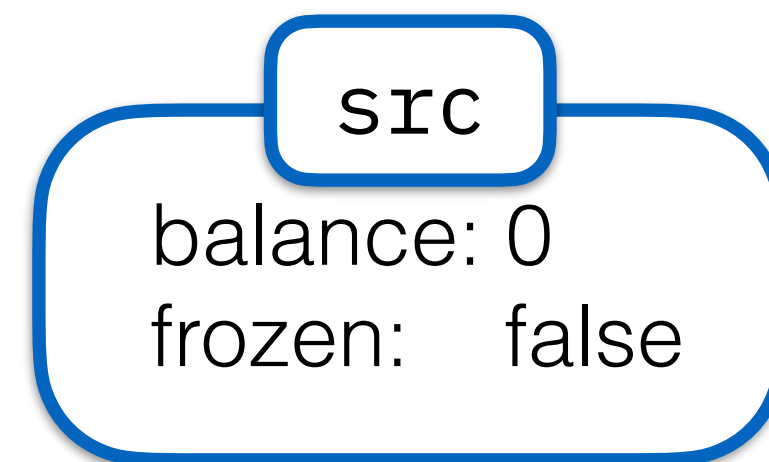
```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A src.balance += 10; }  
  when(dst) { B dst.frozen = true; }  
}
```



when spawns a behaviour and denotes the set of cowns it requires

BoC: Cowns and Behaviours

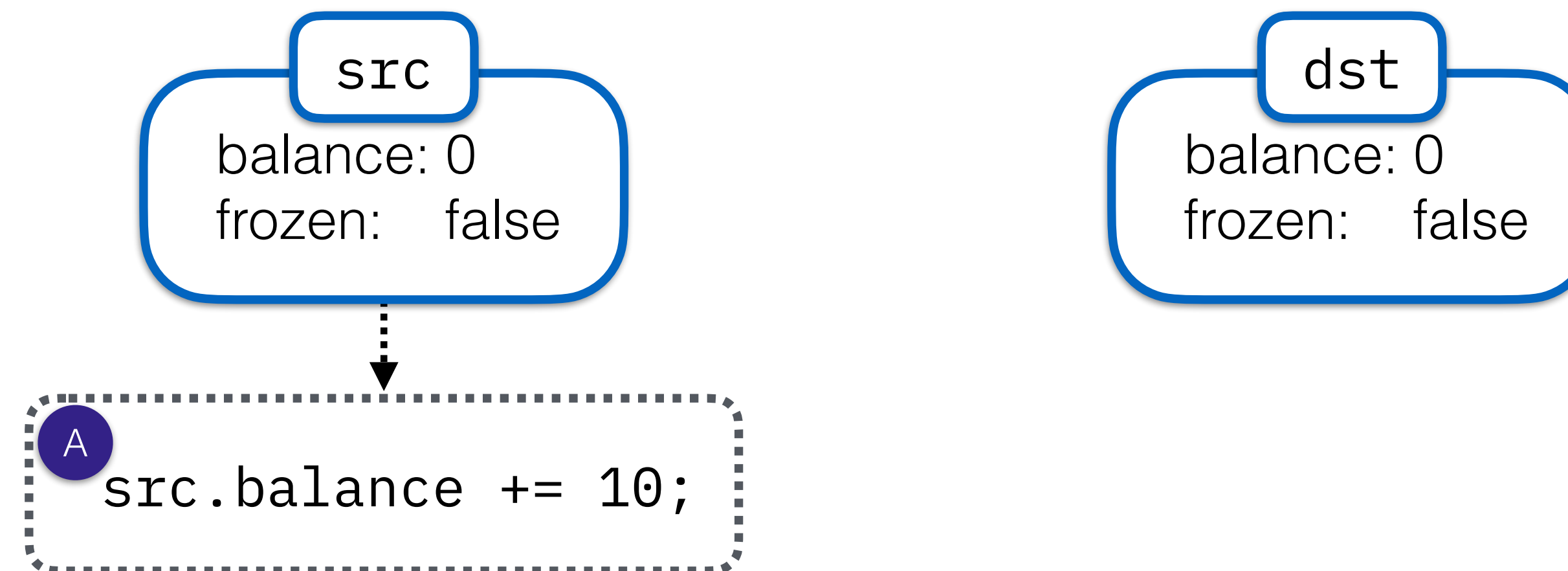
```
def main(src: cown[Account], dst: cown[Account]) {  
➔ when(src) { A src.balance += 10; }  
  when(dst) { B dst.frozen = true; }  
}
```



when spawns a behaviour and denotes the set of cowns it requires

BoC: Cowns and Behaviours

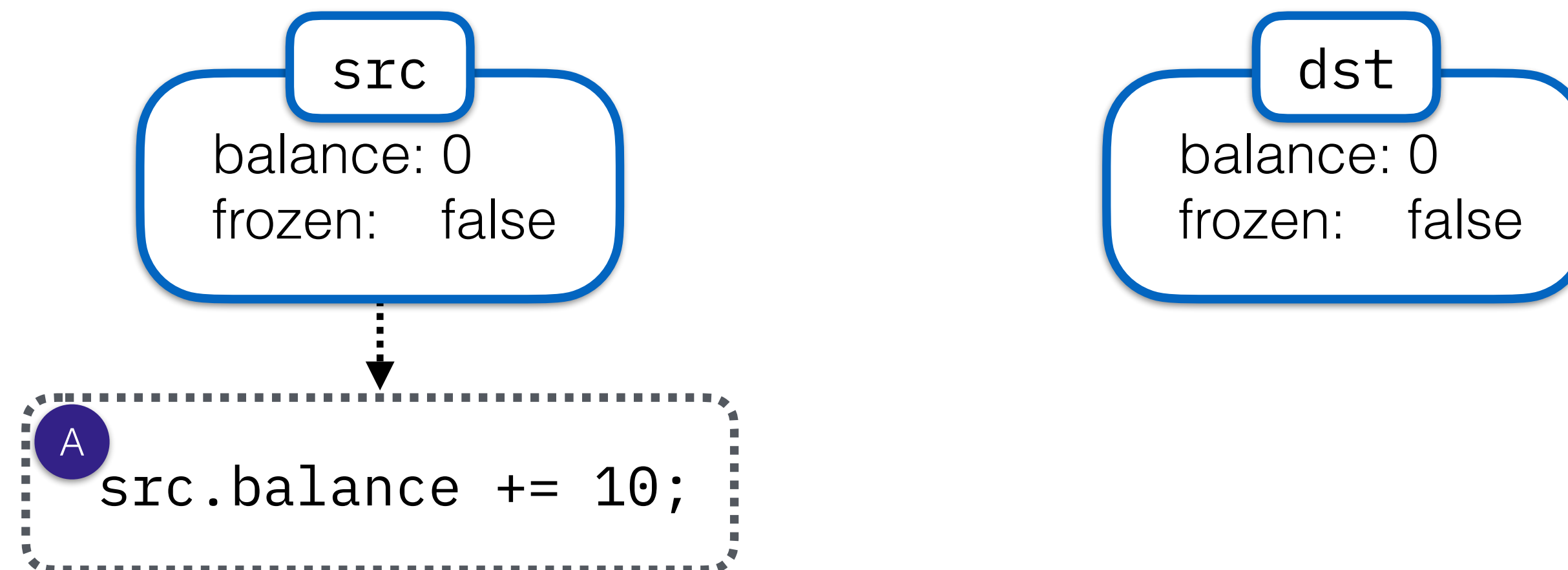
```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A src.balance += 10; }  
➔ when(dst) { B dst.frozen = true; }  
}
```



when spawns a behaviour and denotes the set of cowns it requires

BoC: Cowns and Behaviours

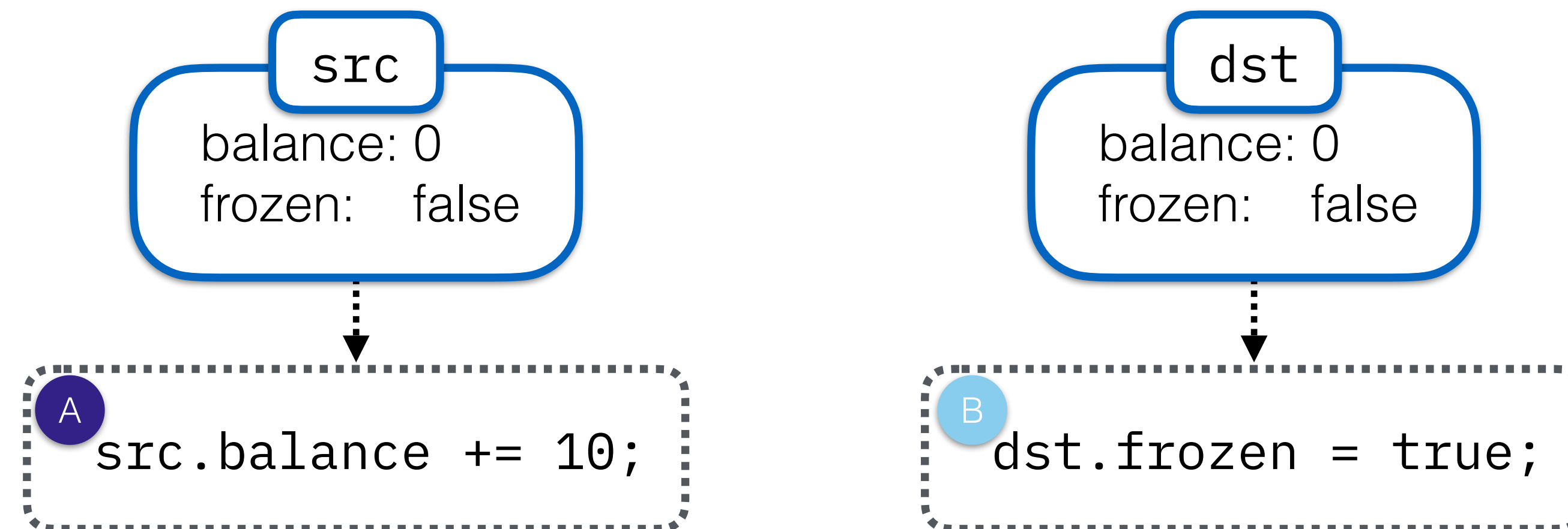
```
def main(src: cown[Account], dst: cown[Account]) {  
    when(src) { A src.balance += 10; }  
    ➔ when(dst) { B dst.frozen = true; }  
}
```



Spawning is synchronous but executing is asynchronous

BoC: Cowns and Behaviours

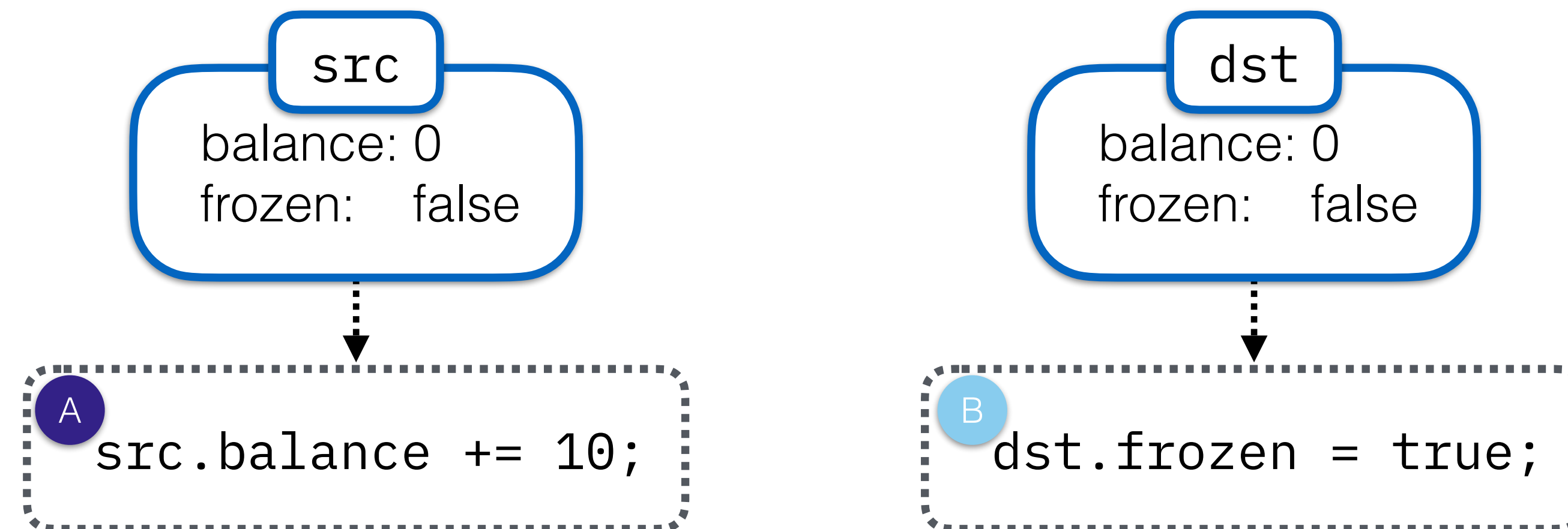
```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A src.balance += 10; }  
  when(dst) { B dst.frozen = true; }  
}
```



Spawning is synchronous but executing is asynchronous

BoC: Cowns and Behaviours

```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A src.balance += 10; }  
  when(dst) { B dst.frozen = true; }  
}
```

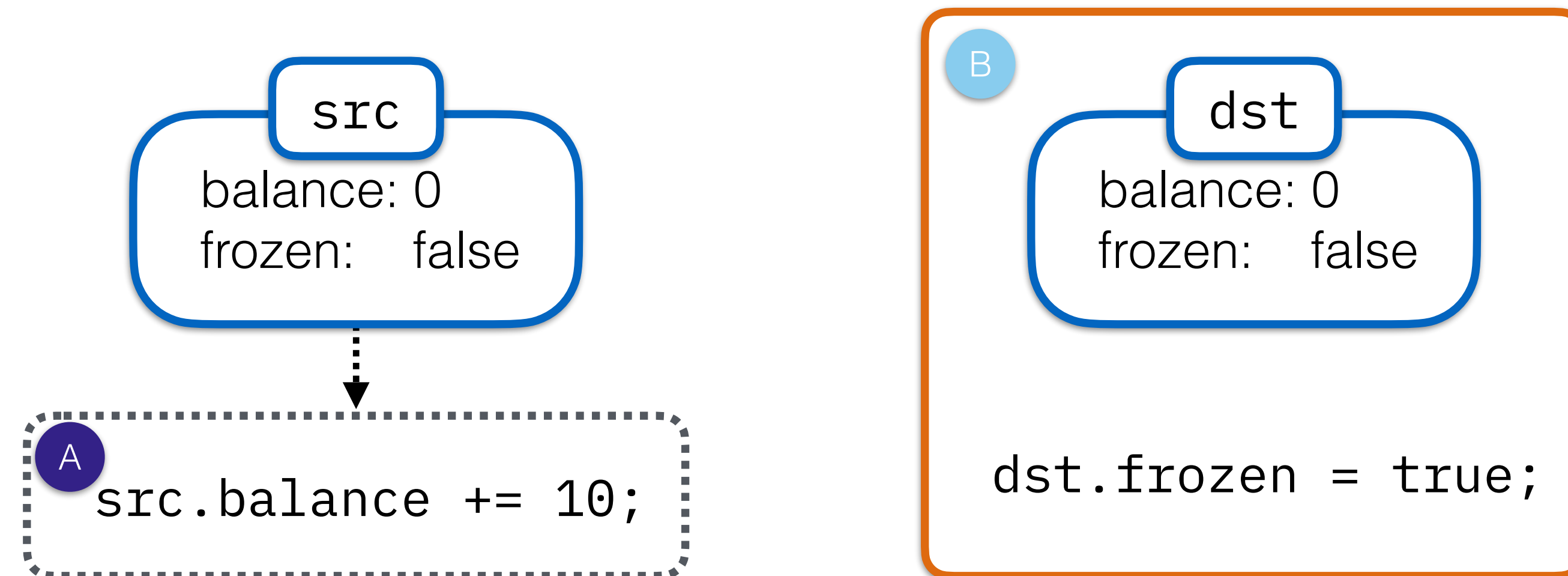


Spawning is synchronous but executing is asynchronous

When the cowns are available the behaviours can execute with access to the cown's state

BoC: Cowns and Behaviours

```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A src.balance += 10; }  
  when(dst) { B dst.frozen = true; }  
}
```

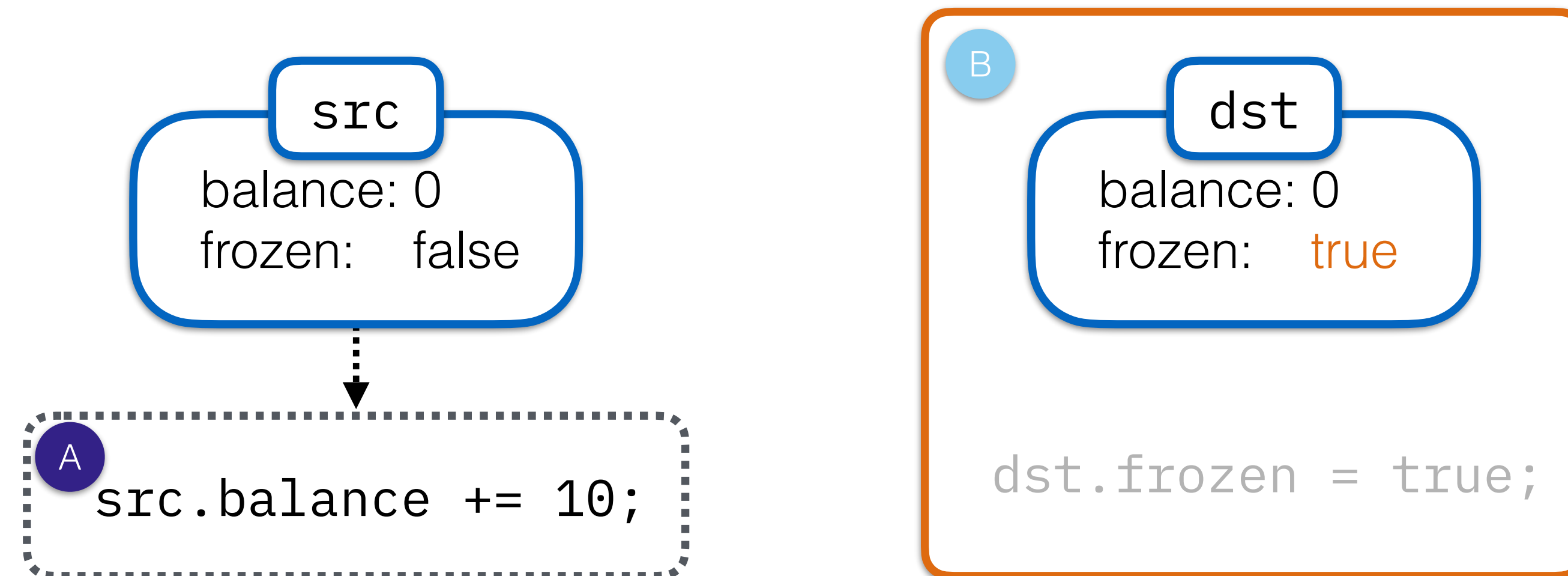


Spawning is synchronous but executing is asynchronous

When the cowns are available the behaviours can execute with access to the cown's state

BoC: Cowns and Behaviours

```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A src.balance += 10; }  
  when(dst) { B dst.frozen = true; }  
}
```

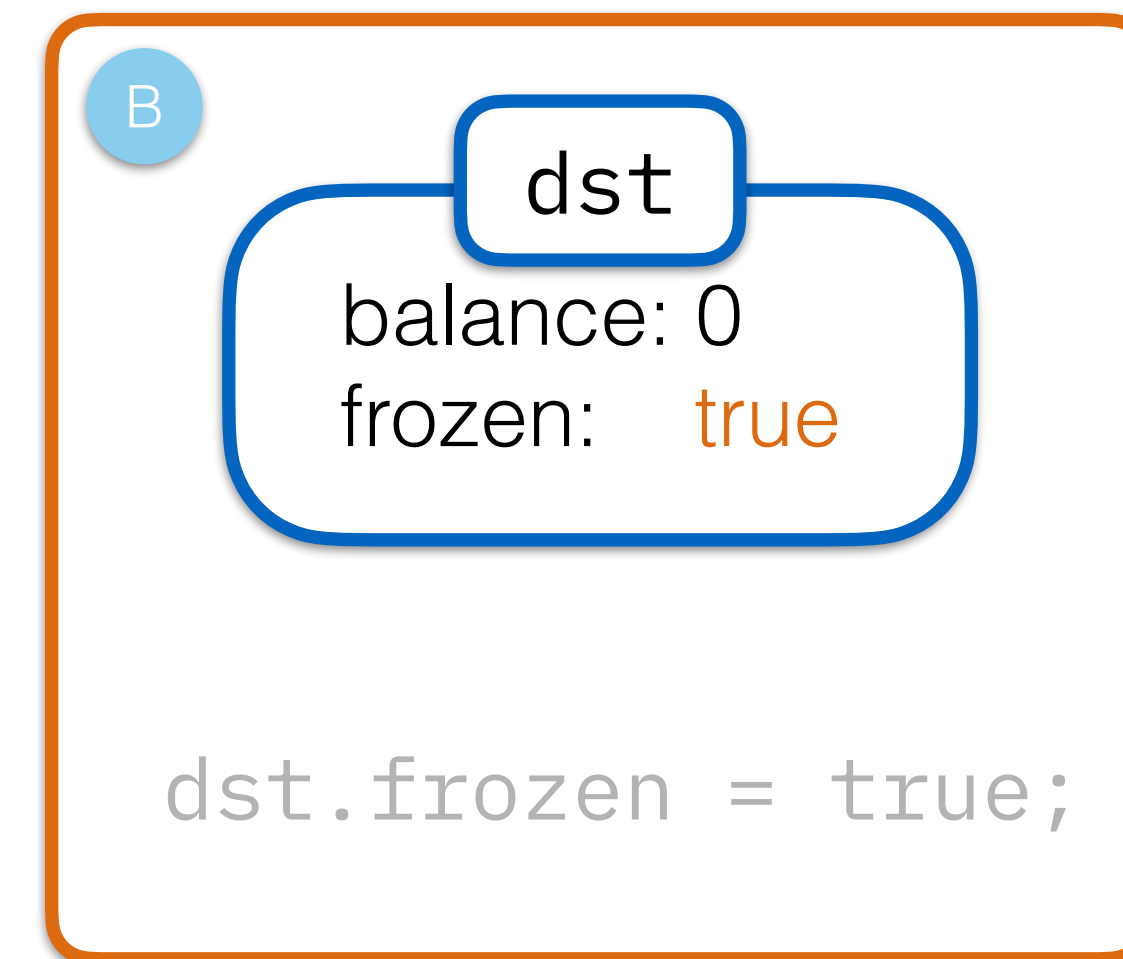
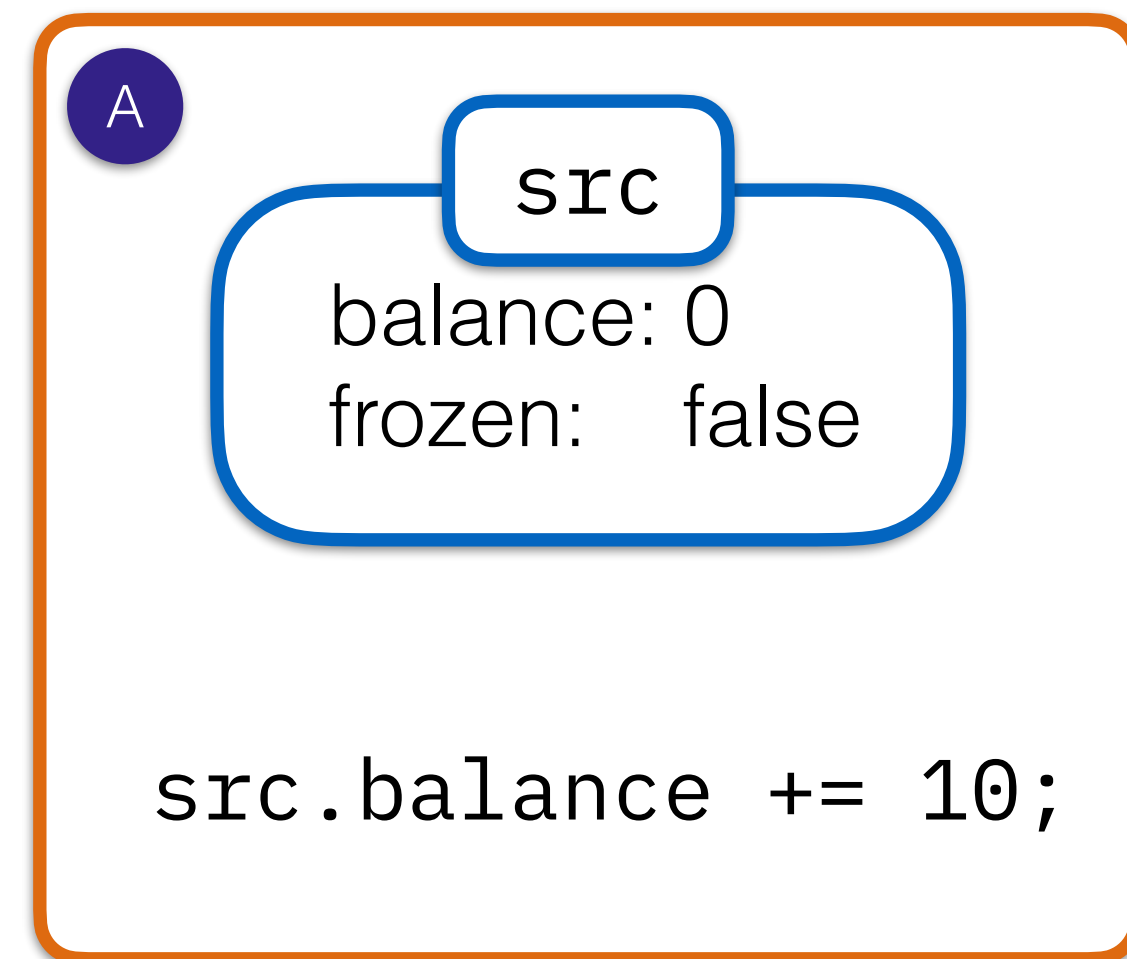


Spawning is synchronous but executing is asynchronous

When the cowns are available the behaviours can execute with access to the cown's state

BoC: Cowns and Behaviours

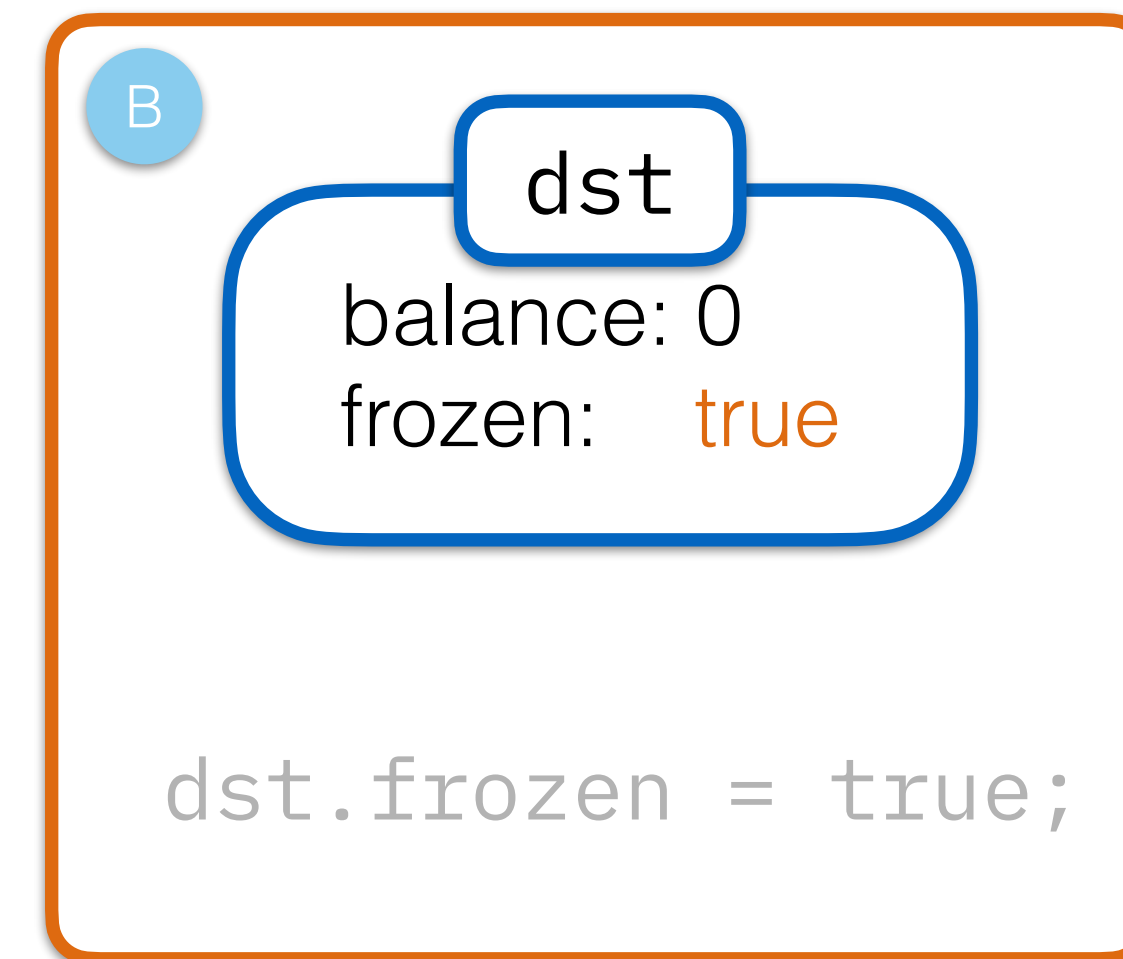
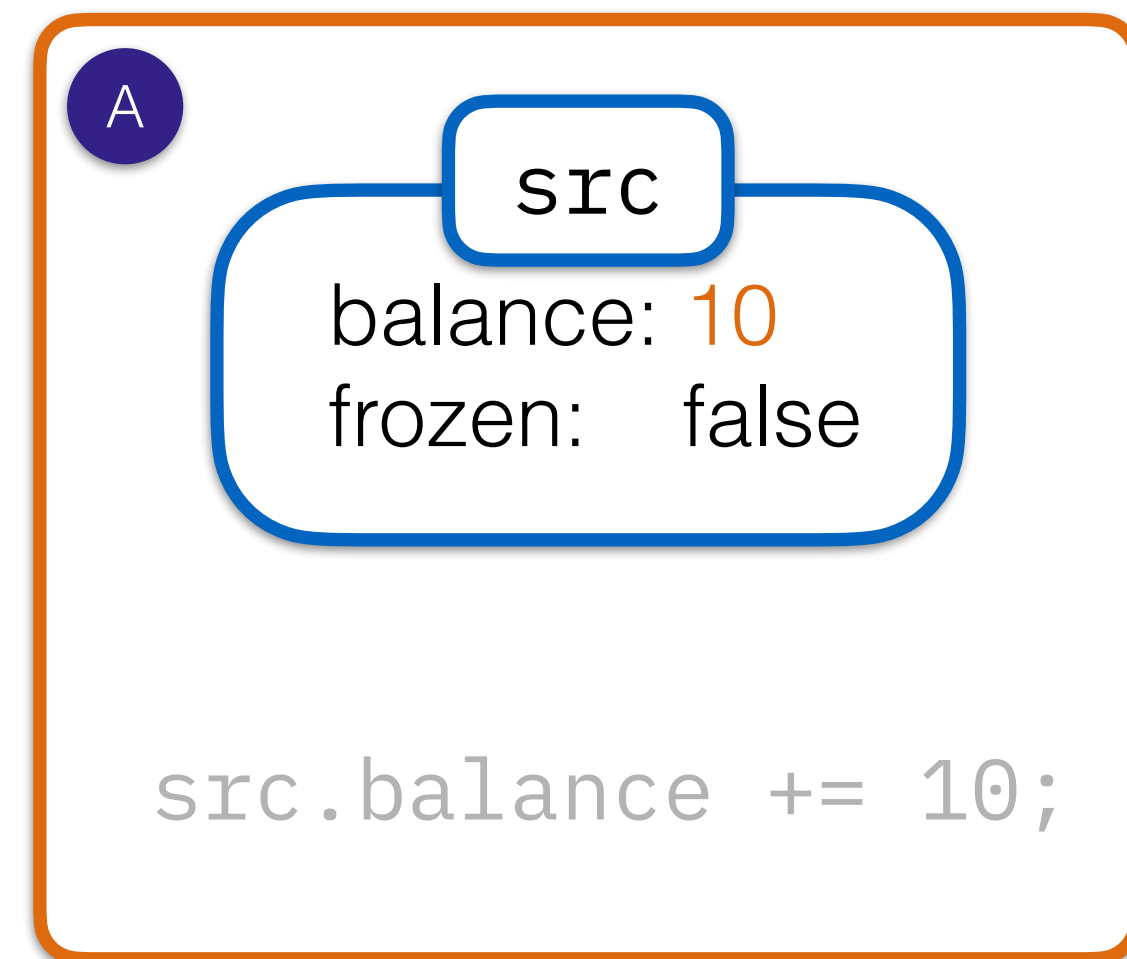
```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A src.balance += 10; }  
  when(dst) { B dst.frozen = true; }  
}
```



Behaviours which require different cowns can run concurrently

BoC: Cowns and Behaviours

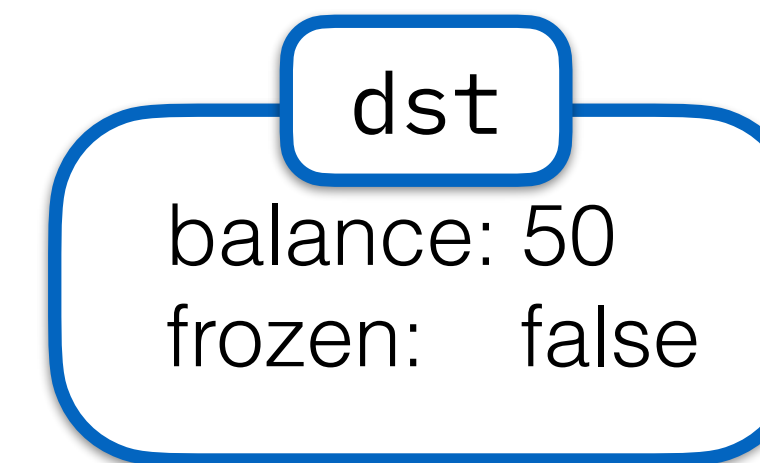
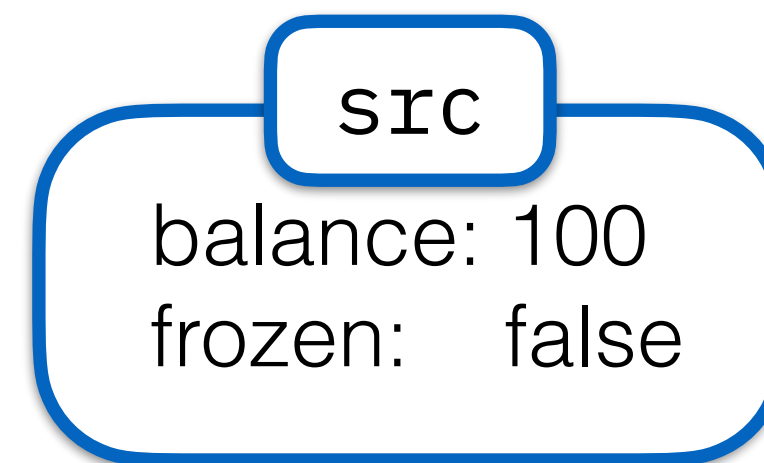
```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A src.balance += 10; }  
  when(dst) { B dst.frozen = true; }  
}
```



Behaviours which require different cowns can run concurrently

BoC: Behaviours Can Use Multiple Cowns

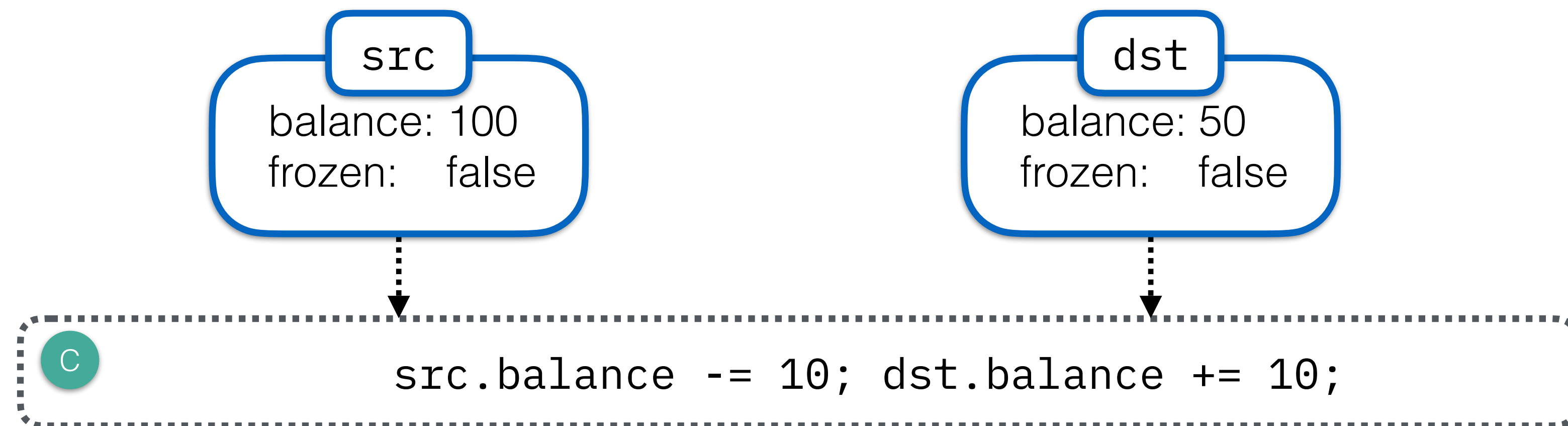
```
def main(src: cown[Account], dst: cown[Account]) {  
➔ when(src, dst) { c  
    src.balance -= 10; dst.balance += 10;  
}
```



Behaviours can require and access multiple cowns at once

BoC: Behaviours Can Use Multiple Cowns

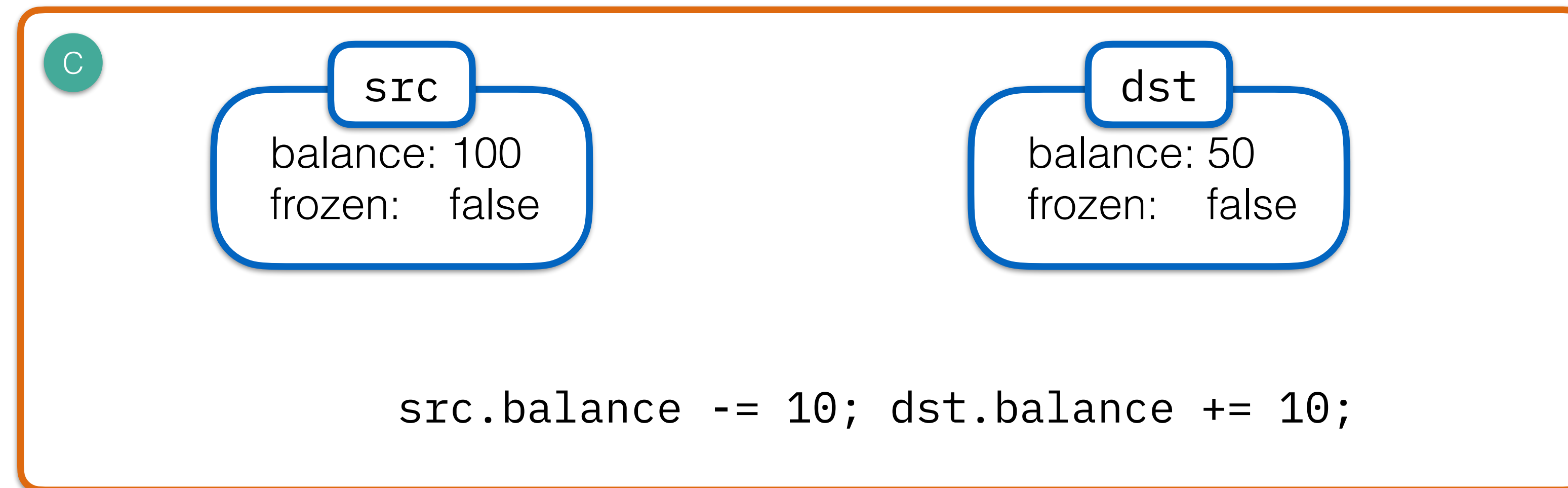
```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src, dst) { C  
    src.balance -= 10; dst.balance += 10;  
  }  
}
```



Behaviours can require and access multiple cowns at once

BoC: Behaviours Can Use Multiple Cowns

```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src, dst) { C  
    src.balance -= 10; dst.balance += 10;  
  }  
}
```



Behaviours can require and access multiple cowns at once

BoC: Behaviours Can Use Multiple Cowns

```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src, dst) { C  
    src.balance -= 10; dst.balance += 10;  
  }  
}
```



Behaviours can require and access multiple cowns at once

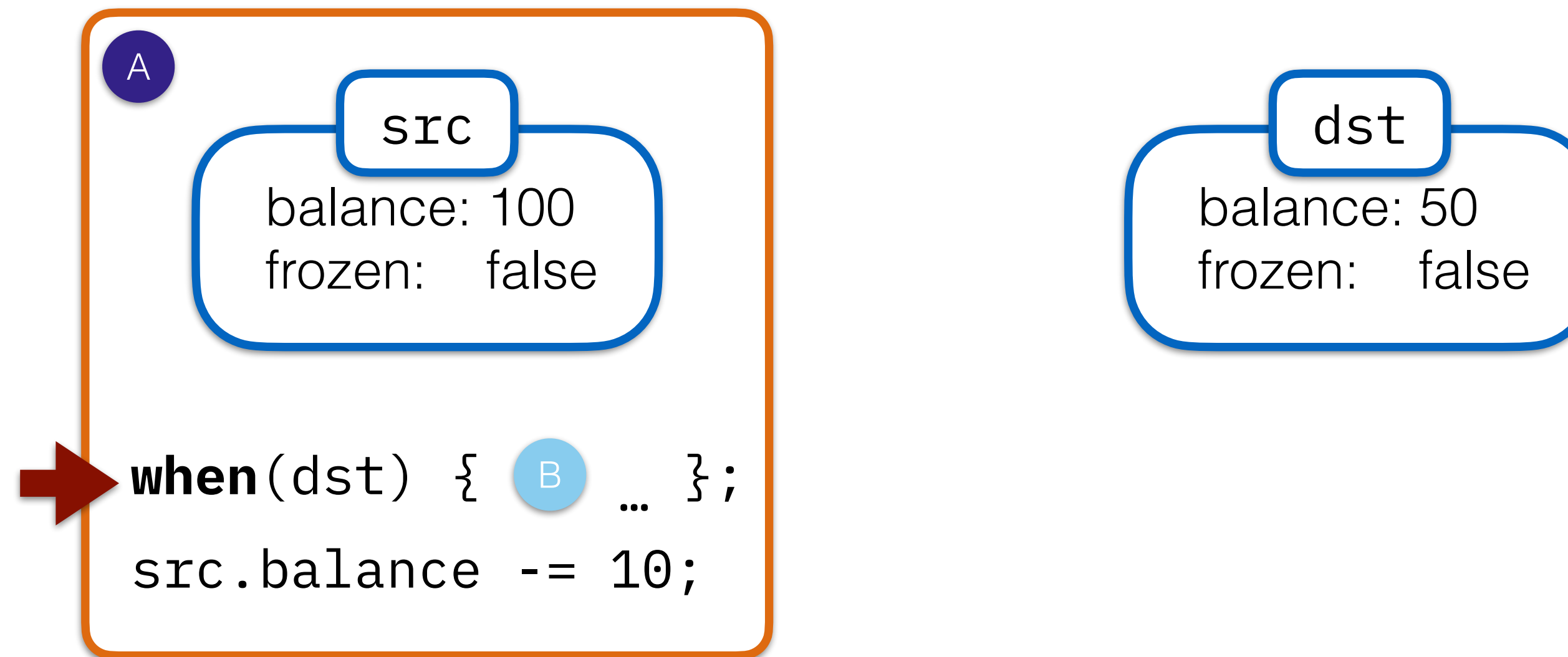
BoC: Behaviours Can Nest

```
def main(src: cown[Account], dst: cown[Account]) {  
➡ when(src) { A  
    when(dst) { B dst.balance += 10; }  
    src.balance -= 10;  
} }
```

Behaviours can nest

BoC: Behaviours Can Nest

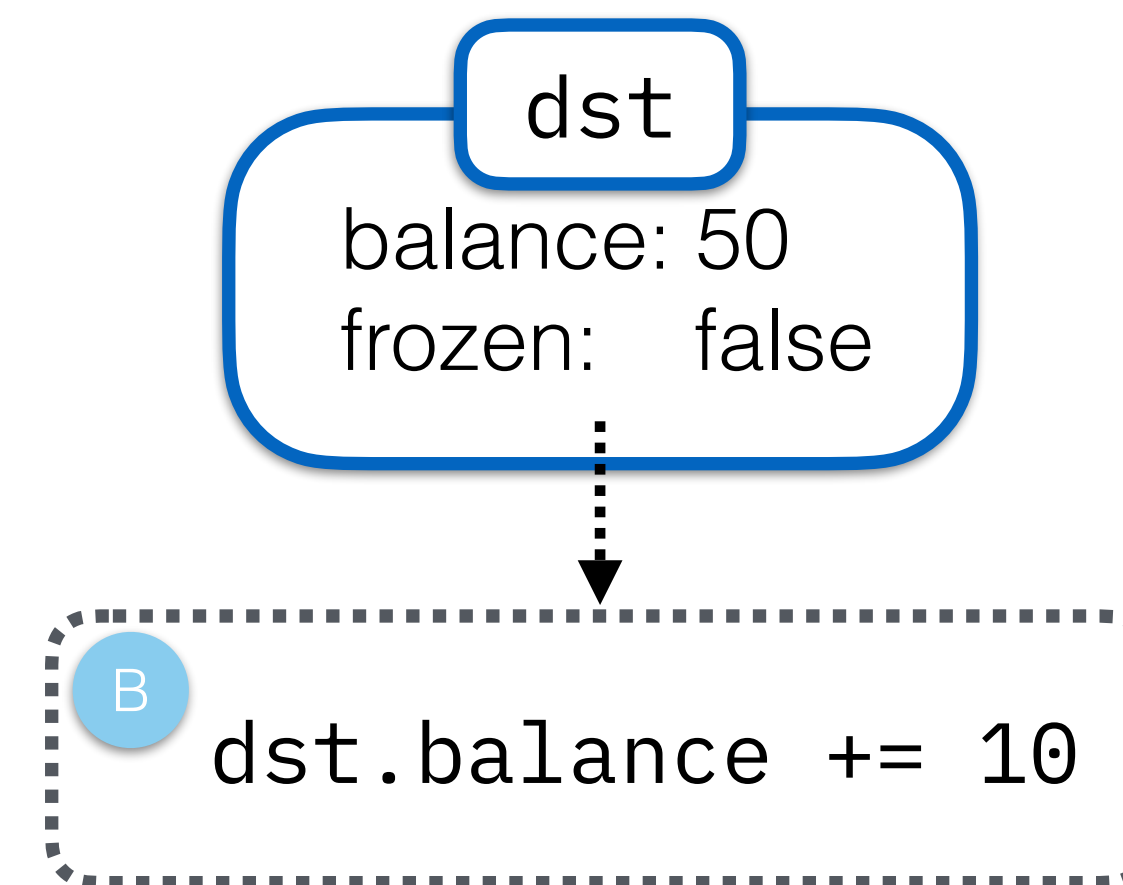
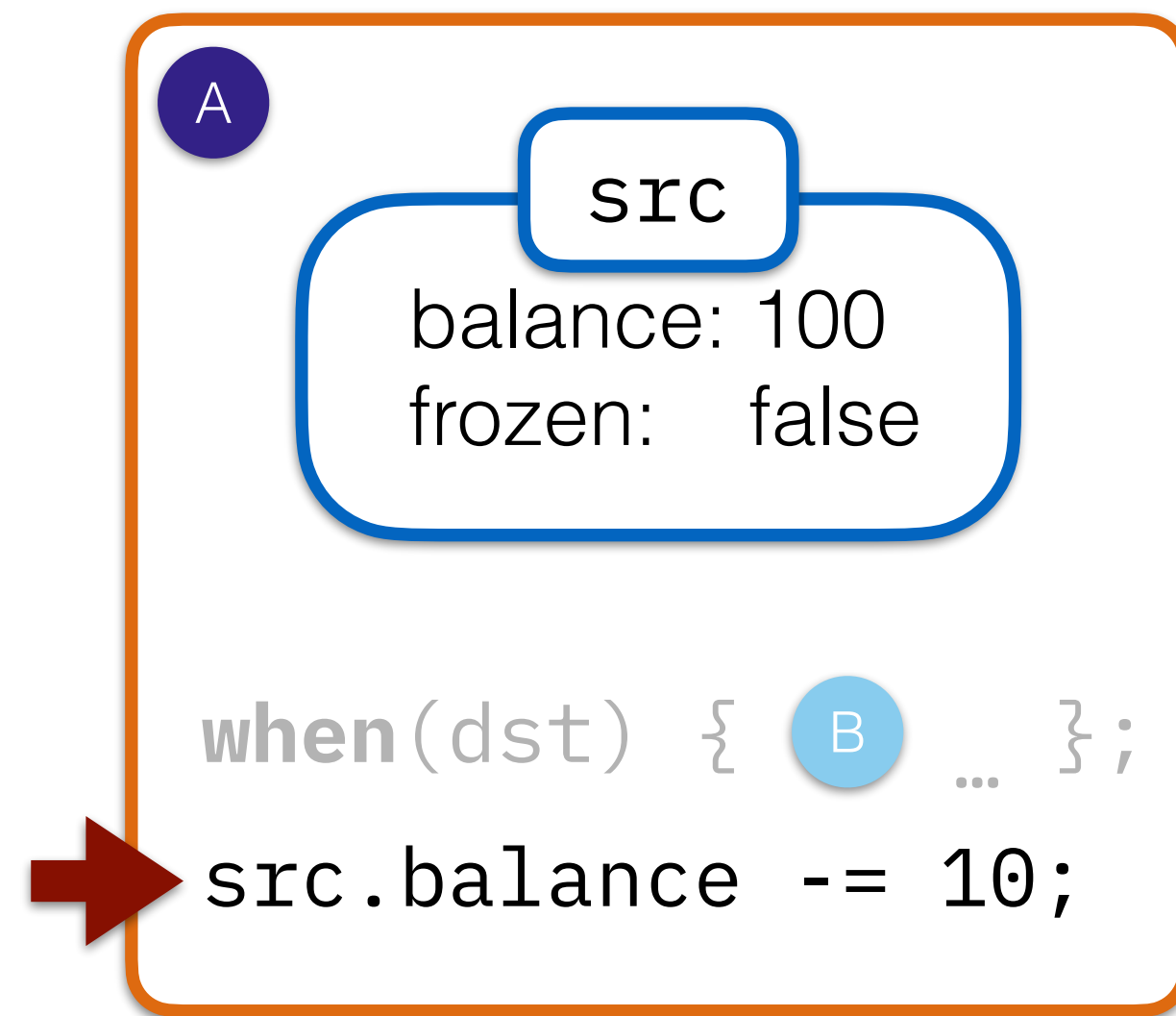
```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A  
    when(dst) { B dst.balance += 10; }  
    src.balance -= 10;  
  }  
}
```



Behaviours can nest

BoC: Behaviours Can Nest

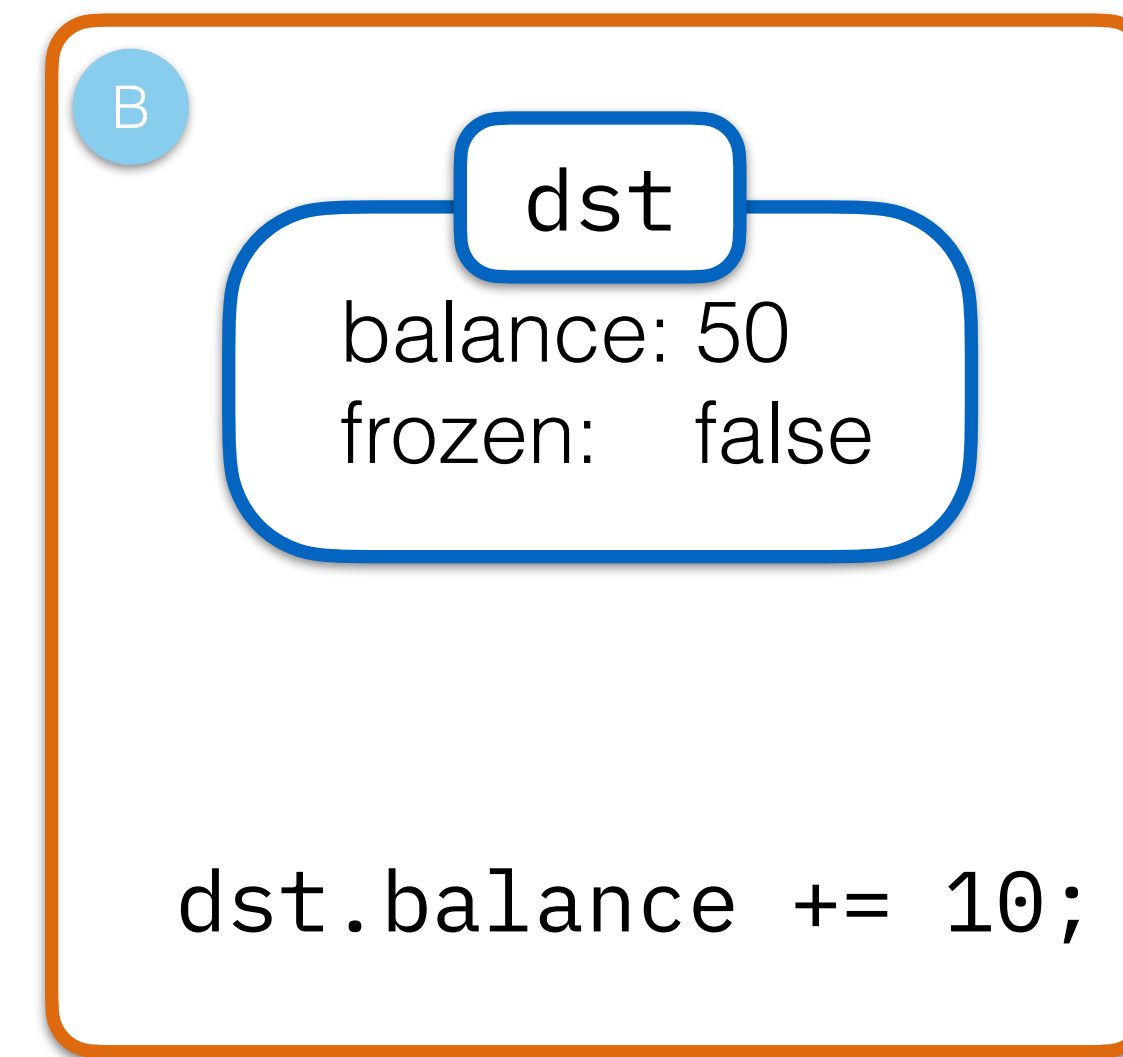
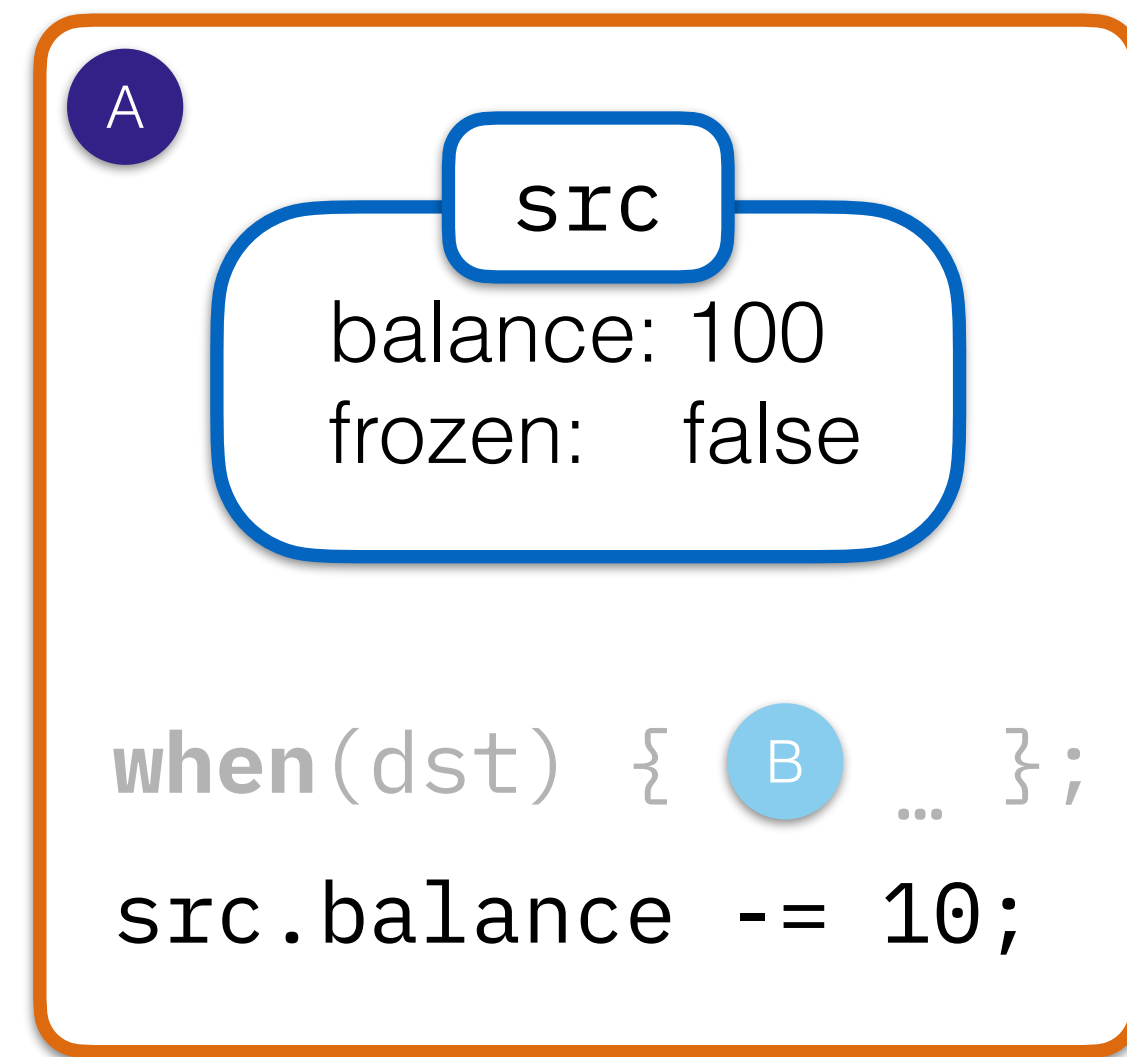
```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A  
    when(dst) { B dst.balance += 10; }  
    src.balance -= 10;  
  }  
}
```



Behaviours can nest

BoC: Behaviours Can Nest

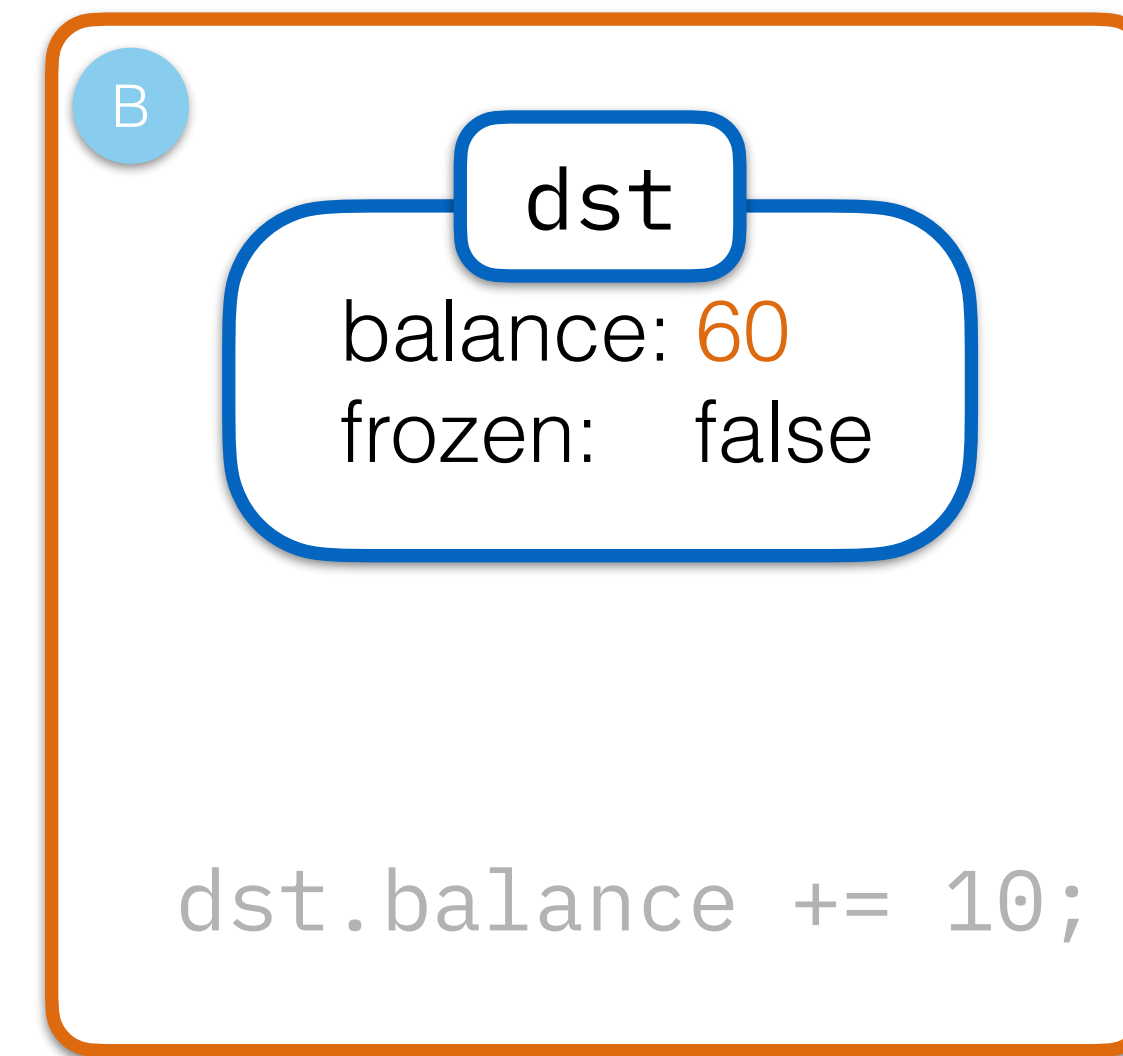
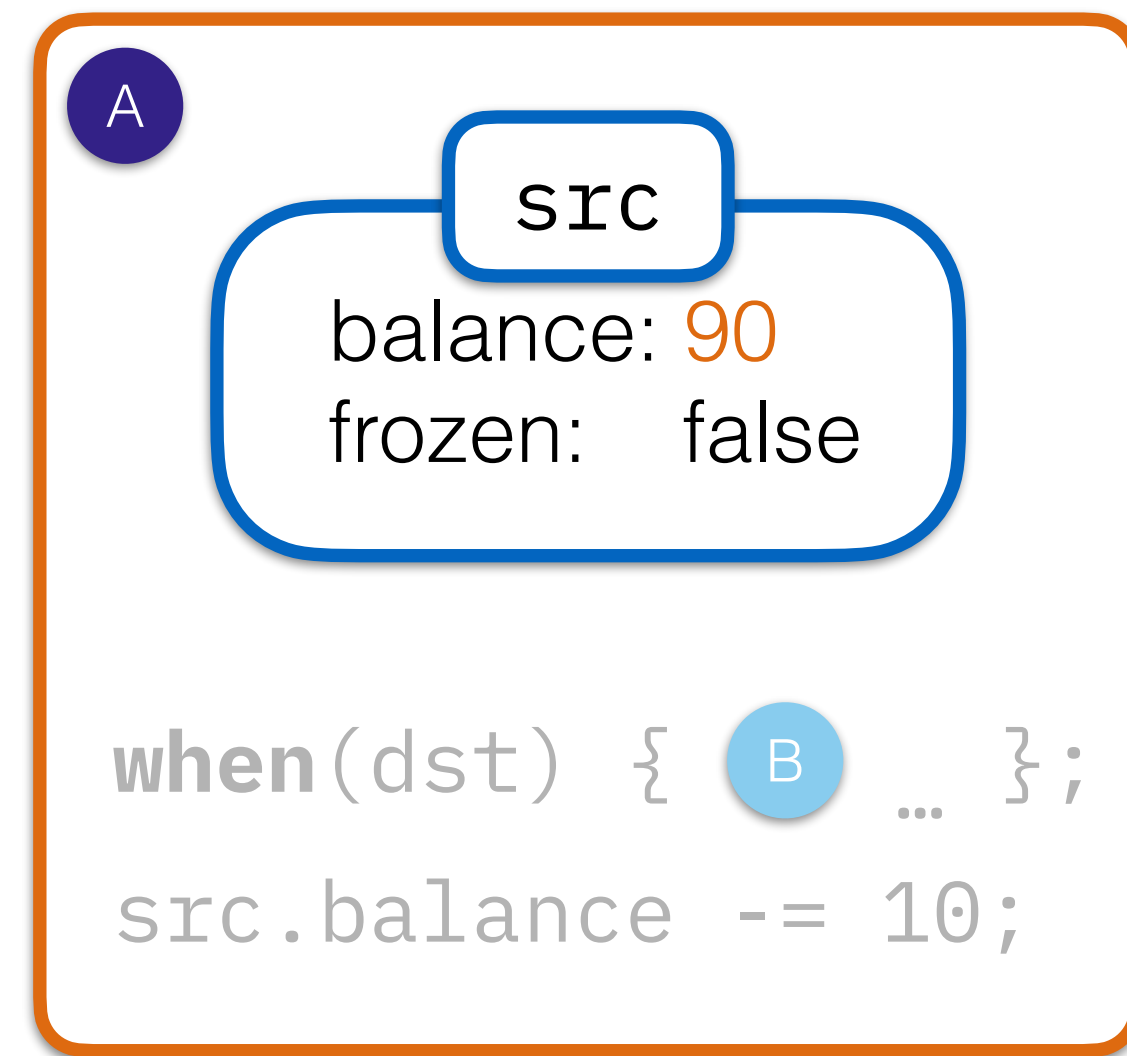
```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A  
    when(dst) { B dst.balance += 10; }  
    src.balance -= 10;  
  }  
}
```



Behaviours can nest

BoC: Behaviours Can Nest

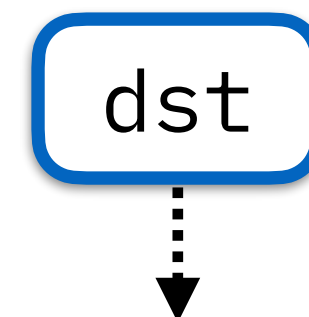
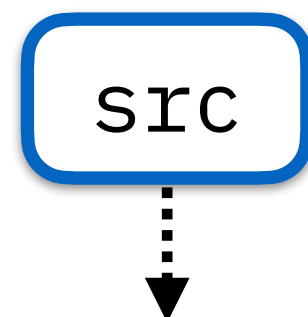
```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A  
    when(dst) { B dst.balance += 10; }  
    src.balance -= 10;  
  }  
}
```



Behaviours can nest

BoC: Behaviours are Ordered

```
def main(src: cown[Account], dst: cown[Account]) {  
➔ when(src) { A src.balance += 10; }  
  when(dst) { B dst.frozen = true; }  
  when(src, dst) { C  
    src.balance -= 10; dst.balance += 10;  
  } }
```



Behaviours are executed with respect to the order they were spawned

BoC: Behaviours are Ordered

```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A src.balance += 10; }  
  when(dst) { B dst.frozen = true; }  
  when(src, dst) { C  
    src.balance -= 10; dst.balance += 10;  
  }  
}
```



Behaviours are executed with respect to the order they were spawned

BoC: Behaviours are Ordered

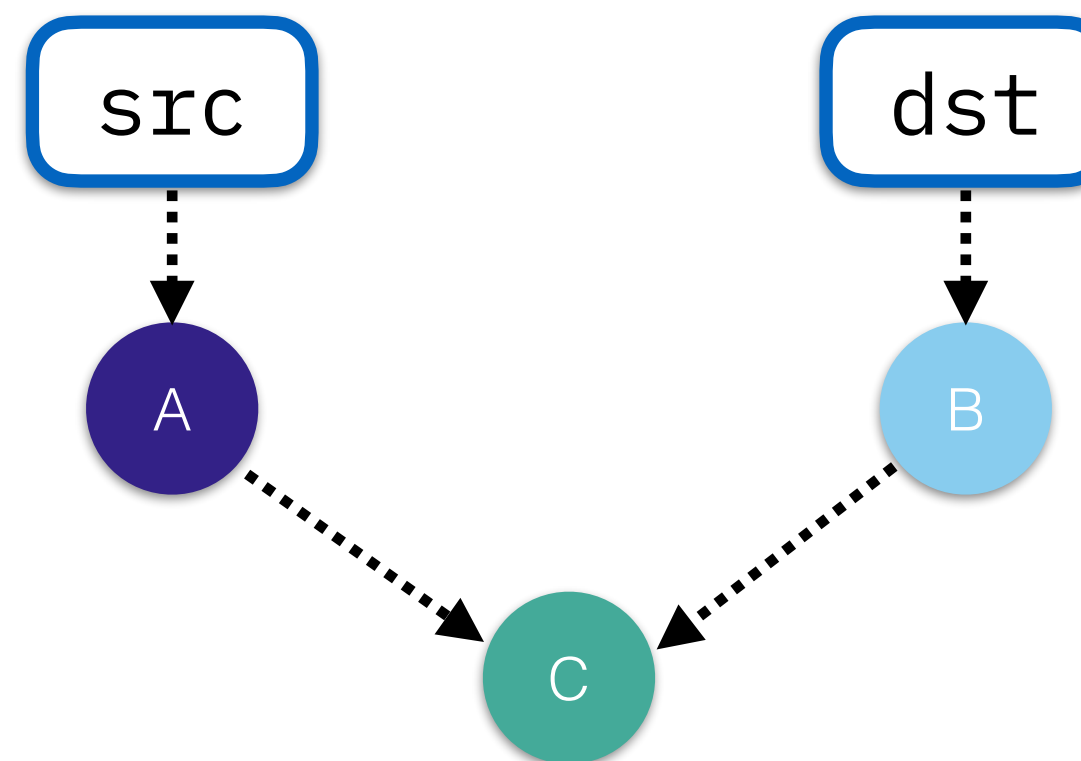
```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A src.balance += 10; }  
  when(dst) { B dst.frozen = true; }  
  ➔ when(src, dst) { C  
    src.balance -= 10; dst.balance += 10;  
  } }
```



Behaviours are executed with respect to the order they were spawned

BoC: Behaviours are Ordered

```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A src.balance += 10; }  
  when(dst) { B dst.frozen = true; }  
  when(src, dst) { C  
    src.balance -= 10; dst.balance += 10;  
  }  
}
```



Behaviours are executed with respect to the order they were spawned



What should this program be able to do?

What should this program be able to do?

```
when(src) { A src.balance += 10; }  
when(dst) { B dst.balance += 10; }
```

```
when(dst) { C print(dst.balance) }  
when(src) { D print(src.balance) }
```

What should this program be able to do?

when(src) { **A** src.balance += 10; }

when(dst) { **B** dst.balance += 10; }

when(dst) { **C** print(dst.balance) }

when(src) { **D** print(src.balance) }

How can this program execute?

```
> ./program
```

```
>
```


What should this program be able to do?

```
when(src) { A src.balance += 10; }  
when(dst) { B dst.balance += 10; }
```

src

```
when(dst) { C print(dst.balance) }  
when(src) { D print(src.balance) }
```

dst

How can this program execute?

```
> ./program  
>
```

What should this program be able to do?

➔ **when**(src) { A src.balance += 10; }
when(dst) { B dst.balance += 10; }

when(dst) { C print(dst.balance) }
when(src) { D print(src.balance) }

src

dst

How can this program execute?

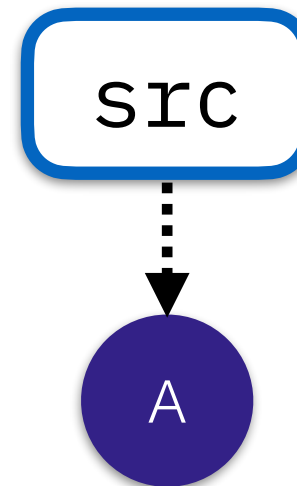
```
> ./program  
>
```

What should this program be able to do?

➔ **when**(dst) { **B** dst.balance += 10; }

when(dst) { **C** print(dst.balance) }

when(src) { **D** print(src.balance) }



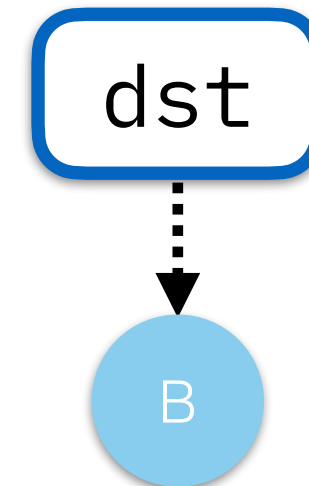
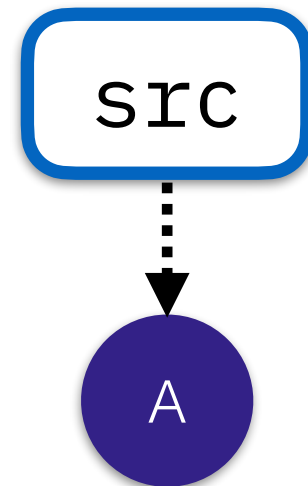
How can this program execute?

```
> ./program
```

```
>
```


What should this program be able to do?

→ **when**(dst) { C print(dst.balance) }
when(src) { D print(src.balance) }



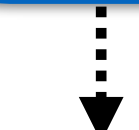
How can this program execute?

```
> ./program  
>
```

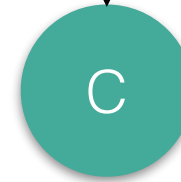
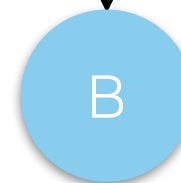
What should this program be able to do?

➔ **when**(src) { **D** print(src.balance) }

src



dst



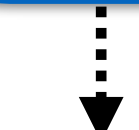
How can this program execute?

```
> ./program
```

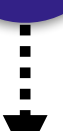
```
>
```

What should this program be able to do?

src

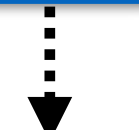


A



D

dst



B



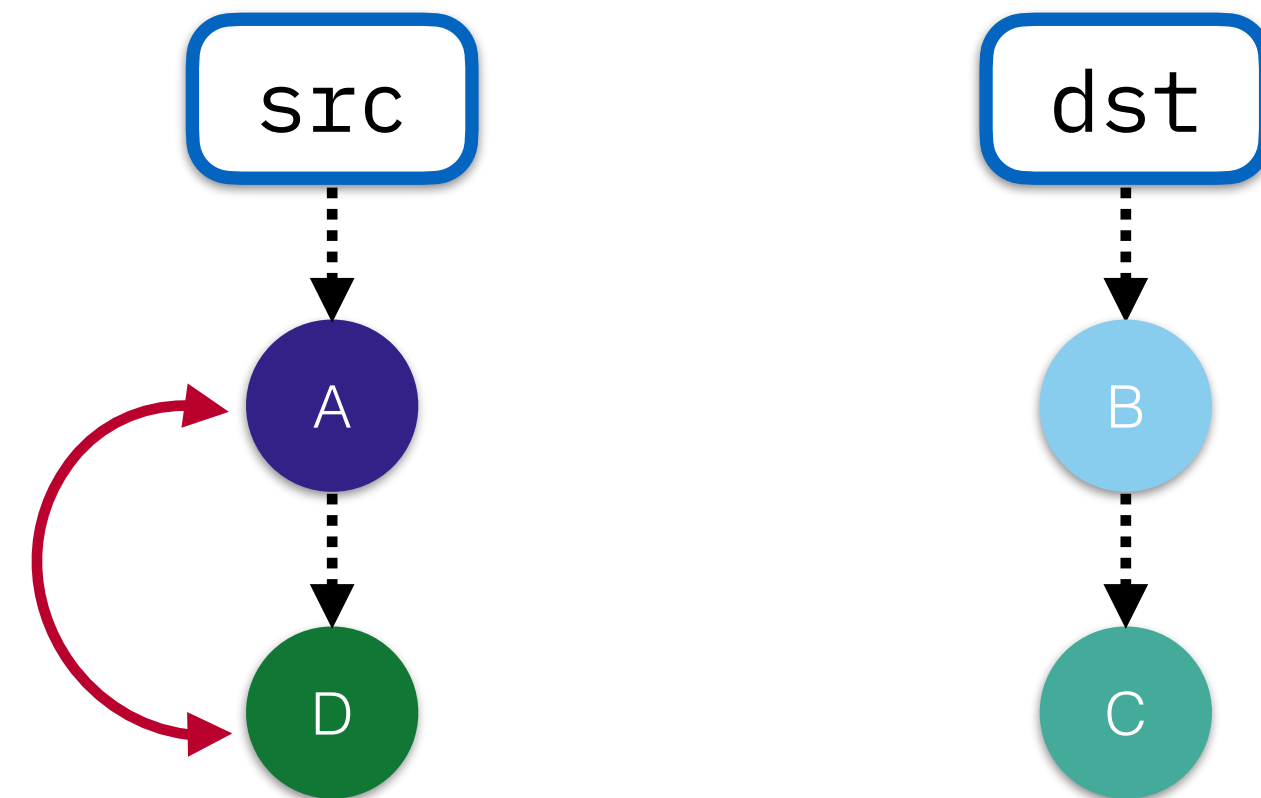
C

How can this program execute?

```
> ./program  
> src=10, dst=10
```



What should this program be able to do?



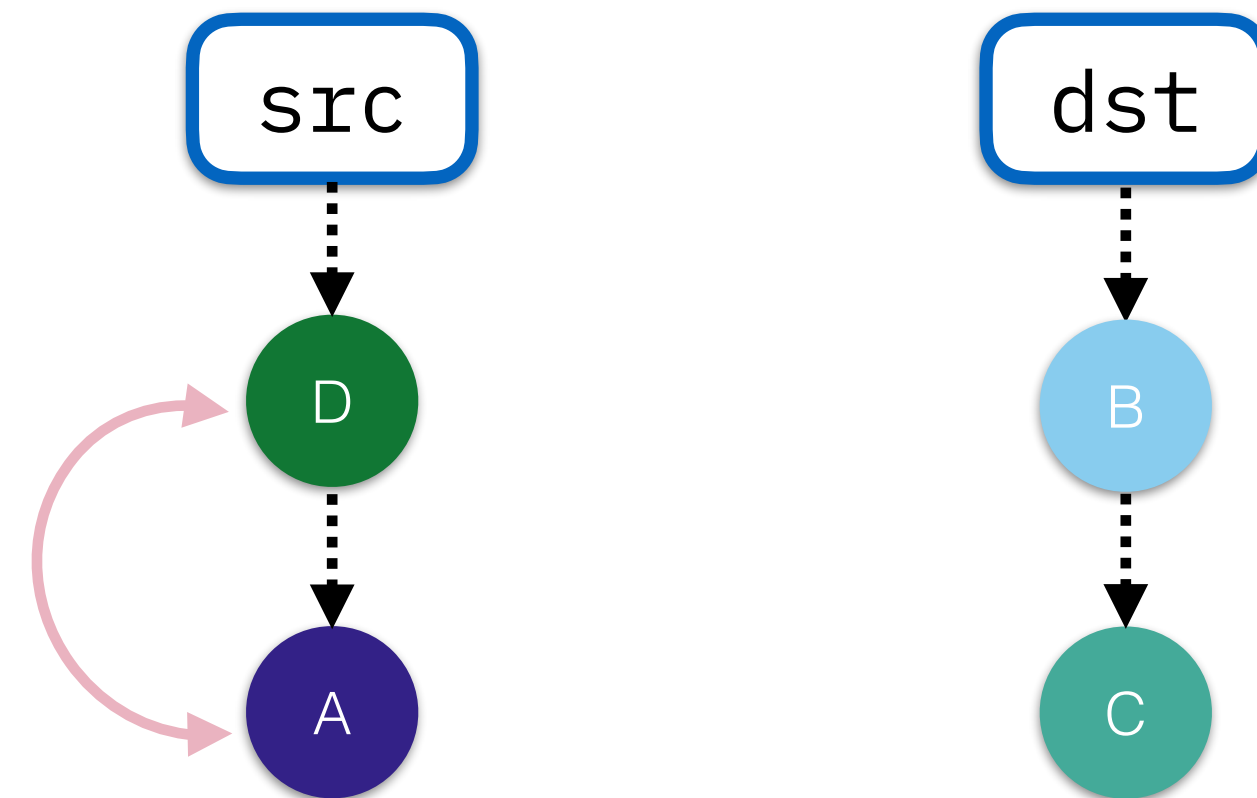
How can this program execute?

Is it possible for **D** then **A** ?

```
> ./program  
> src=10, dst=10
```



What should this program be able to do?



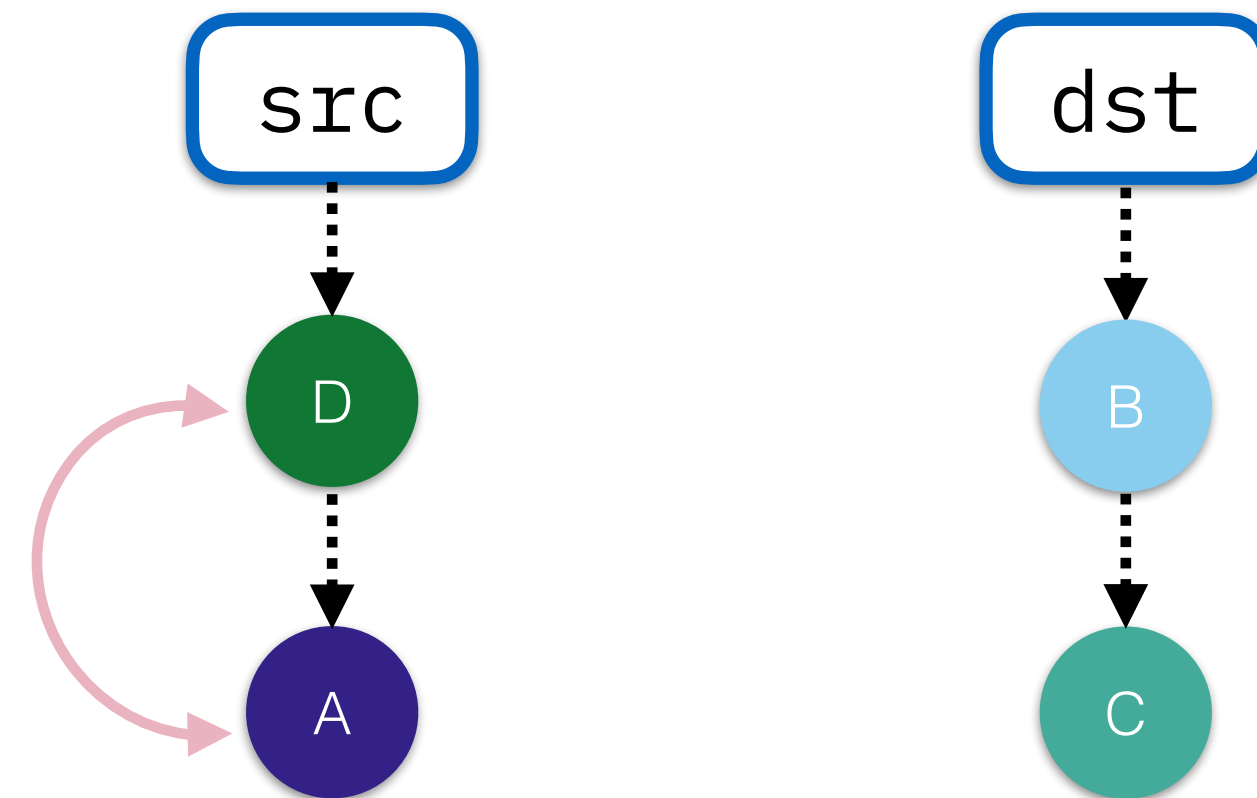
How can this program execute?

Is it possible for **D** then **A** ?

```
> ./program  
> src=0, dst=10
```

?

What should this program be able to do?



How can this program execute?

Is it possible for **D** then **A** ?

```
> ./program  
> src=0, dst=10
```



This outcome should be fine but it is not possible under the operational semantics

What should this program be able to do?

```
when(src) { A src.balance += 10; }  
when(dst) { B dst.balance += 10; }
```

```
when(dst) { C print(dst.balance) }  
when(src) { D print(src.balance) }
```

Order

Permitted

What should this program be able to do?

```
when(src) { A src.balance += 10; }  
when(dst) { B dst.balance += 10; }
```

```
when(dst) { C print(dst.balance) }  
when(src) { D print(src.balance) }
```

	src=0, dst=0	src=0, dst=10	src=10, dst=0	src=10, dst=10
Order	C B D A C D A B C D B A D A C B D C A B D C B A	B C D A B D A C B D C A D A B C D A B C D B C A	A C B D A C D B A D C B C A B D C A D B C B A D	A B C D A B D C A D B C B A C D B A D C B C A D
Permitted	✓	⚠	✓	✓



Some orders get in the way



Some orders get in the way

Can we transform BoC programs to enable more concurrency and improve performance?

Some orders get in the way

```
when(src) {  src.balance += 10; }  
when(dst) {  dst.balance += 10; }
```

```
when(dst) {  print(dst.balance) }  
when(src) {  print(src.balance) }
```

Can we transform BoC programs to enable more concurrency and improve performance?

Some orders get in the way

```
when(src) {  src.balance += 10; }  
when(dst) {  dst.balance += 10; }
```

```
when(src) {  src.open(); }  
when(dst) {  print(dst.balance) }  
when(src) {  print(src.balance) }
```

Can we transform BoC programs to enable more concurrency and improve performance?

Some orders get in the way

```
when(src) { A src.balance += 10; }  
when(dst) { B dst.balance += 10; }
```

```
when(src) { E src.open(); }  
when(dst) { C print(dst.balance) }  
when(src) { D print(src.balance) }
```



Can we transform BoC programs to enable more concurrency and improve performance?

Some orders get in the way

```
when(src) {  src.balance += 10; }  
when(dst) {  dst.balance += 10; }
```

```
when(src) {  src.open(); }  
 when(src) {  print(src.balance) }  
 when(dst) {  print(dst.balance) }
```

Can we transform BoC programs to enable more concurrency and improve performance?

Some orders get in the way

```
when(src) { A src.balance += 10; }  
when(dst) { B dst.balance += 10; }
```

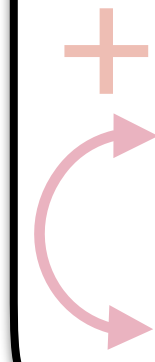
```
when(src) { E src.open(); }  
+  
↪ when(src) { D print(src.balance) }  
↪ when(dst) { C print(dst.balance) }
```

Can we transform BoC programs to enable more concurrency and improve performance?

Some orders get in the way

```
when(src) { A src.balance += 10; }  
when(dst) { B dst.balance += 10; }
```

```
when(src) { E src.open();  
           D print(src.balance) }  
when(dst) { C print(dst.balance) }
```

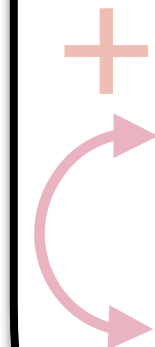


Can we transform BoC programs to enable more concurrency and improve performance?

Some orders get in the way

```
when(src) { A src.balance += 10; }  
when(dst) { B dst.balance += 10; }
```

```
when(src) { E src.open();  
           D print(src.balance) }  
when(dst) { C print(dst.balance) }
```



Can we transform BoC programs to enable more concurrency and improve performance?

We are restricted if we rely on the operational semantics as the design **X**



But some orders are the point!

But some orders are the point!

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    account.balance += amount  
    val id = account.id()  
    when(log) { B log.record(id, amount) }  
} }
```

But some orders are the point!

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    account.balance += amount  
    val id = account.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

Concurrent updates and logs should agree on the order things happened

But some orders are the point!

```
def update(account: cown[Account], amount: int) {  
  → → when(account) { A  
    account.balance += amount  
    val id = account.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)

account

log

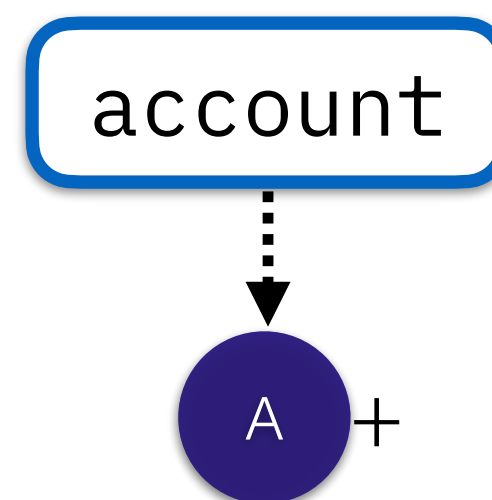
Concurrent updates and logs should agree on the order things happened

But some orders are the point!

```
def update(account: cown[Account], amount: int) {  
→  when(account) { A  
    → account.balance += amount  
      val id = account.id()  
      when(log) { B log.record(id, amount) }  
} }
```

update(src, +100)

update(src, -100)



log

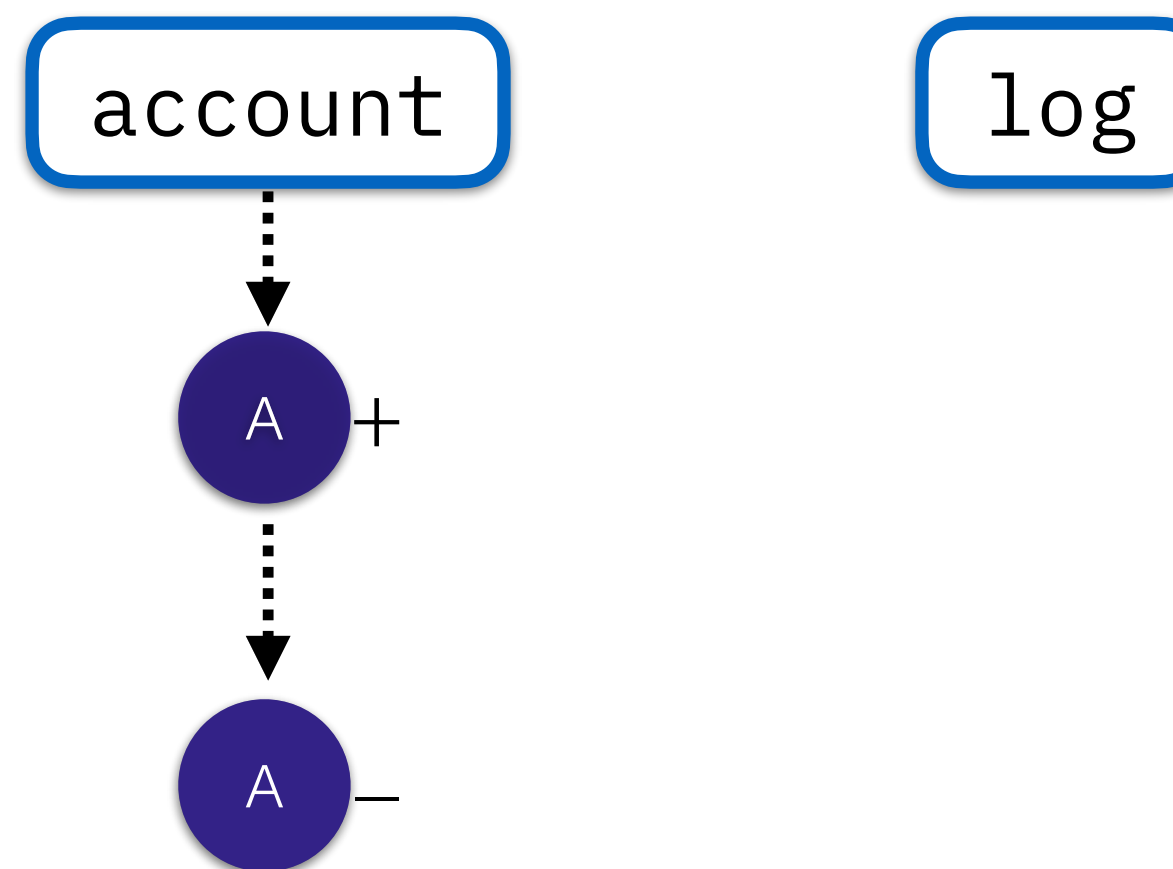
Concurrent updates and logs should agree on the order things happened

But some orders are the point!

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
→→ account.balance += amount  
    val id = account.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)



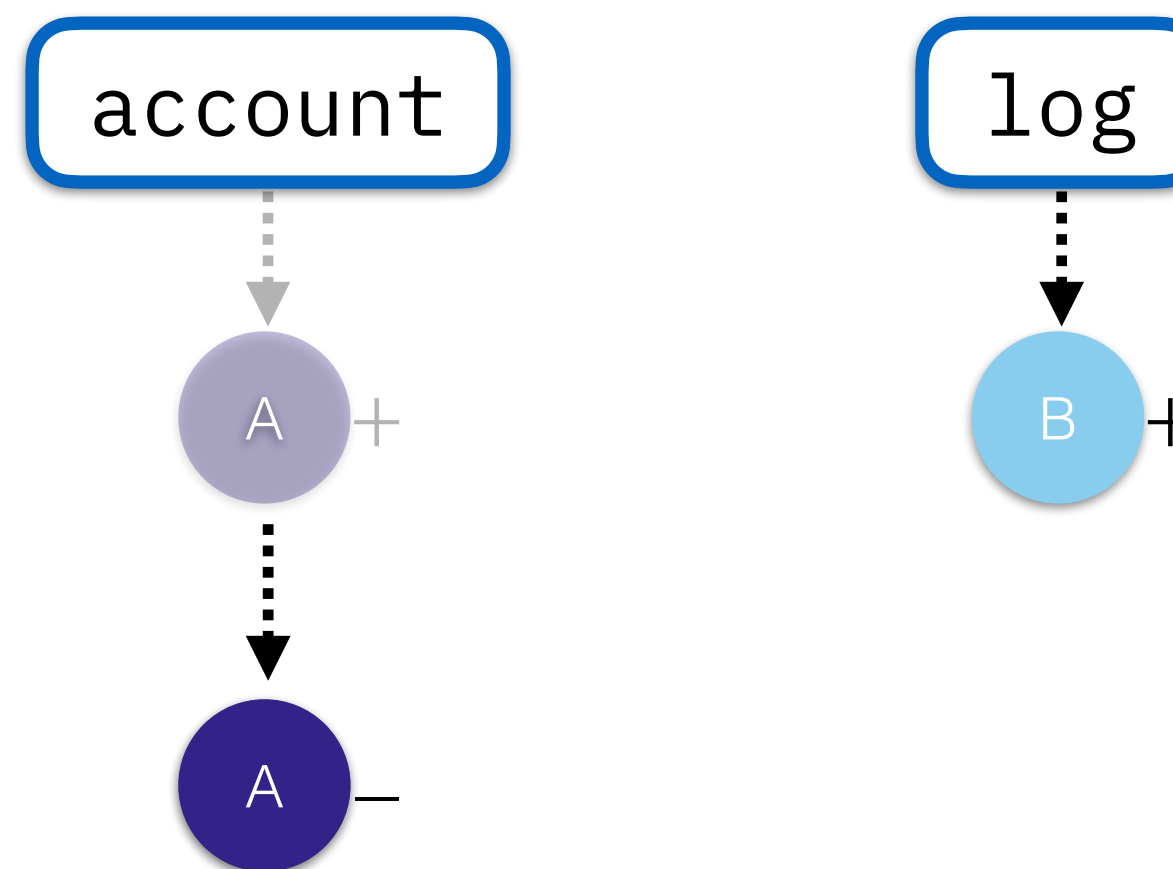
Concurrent updates and logs should agree on the order things happened

But some orders are the point!

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    ➔ account.balance += amount  
    val id = account.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)



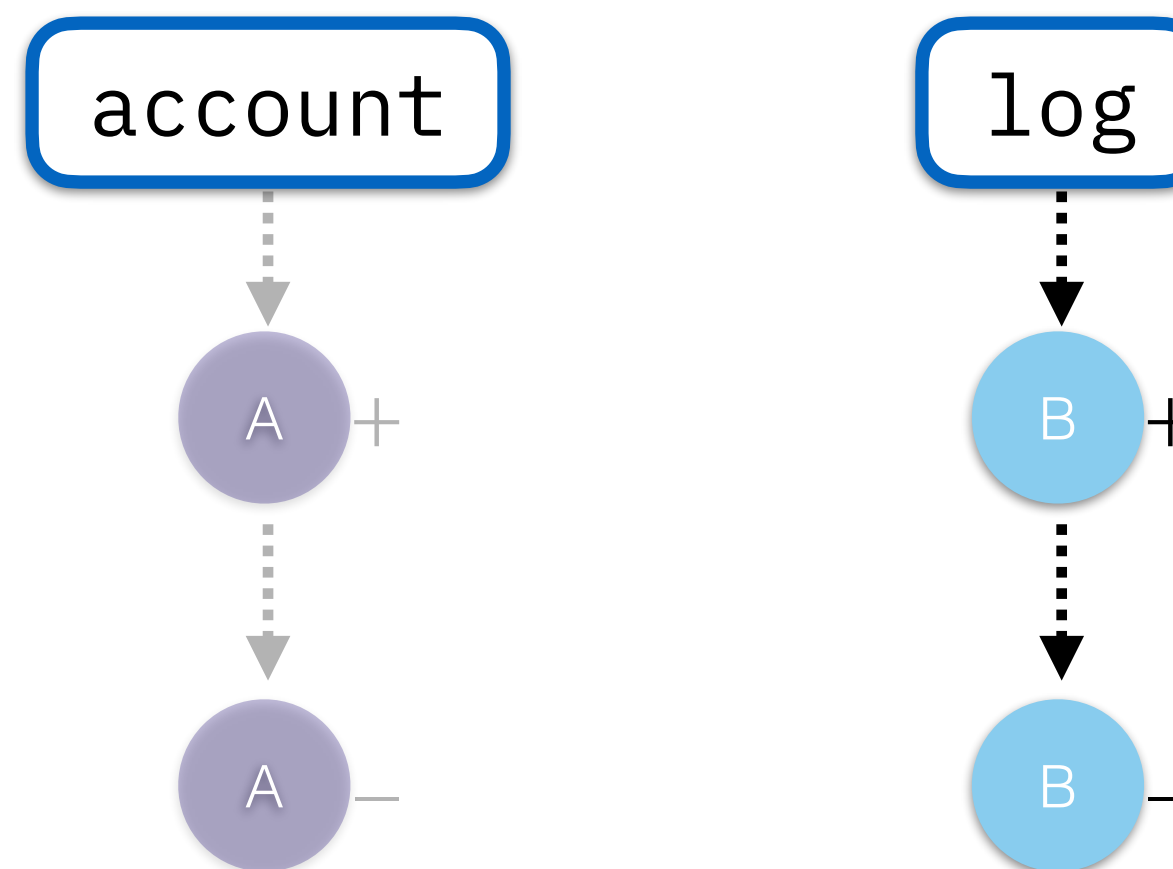
Concurrent updates and logs should agree on the order things happened

But some orders are the point!

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    account.balance += amount  
    val id = account.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)



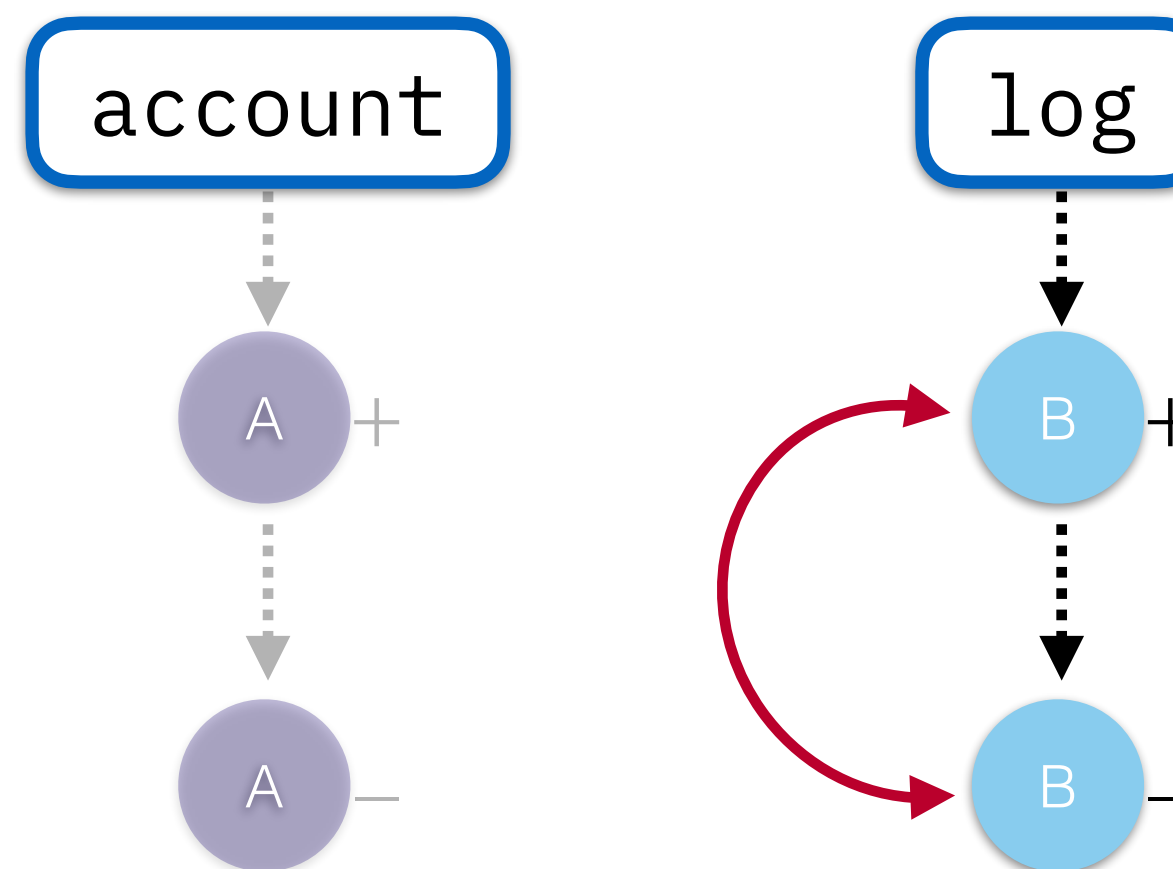
Concurrent updates and logs should agree on the order things happened

But some orders are the point!

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    account.balance += amount  
    val id = account.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)



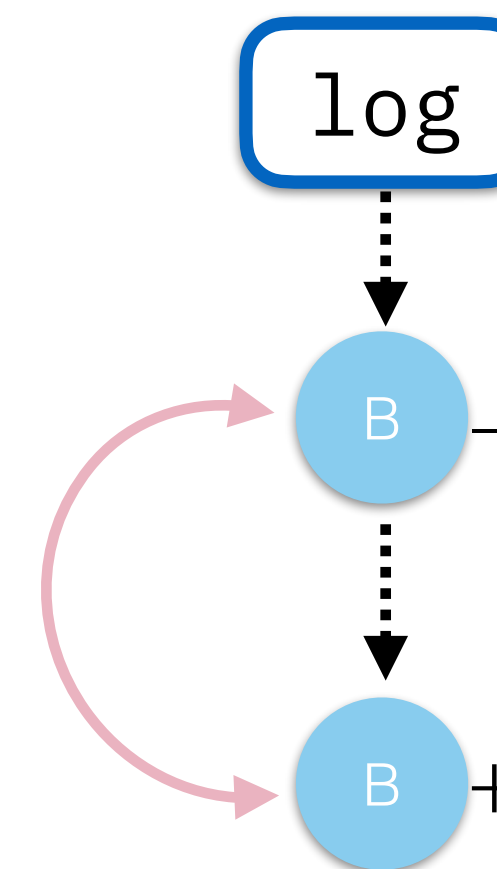
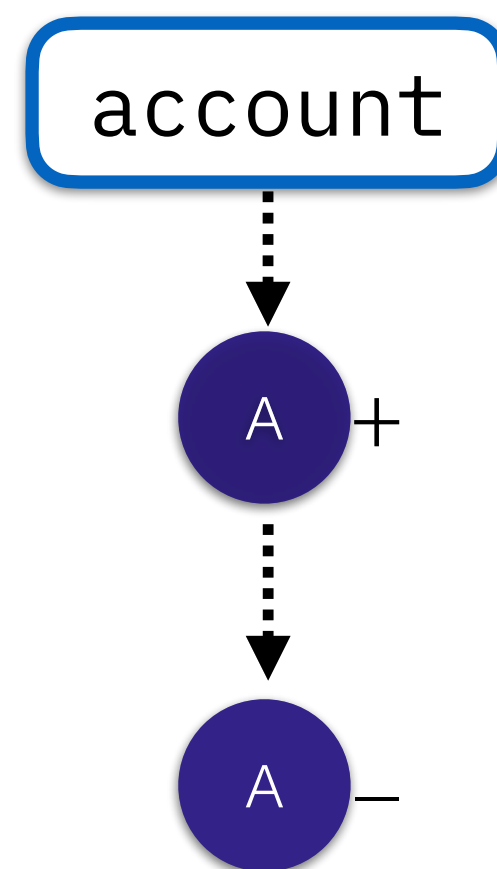
Concurrent updates and logs should agree on the order things happened

But some orders are the point!

```
def update(account: cown[Account], amount: int) {  
  → when(account) { A  
    account.balance += amount  
    val id = account.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)



Intentionally disallowed

Concurrent updates and logs should agree on the order things happened

But some orders are the point!

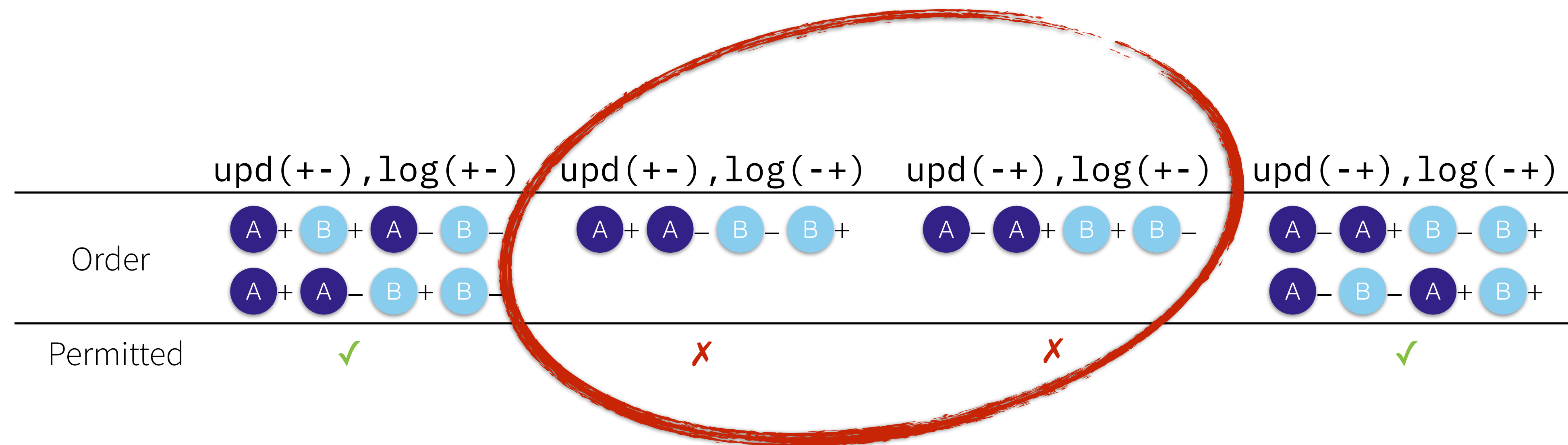
```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    account.balance += amount  
    val id = account.id()  
    when(log) { B log.record(id, amount) }  
} }
```

Order

Permitted

But some orders are the point!

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    account.balance += amount  
    val id = account.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```





Ordering Design Insights

Intrinsic — Program structure determines which orders are allowed

Incidental — Scheduling and runtime must not

Ordering Design Insights

Intrinsic — Program structure determines which orders are allowed ✓

Incidental — Scheduling and runtime must not ✗



What can we do?

Option A

Build a new operational semantics

Option B

Build an axiomatic concurrency model

What can we do?

Option A

Build a new operational semantics

- Scheduling behaviours becomes more complex
- Capturing all orders is difficult
- We may be too permissive or restrictive again

Option B

Build an axiomatic concurrency model

What can we do?

Option A

Build a new operational semantics

- Scheduling behaviours becomes more complex
- Capturing all orders is difficult
- We may be too permissive or restrictive again

Option B

Build an axiomatic concurrency model

- Take inspiration from weak memory models
- Designed to reason about optimisation
- Separate intentional and incidental causality



An Axiomatic Concurrency Model

```
when(x) {   
  x.balance += 10;  
  when(x) {  }  
}
```

Build around candidate executions

Executions are the events of the program and their order

An Axiomatic Concurrency Model

➔ `when(x) {` 
 `x.balance += 10;`
 `when(x) {`  `}`
 `}`

Build around candidate executions

Executions are the events of the program and their order

Each behaviour has three lifecycle events: Spawn, Run and Complete

`S` 

An Axiomatic Concurrency Model

```
when(x) {   
→ x.balance += 10;  
  when(x) {  }  
}
```

Build around candidate executions

Executions are the events of the program and their order

Each behaviour has three lifecycle events: Spawn, Run and Complete

S_A R_A

An Axiomatic Concurrency Model

```
when(x) {   
  x.balance += 10;  
  when(x) {  }  
   }
```

Build around candidate executions

Executions are the events of the program and their order

Each behaviour has three lifecycle events: Spawn, Run and Complete

S

R

C

An Axiomatic Concurrency Model

```
when(x) {   
  x.balance += 10;  
  when(x) {  }  
}
```

Build around candidate executions

Executions are the events of the program and their order

Each behaviour has three lifecycle events: Spawn, Run and Complete

S

R

S

C

R

C

An Axiomatic Concurrency Model

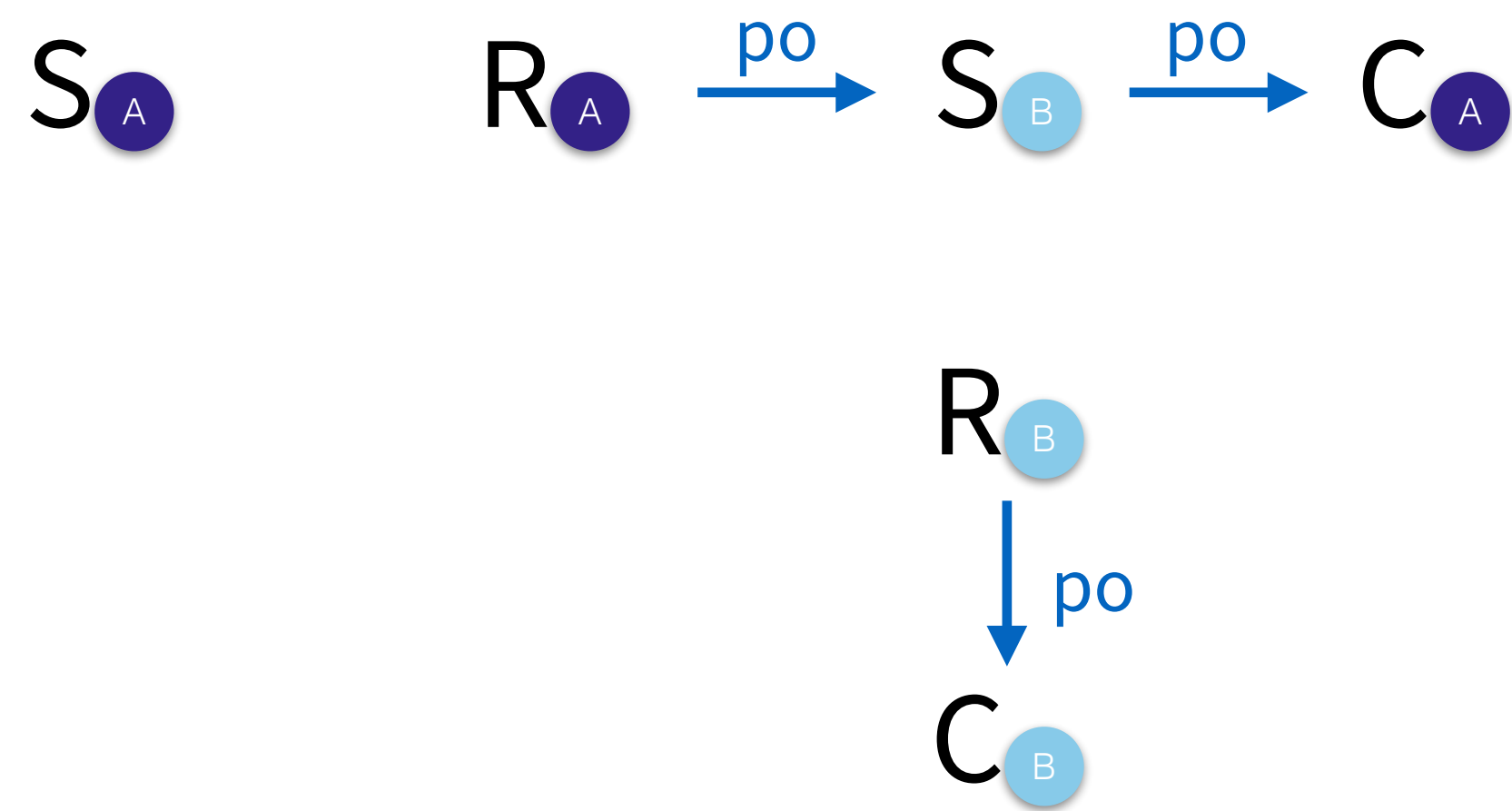
```
when(x) { A
  x.balance += 10;
  when(x) { B }
}
```

Build around candidate executions

Executions are the events of the program and their order

Each behaviour has three lifecycle events: Spawn, Run and Complete

po is the program order



An Axiomatic Concurrency Model

```
when(x) { A  
  x.balance += 10;  
  when(x) { B }  
}
```

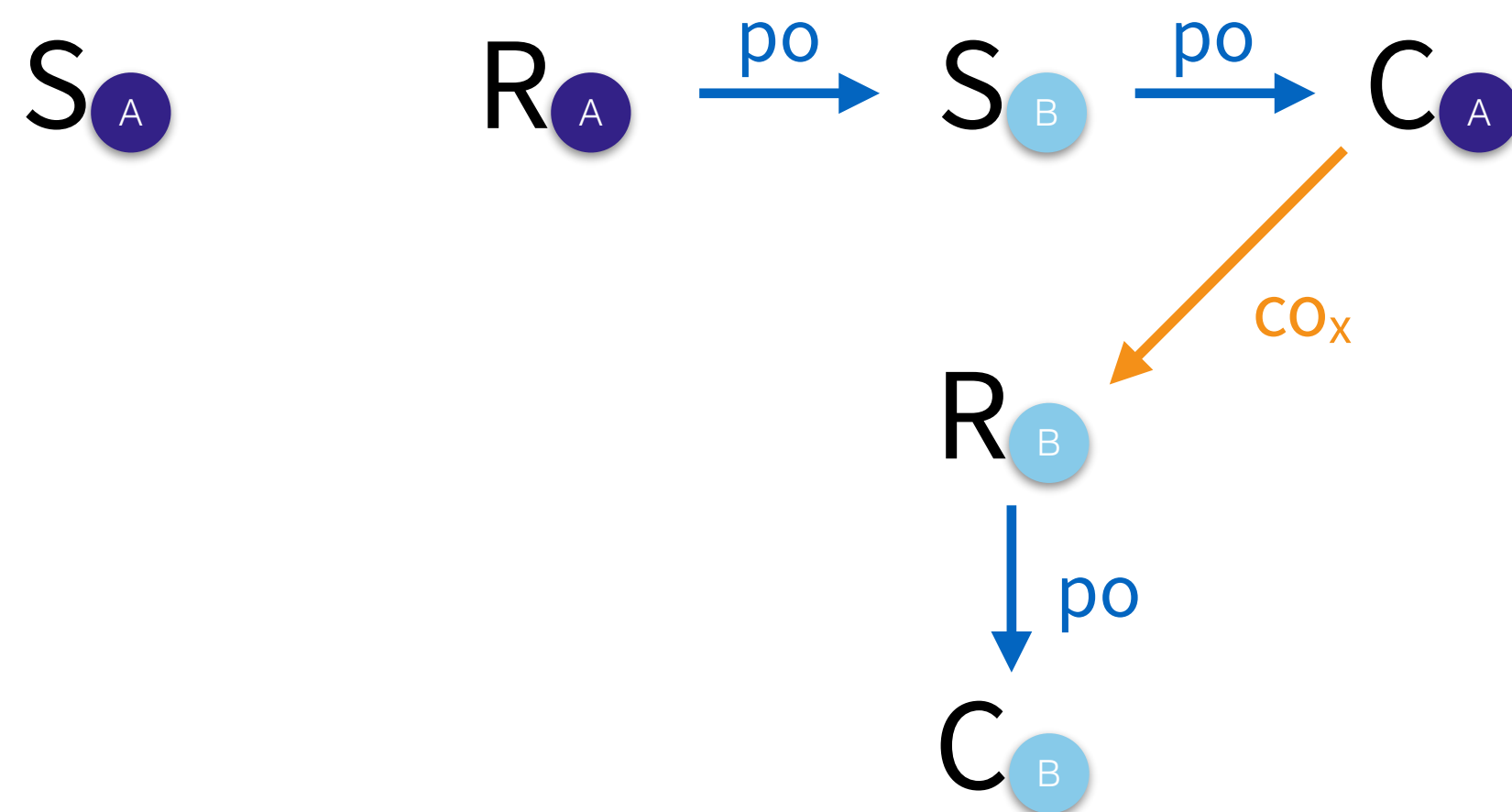
Build around candidate executions

Executions are the events of the program and their order

Each behaviour has three lifecycle events: Spawn, Run and Complete

po is the program order

co is the cown order



An Axiomatic Concurrency Model

```
when(x) { A
  x.balance += 10;
  when(x) { B }
}
```

Build around candidate executions

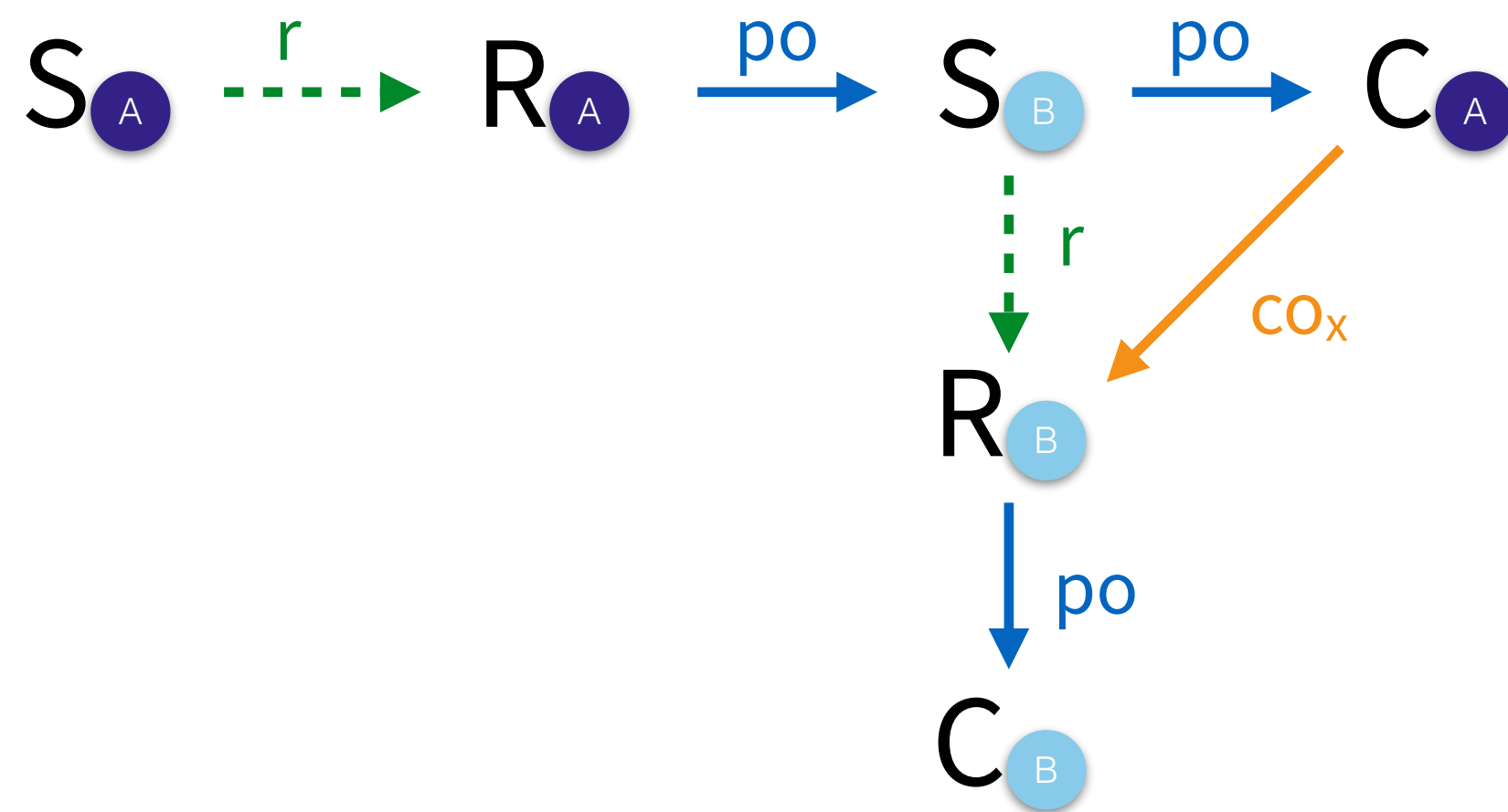
Executions are the events of the program and their order

Each behaviour has three lifecycle events: Spawn, Run and Complete

po is the program order

co is the cown order

r is the derived run order



An Axiomatic Concurrency Model

```
when(x) { A
  x.balance += 10;
  when(x) { B }
}
```

Build around candidate executions

Executions are the events of the program and their order

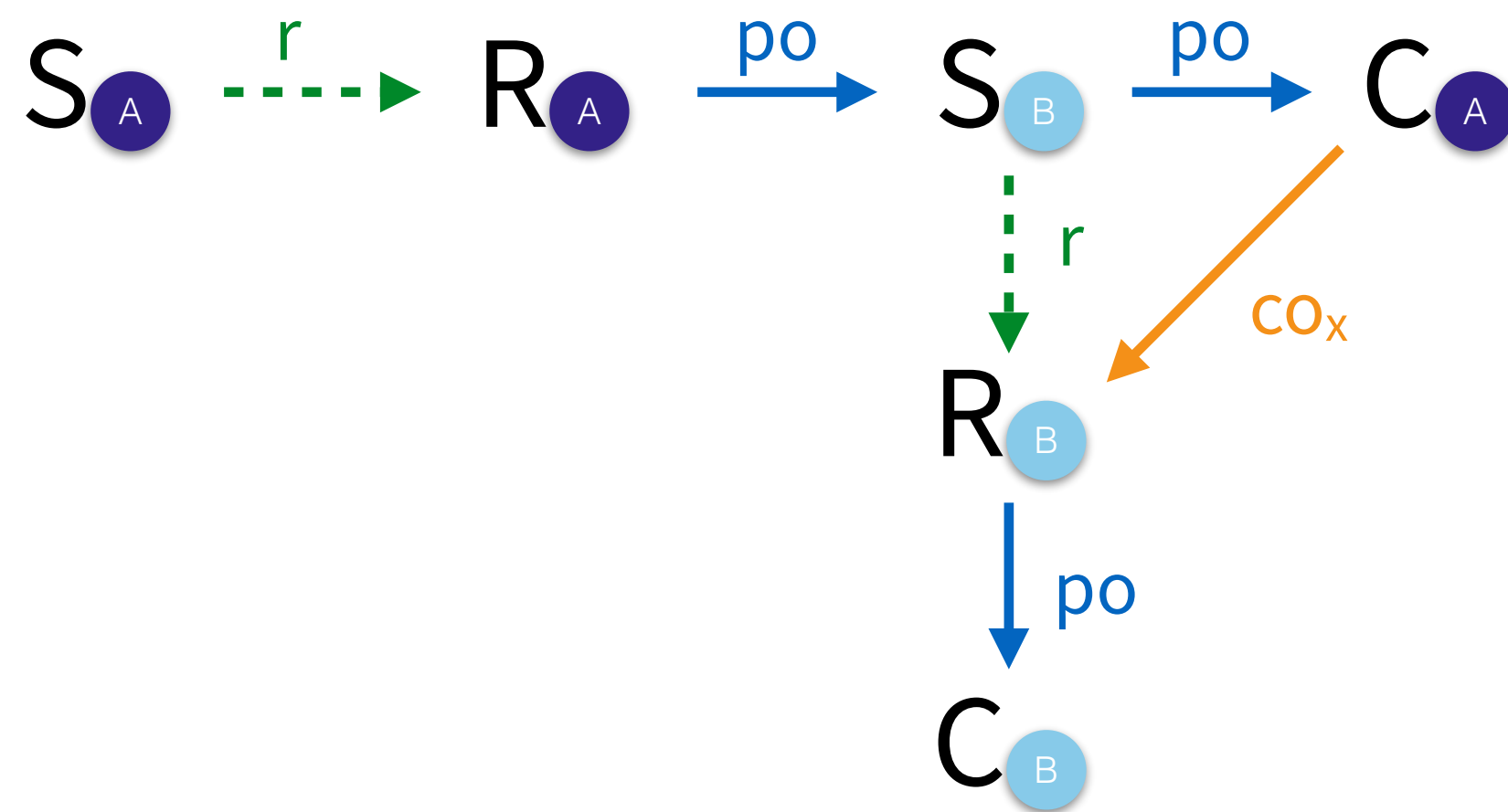
Each behaviour has three lifecycle events: Spawn, Run and Complete

po is the program order

co is the cown order

r is the derived run order

hb is the derived happens before order



An Axiomatic Concurrency Model

```
when(x) { A  
  x.balance += 10;  
  when(x) { B }  
}
```

Build around candidate executions

Executions are the events of the program and their order

Each behaviour has three lifecycle events: Spawn, Run and Complete

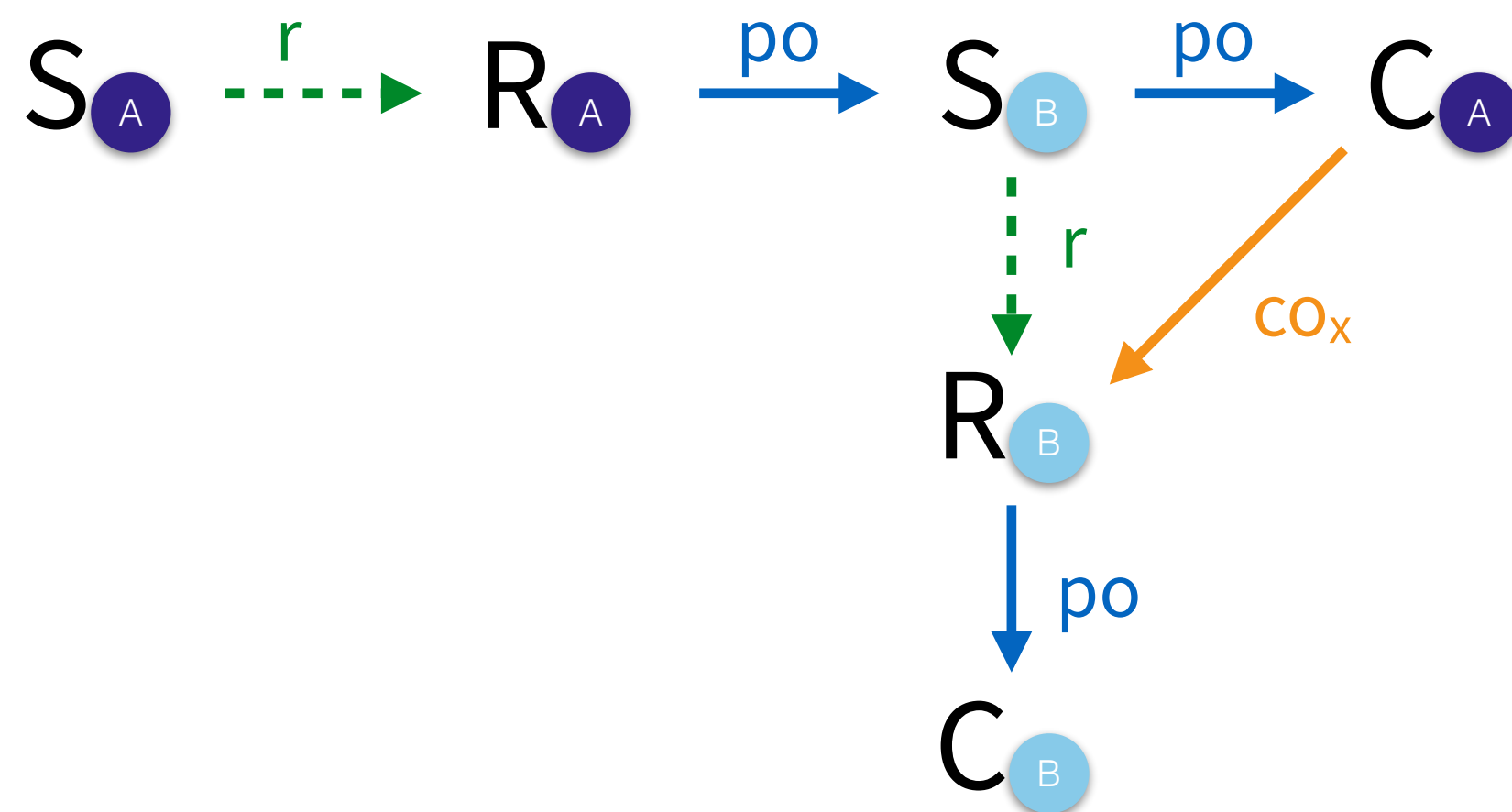
po is the program order

co is the cown order

r is the derived run order

hb is the derived happens before order

If **behaviours share a cown** and have a
path between their spawns then
a complete happens-before a run



An Axiomatic Concurrency Model

```
when(x) { A
  x.balance += 10;
  when(x) { B }
}
```

Build around candidate executions

Executions are the events of the program and their order

Each behaviour has three lifecycle events: Spawn, Run and Complete

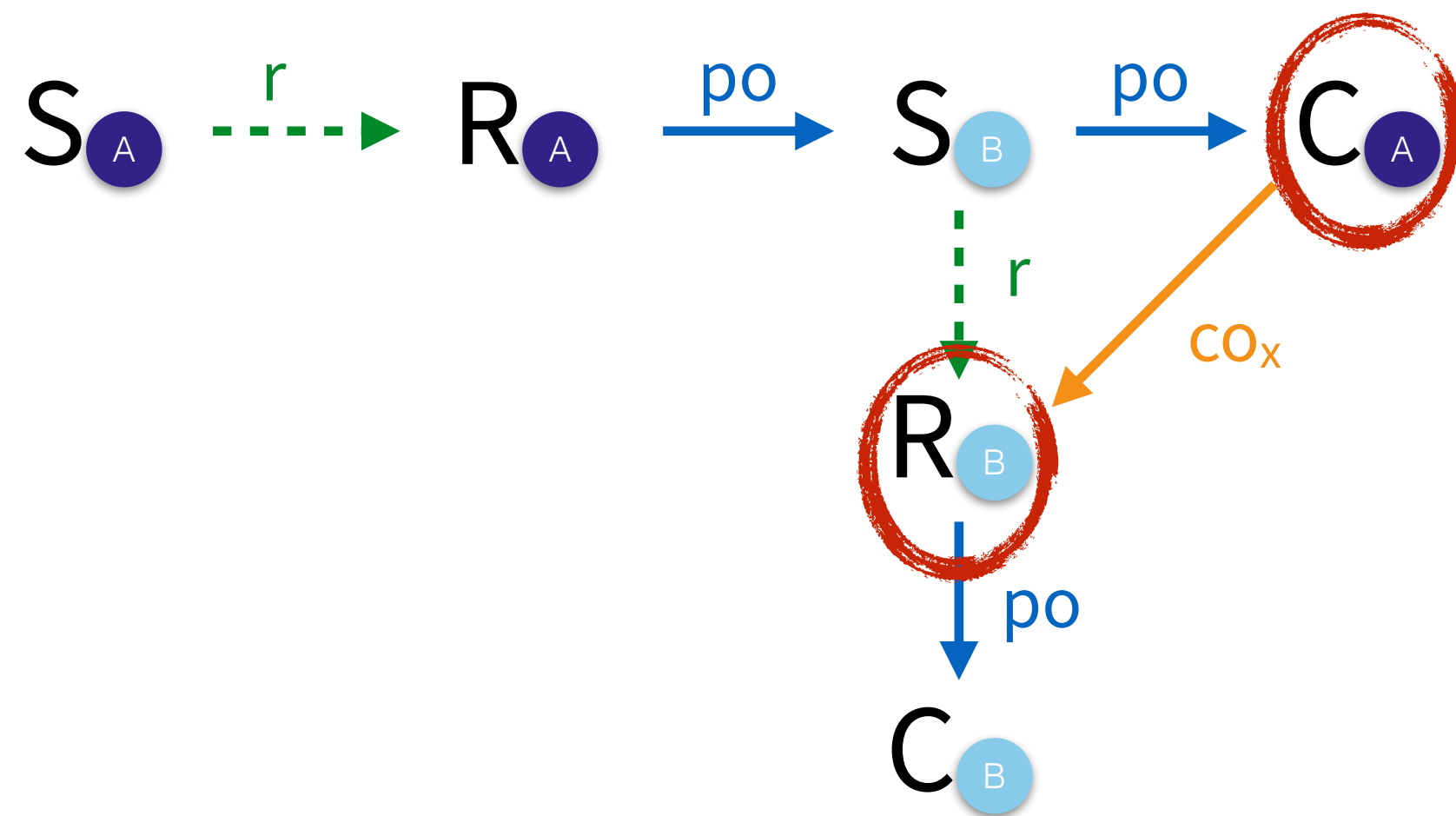
po is the program order

co is the cown order

r is the derived run order

hb is the derived happens before order

If **behaviours share a cown** and have a **path between their spawns** then **a complete happens-before a run**



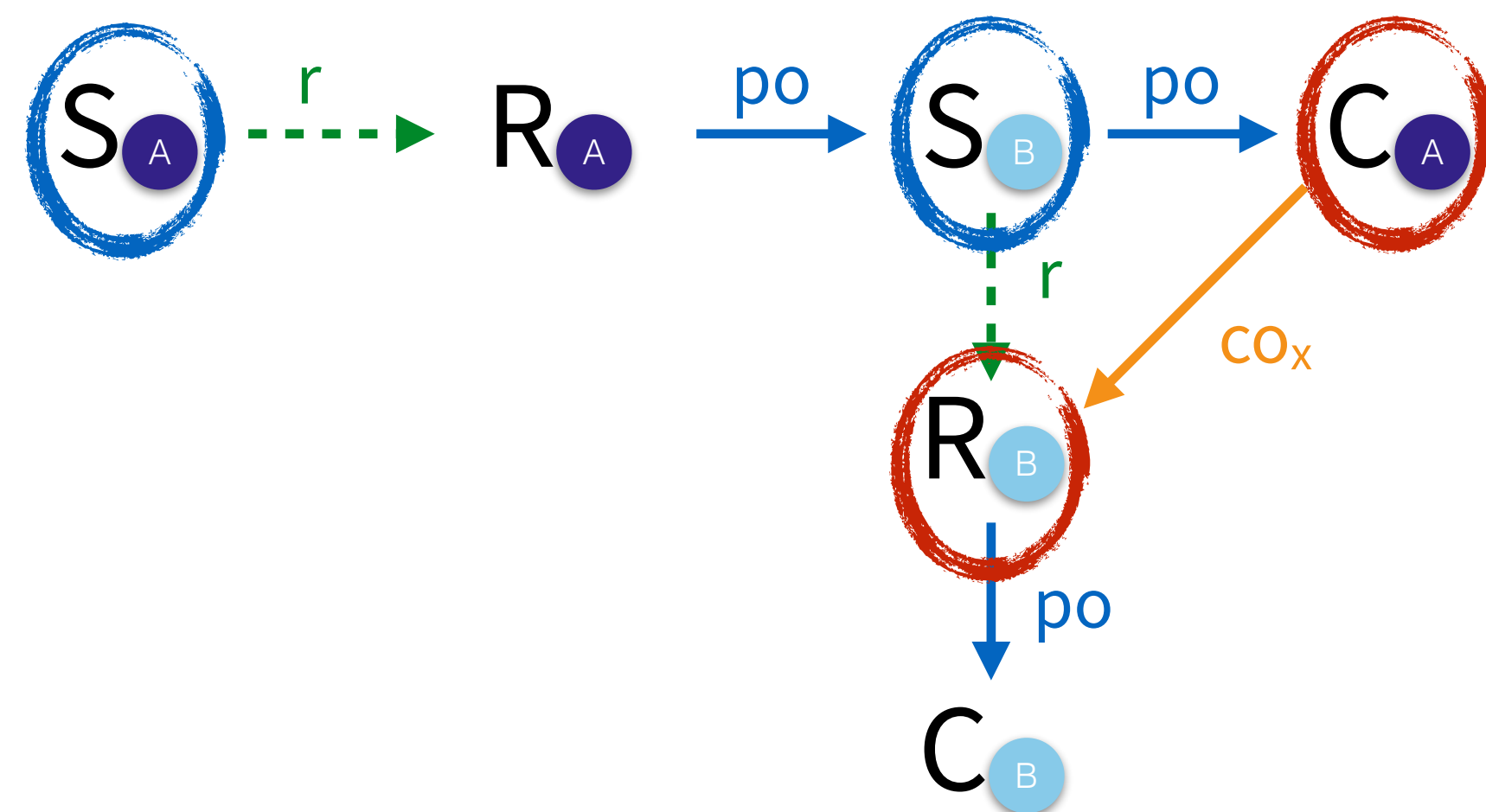
An Axiomatic Concurrency Model

```
when(x) { A
  x.balance += 10;
  when(x) { B }
}
```

Build around candidate executions

Executions are the events of the program and their order

Each behaviour has three lifecycle events: Spawn, Run and Complete



po is the program order

co is the cown order

r is the derived run order

hb is the derived happens before order

If **behaviours share a cown** and have a **path between their spawns** then **a complete happens-before a run**

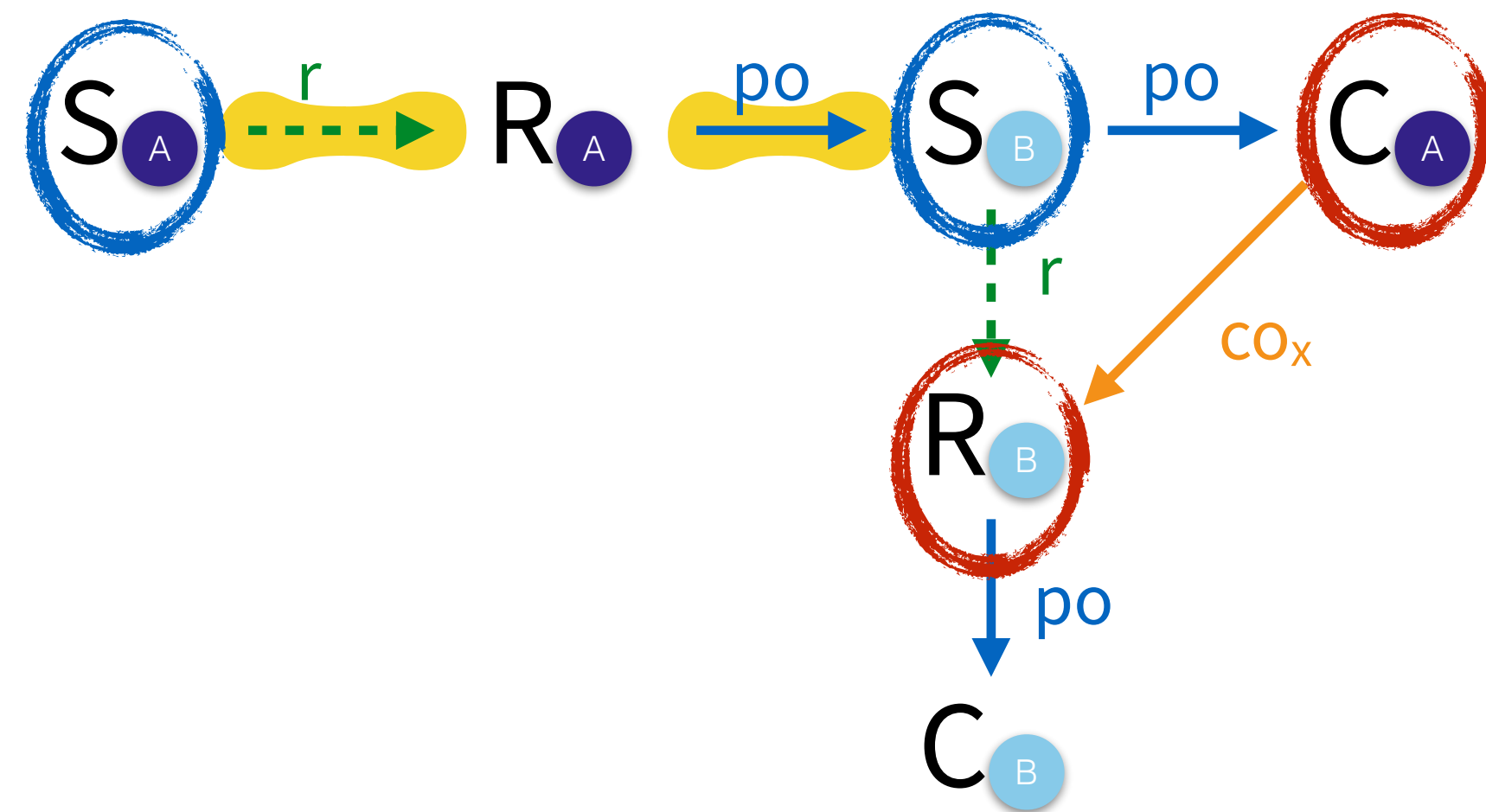
An Axiomatic Concurrency Model

```
when(x) { A
  x.balance += 10;
  when(x) { B }
}
```

Build around candidate executions

Executions are the events of the program and their order

Each behaviour has three lifecycle events: Spawn, Run and Complete



po is the program order

co is the cow order

r is the derived run order

hb is the derived happens before order

If **behaviours share a cow** and have a **path between their spawns** then **a complete happens-before a run**

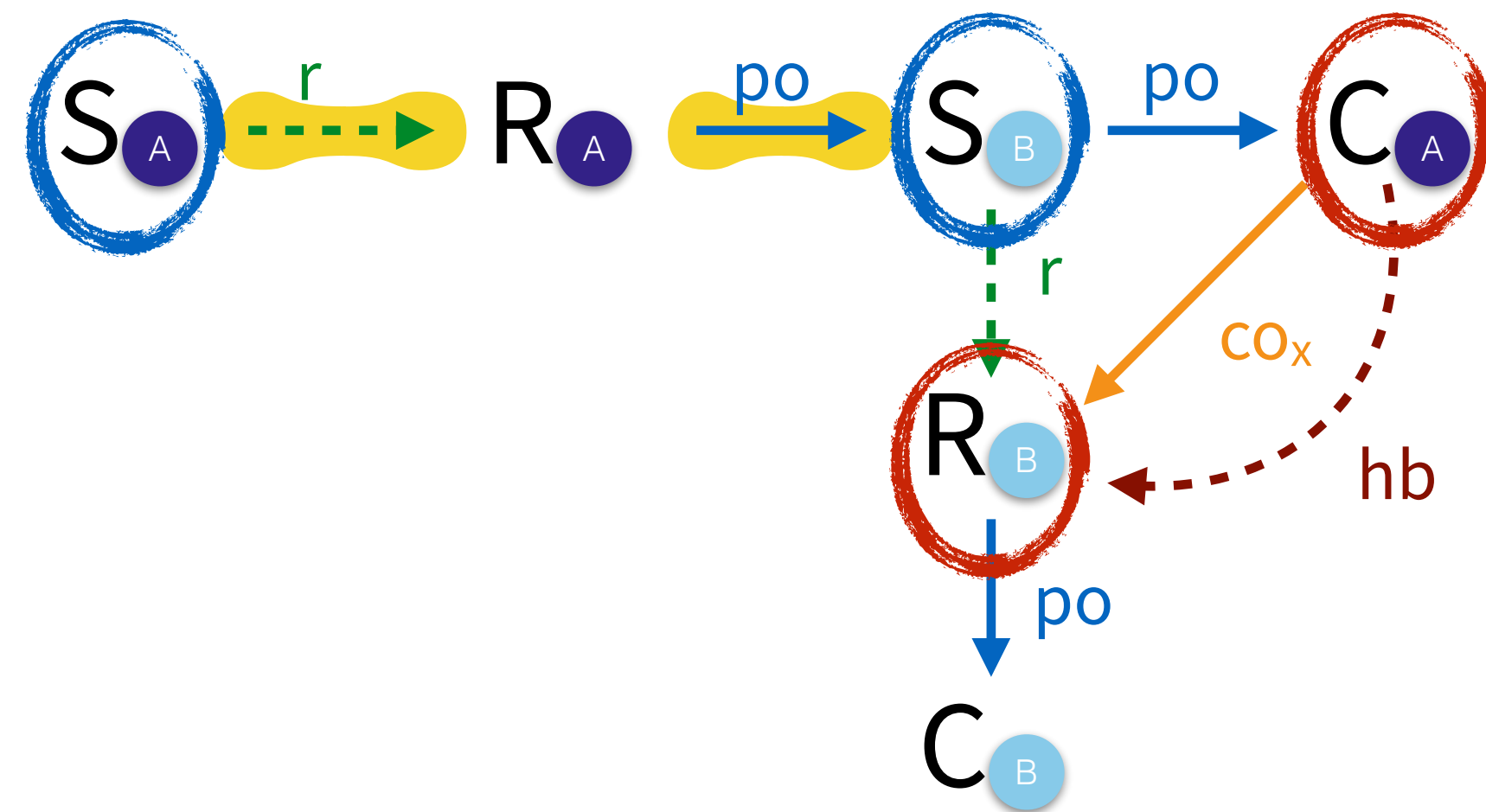
An Axiomatic Concurrency Model

```
when(x) { A
  x.balance += 10;
  when(x) { B }
}
```

Build around candidate executions

Executions are the events of the program and their order

Each behaviour has three lifecycle events: Spawn, Run and Complete



po is the program order

co is the cow order

r is the derived run order

hb is the derived happens before order

If **behaviours share a cow** and have a **path between their spawns** then **a complete happens-before a run**

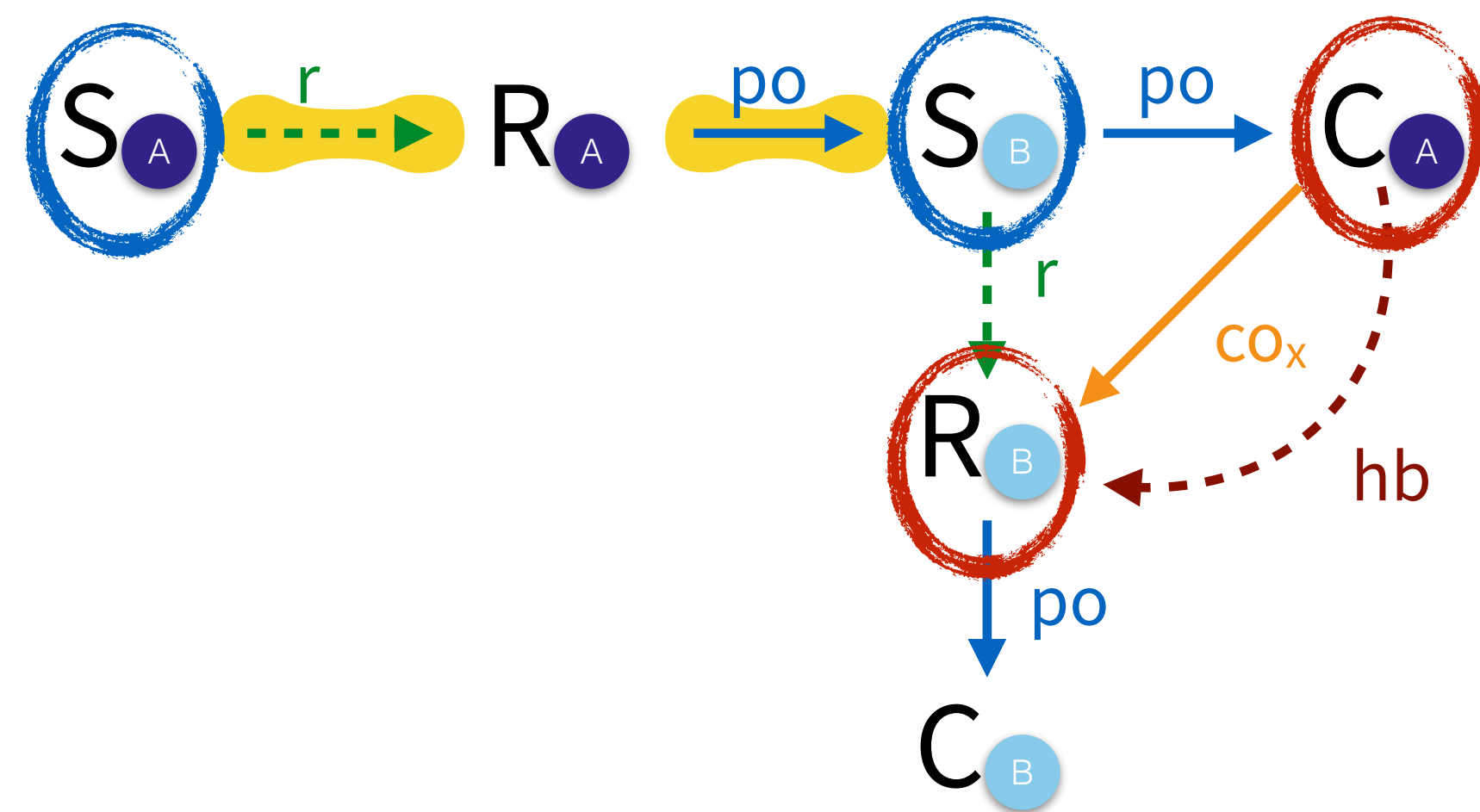
An Axiomatic Concurrency Model

```
when(x) { A  
  x.balance += 10;  
  when(x) { B }  
}
```

Build around candidate executions

Executions are the events of the program and their order

Each behaviour has three lifecycle events: Spawn, Run and Complete



po is the program order

co is the cow order

r is the derived run order

hb is the derived happens before order

If **behaviours share a cow** and have a **path between their spawns** then a **complete happens-before a run**

Executions are valid if acyclic

Validating Causality

```
def update(acc: cown[Account], amount: int) {  
  when(acc) { A  
    acc.balance += amount  
    val id = acc.id()  
    when(log) { B log.record(id, amount) }  
}
```

update(src, +100)

update(src, -100)

Validating Causality

```
def update(acc: cown[Account], amount: int) {  
  when(acc) { A  
    acc.balance += amount  
    val id = acc.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)

We are given the program events

S_{A+} R_{A+} S_{B+} C_{A+} R_{B+} C_{B+}

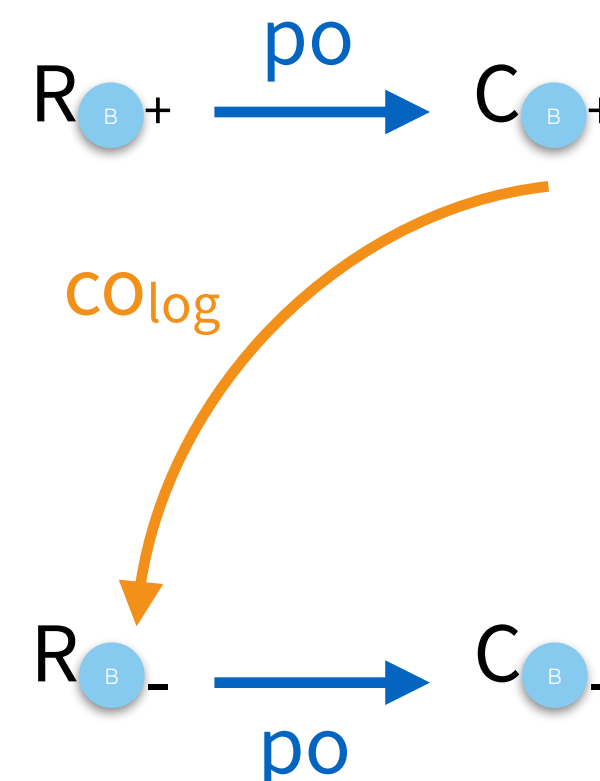
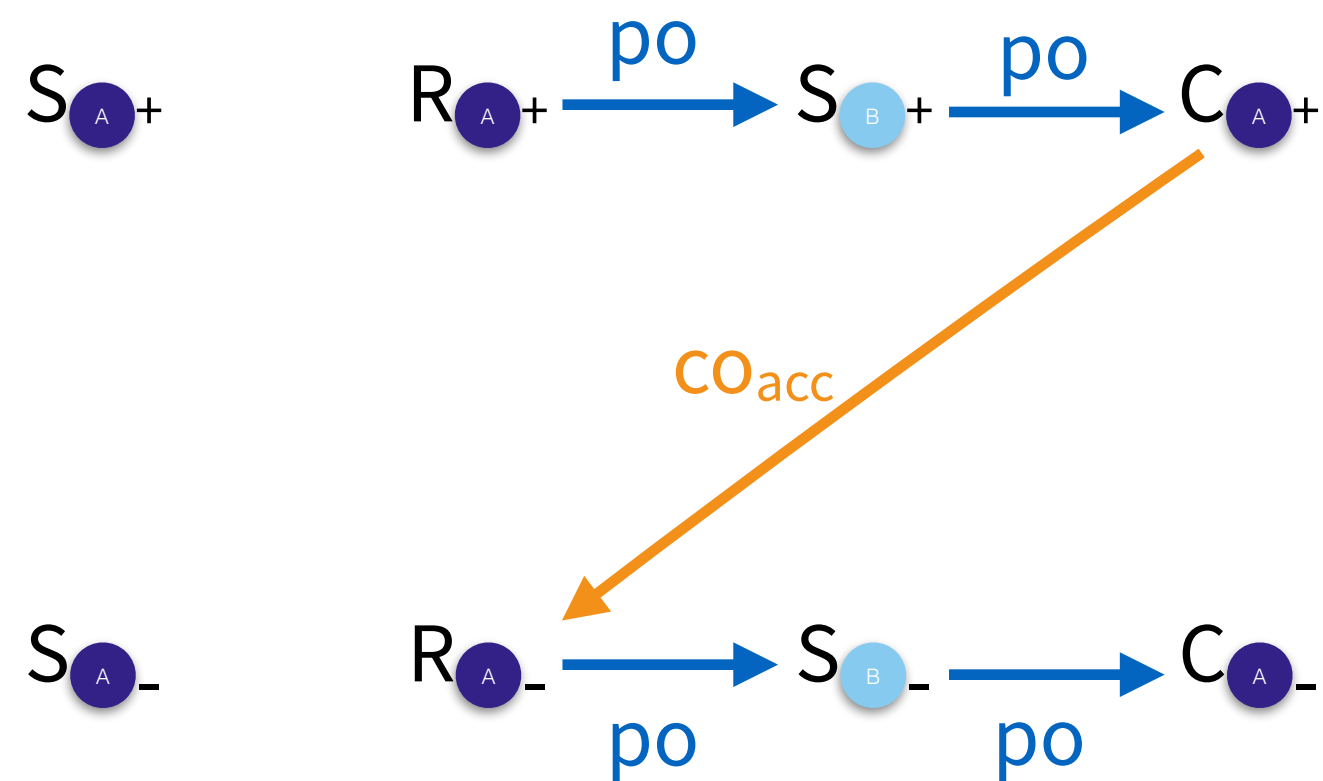
S_{A-} R_{A-} S_{B-} C_{A-} R_{B-} C_{B-}

Validating Causality

```
def update(acc: cown[Account], amount: int) {  
  when(acc) { A  
    acc.balance += amount  
    val id = acc.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)



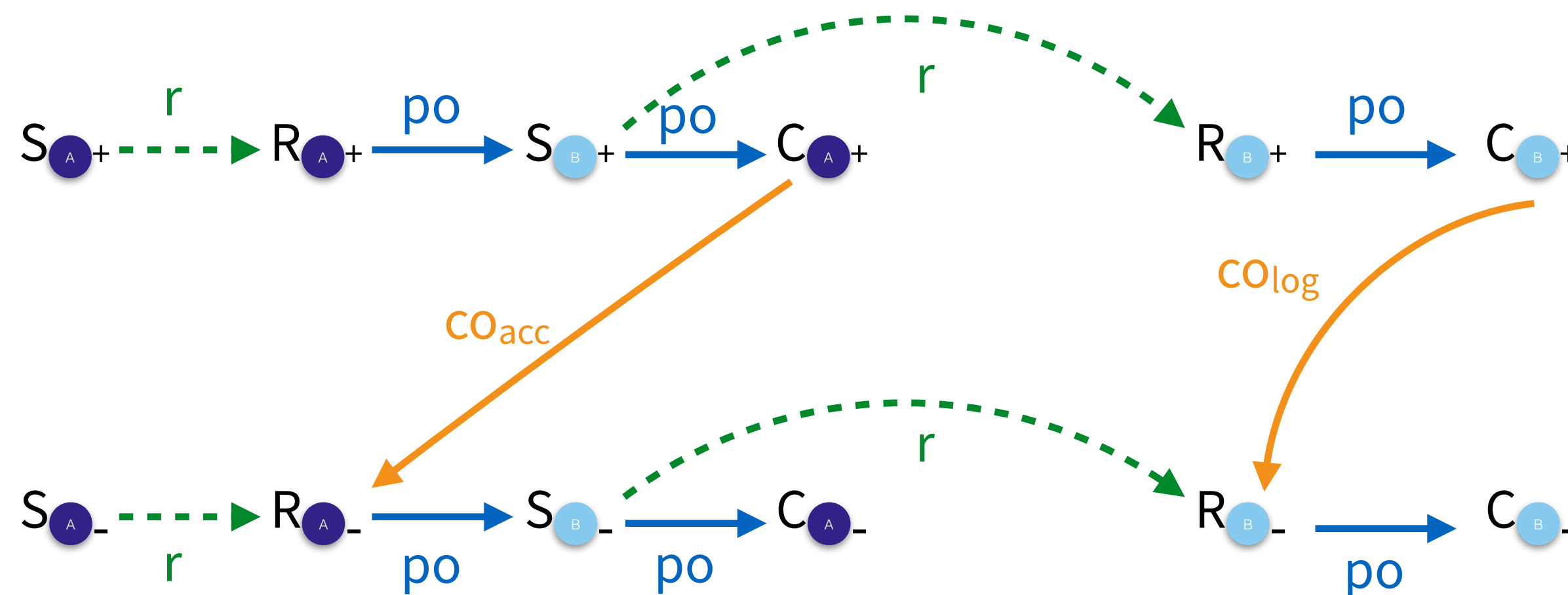
We are given the program events
We are given the program and cown orders

Validating Causality

```
def update(acc: cown[Account], amount: int) {  
  when(acc) { A  
    acc.balance += amount  
    val id = acc.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)



We are given the program events

We are given the program and cown orders

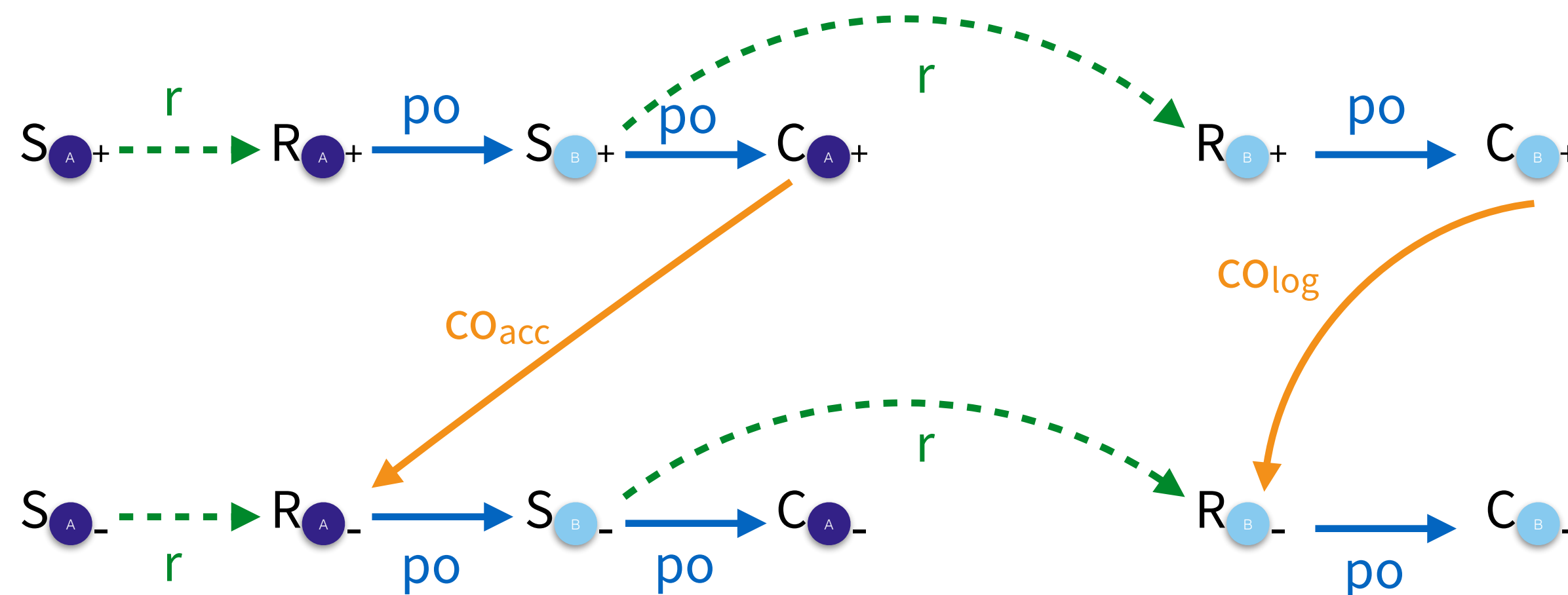
We derive the run orders

Validating Causality

```
def update(acc: cown[Account], amount: int) {  
  when(acc) { A  
    acc.balance += amount  
    val id = acc.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)



We are given the program events

We are given the program and cown orders

We derive the run orders

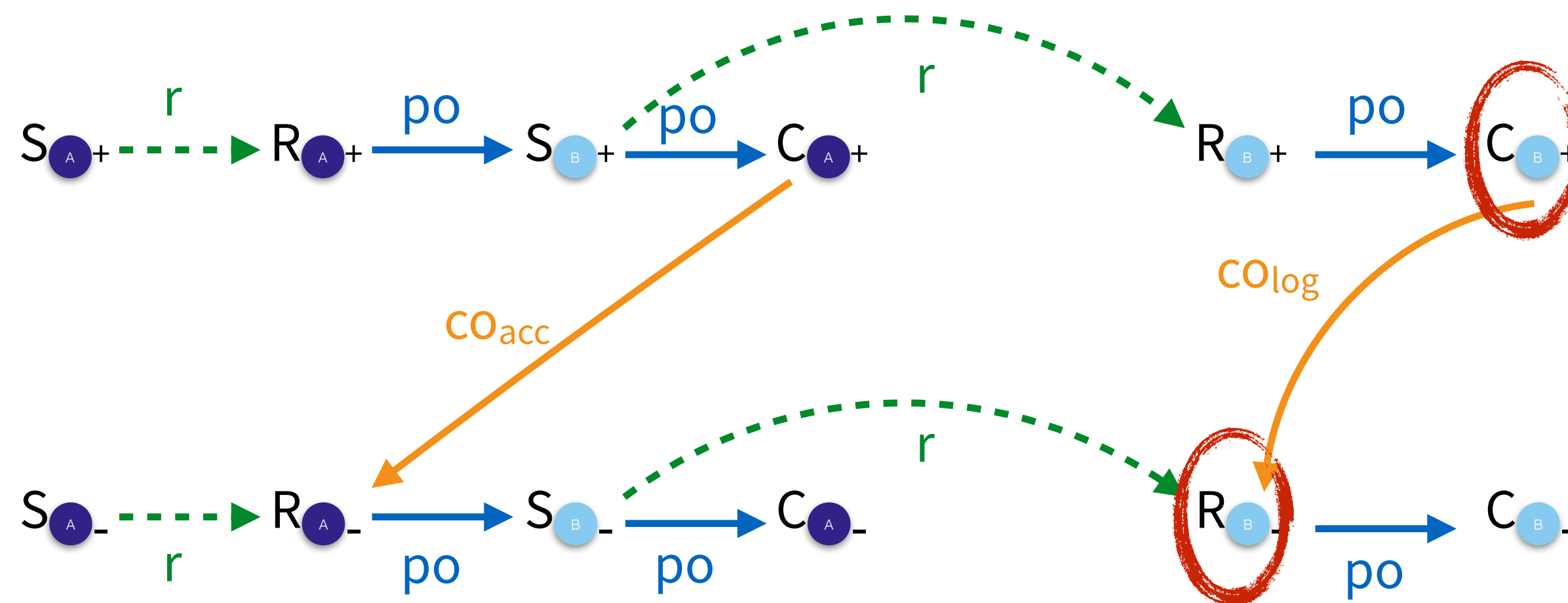
We derive the happens before orders

Validating Causality

```
def update(acc: cown[Account], amount: int) {  
  when(acc) { A  
    acc.balance += amount  
    val id = acc.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)



We are given the program events

We are given the program and cown orders

We derive the run orders

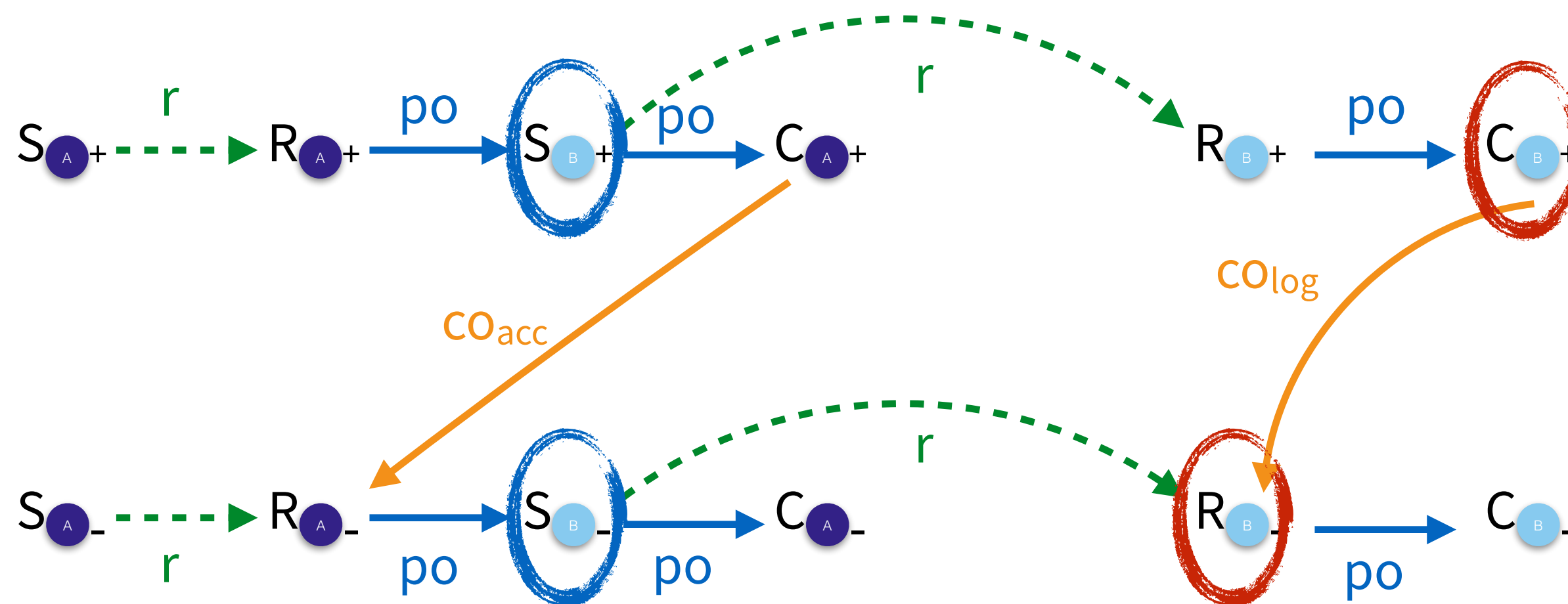
We derive the happens before orders

Validating Causality

```
def update(acc: cown[Account], amount: int) {  
  when(acc) { A  
    acc.balance += amount  
    val id = acc.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)



We are given the program events

We are given the program and cown orders

We derive the run orders

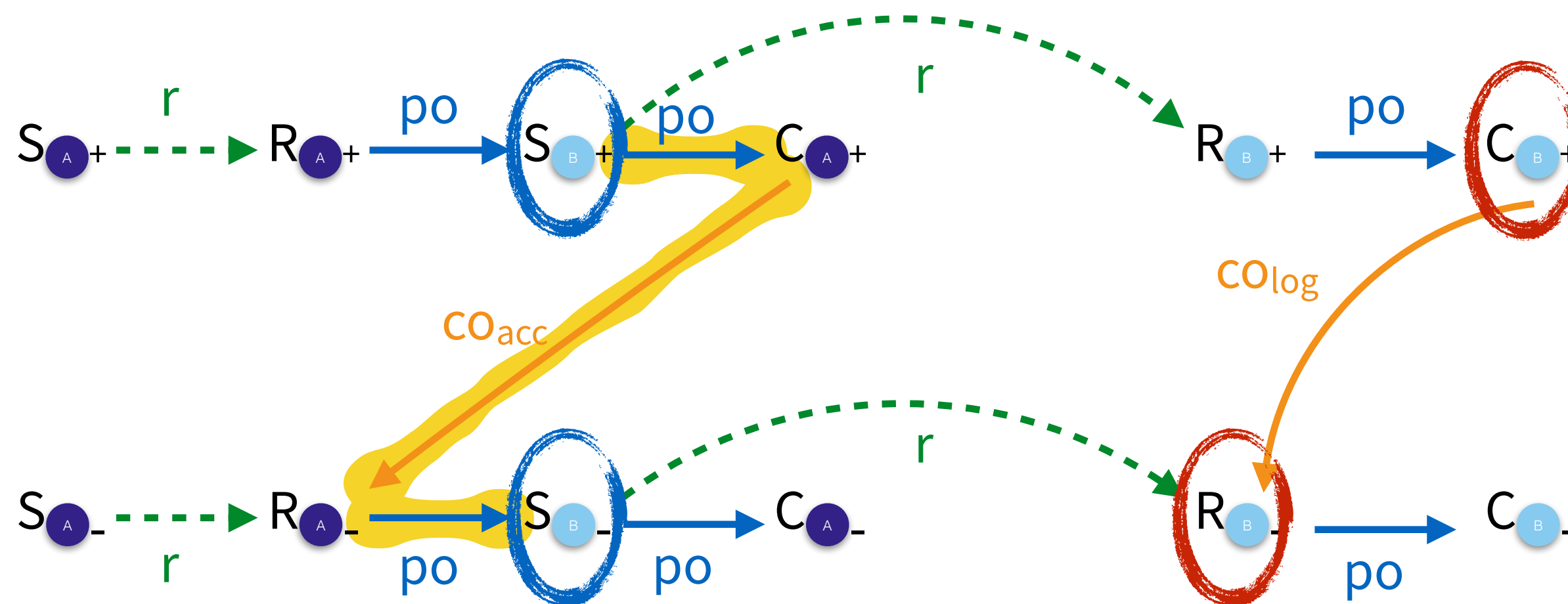
We derive the happens before orders

Validating Causality

```
def update(acc: cown[Account], amount: int) {  
  when(acc) { A  
    acc.balance += amount  
    val id = acc.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)



We are given the program events

We are given the program and cown orders

We derive the run orders

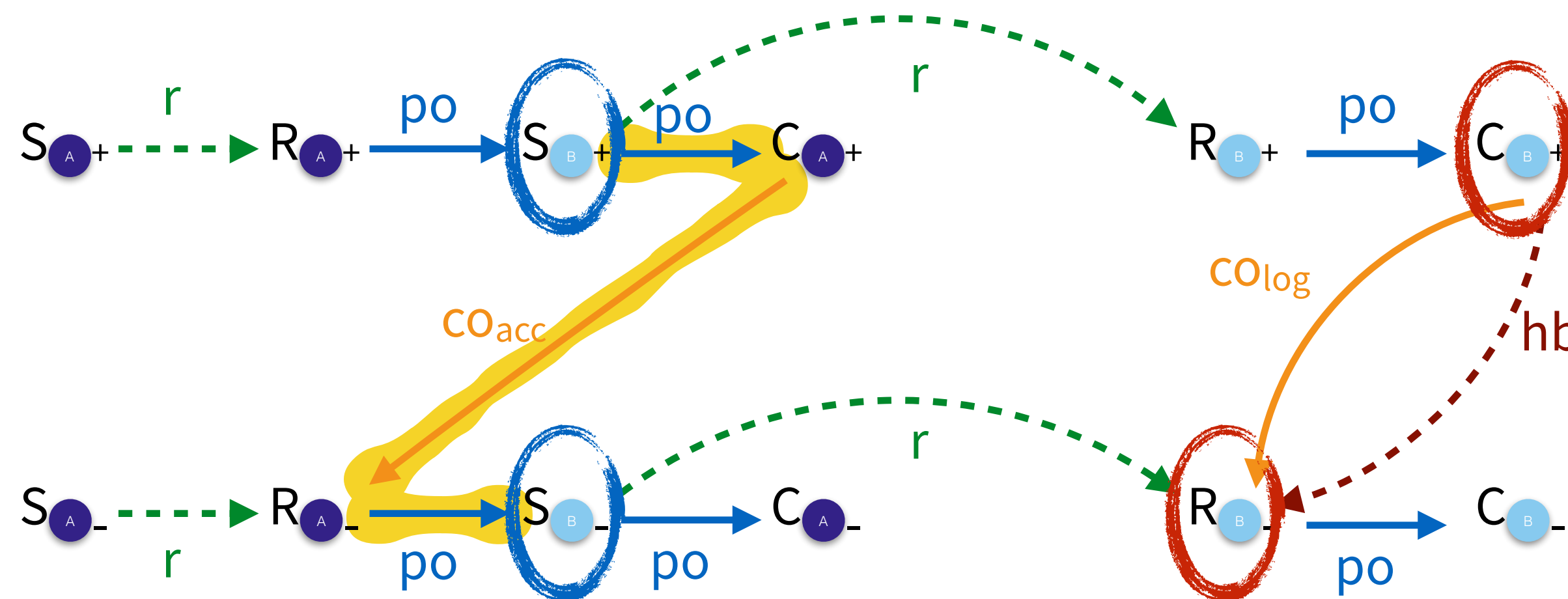
We derive the happens before orders

Validating Causality

```
def update(acc: cown[Account], amount: int) {  
  when(acc) { A  
    acc.balance += amount  
    val id = acc.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)



We are given the program events

We are given the program and cown orders

We derive the run orders

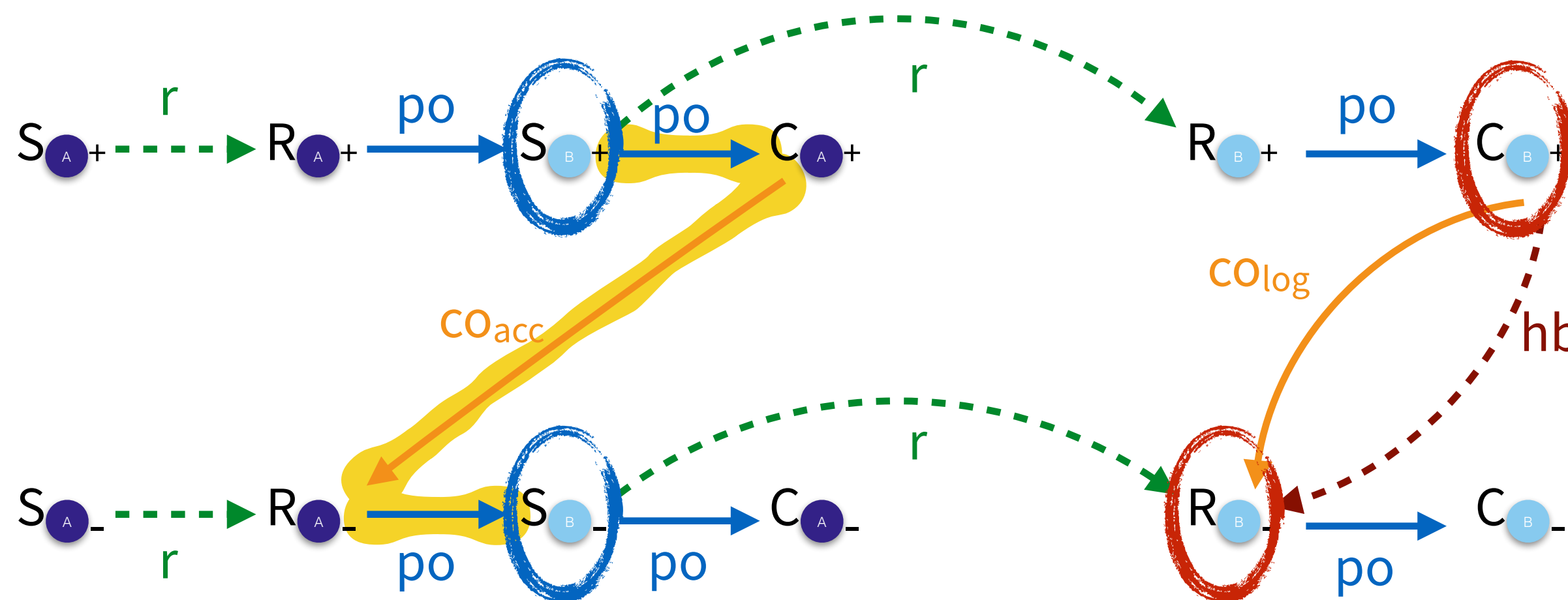
We derive the happens before orders

Validating Causality

```
def update(acc: cown[Account], amount: int) {  
  when(acc) { A  
    acc.balance += amount  
    val id = acc.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)



We are given the program events

We are given the program and cown orders

We derive the run orders

We derive the happens before orders

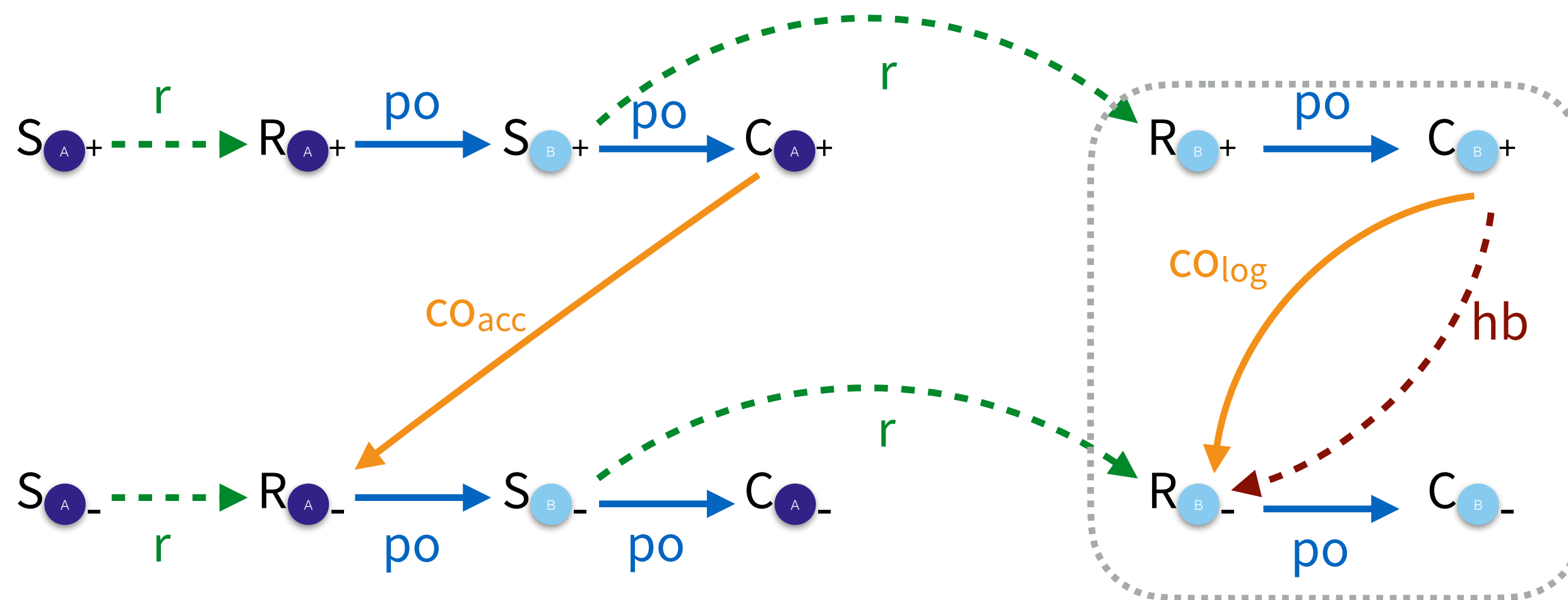
We test for acyclicity ✓

Validating Causality

```
def update(acc: cown[Account], amount: int) {  
  when(acc) { A  
    acc.balance += amount  
    val id = acc.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)

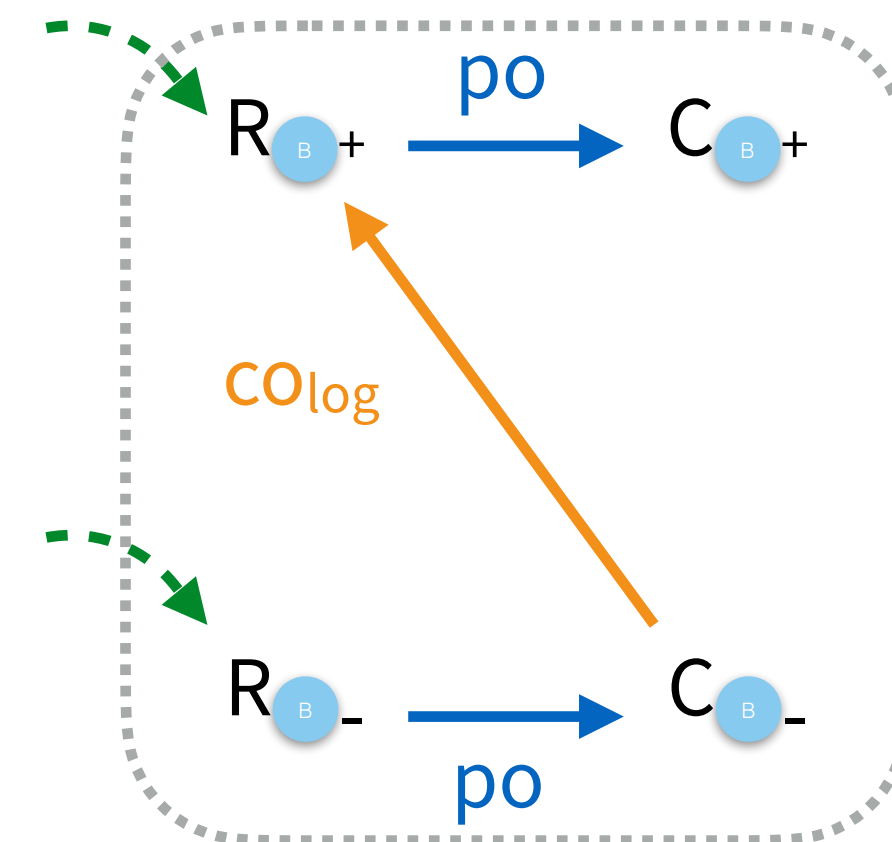
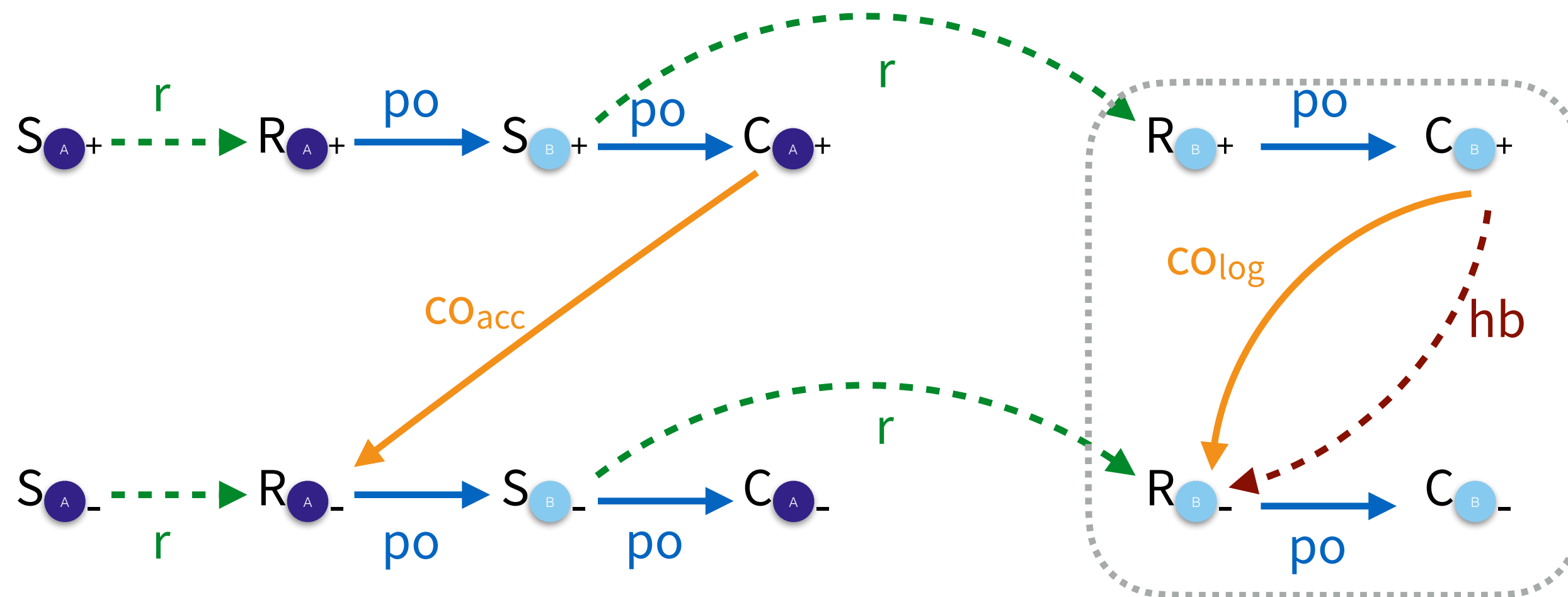


Validating Causality

```
def update(acc: cown[Account], amount: int) {  
  when(acc) { A  
    acc.balance += amount  
    val id = acc.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)

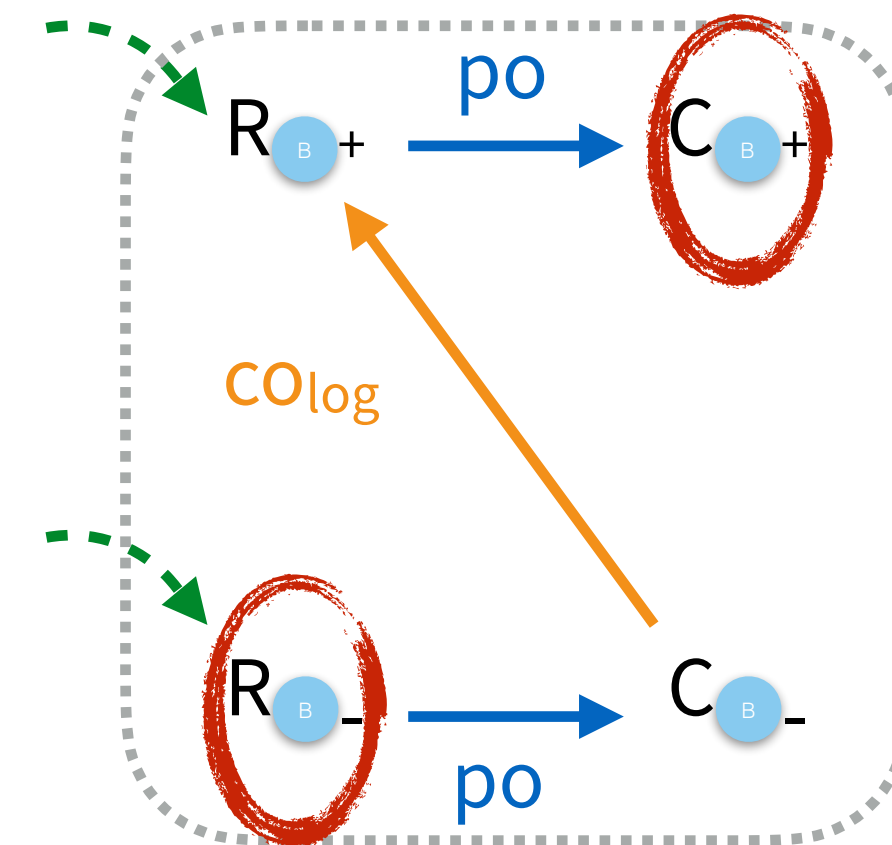
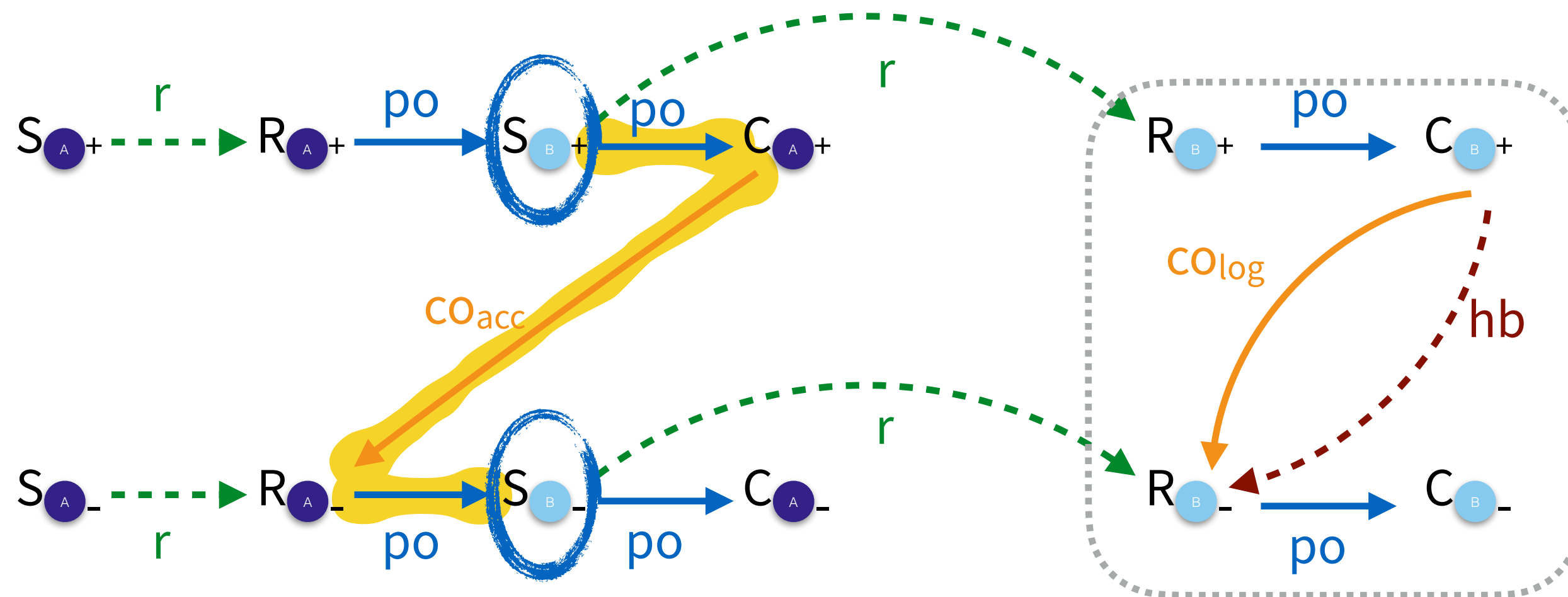


Validating Causality

```
def update(acc: cown[Account], amount: int) {  
  when(acc) { A  
    acc.balance += amount  
    val id = acc.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

```
update(src, +100)
```

```
update(src, -100)
```

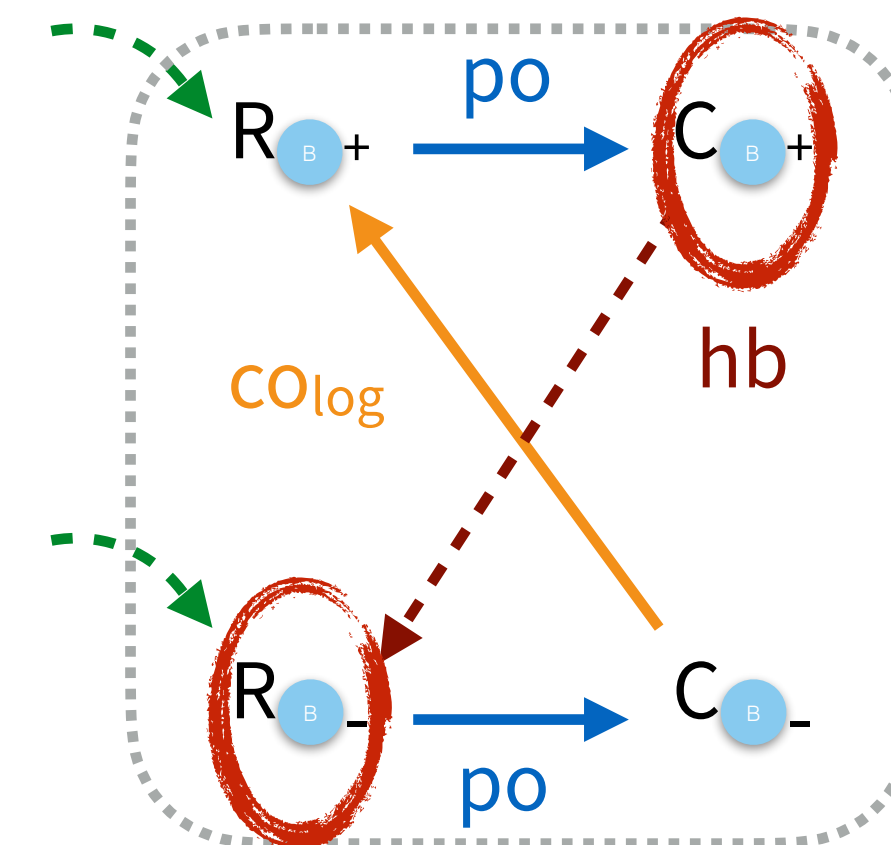
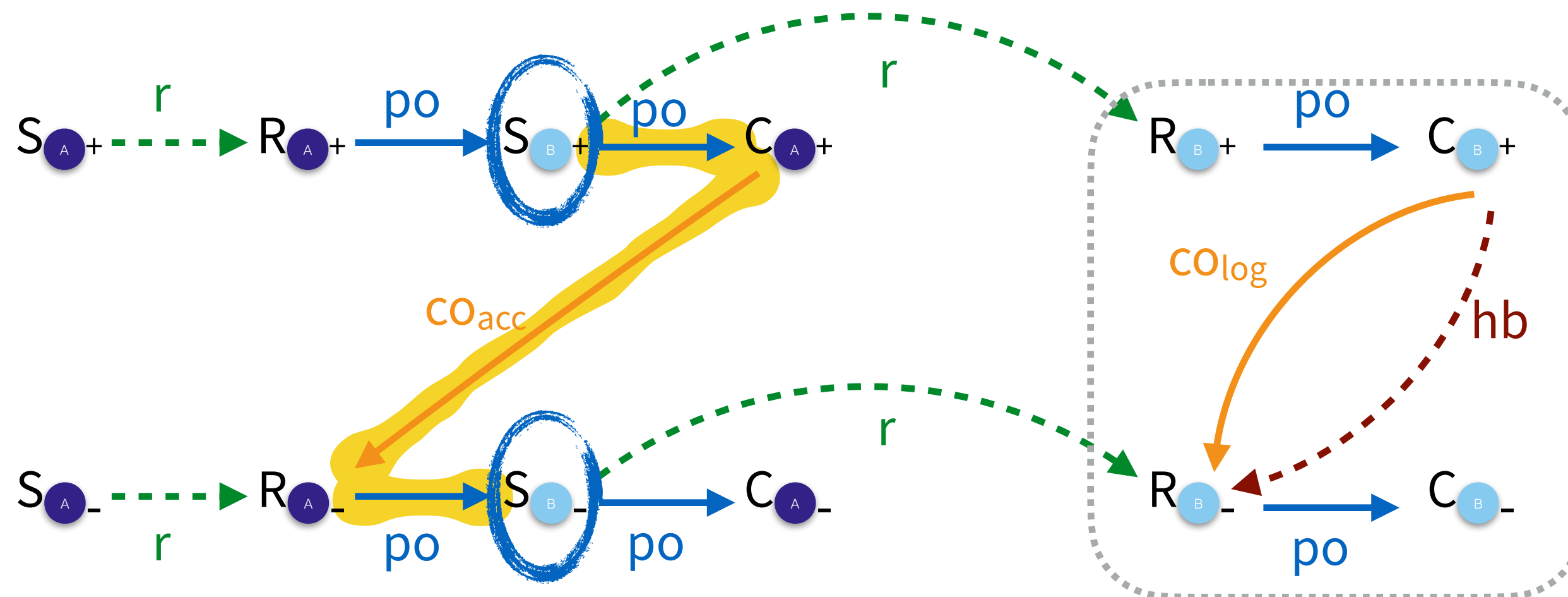


Validating Causality

```
def update(acc: cown[Account], amount: int) {  
  when(acc) { A  
    acc.balance += amount  
    val id = acc.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

```
update(src, +100)
```

```
update(src, -100)
```

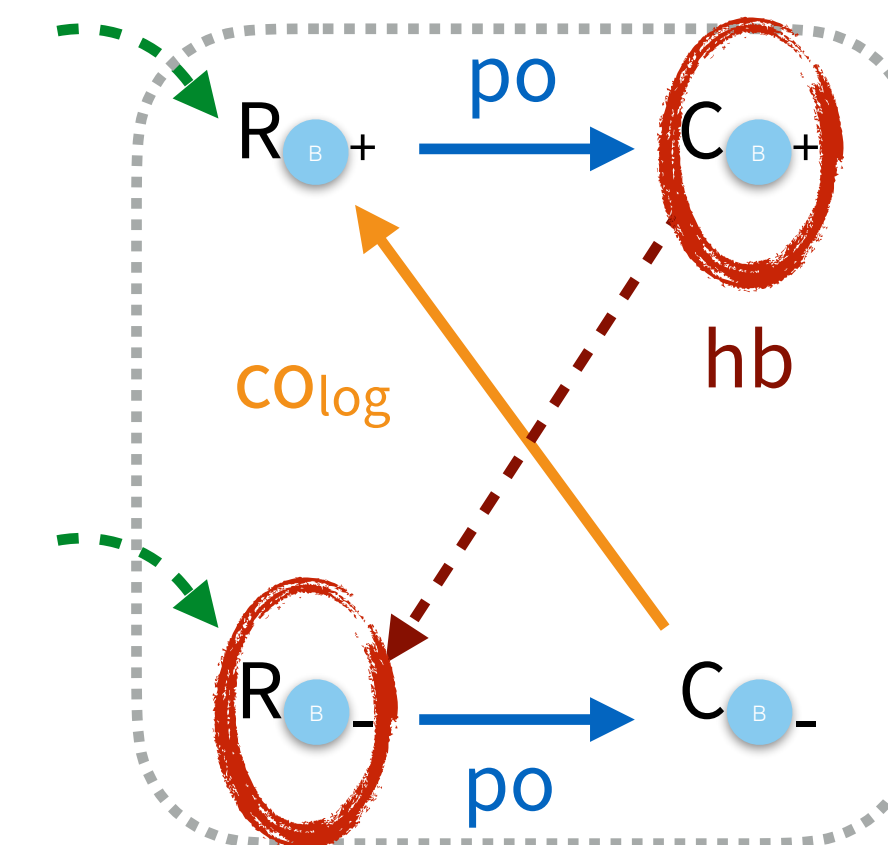
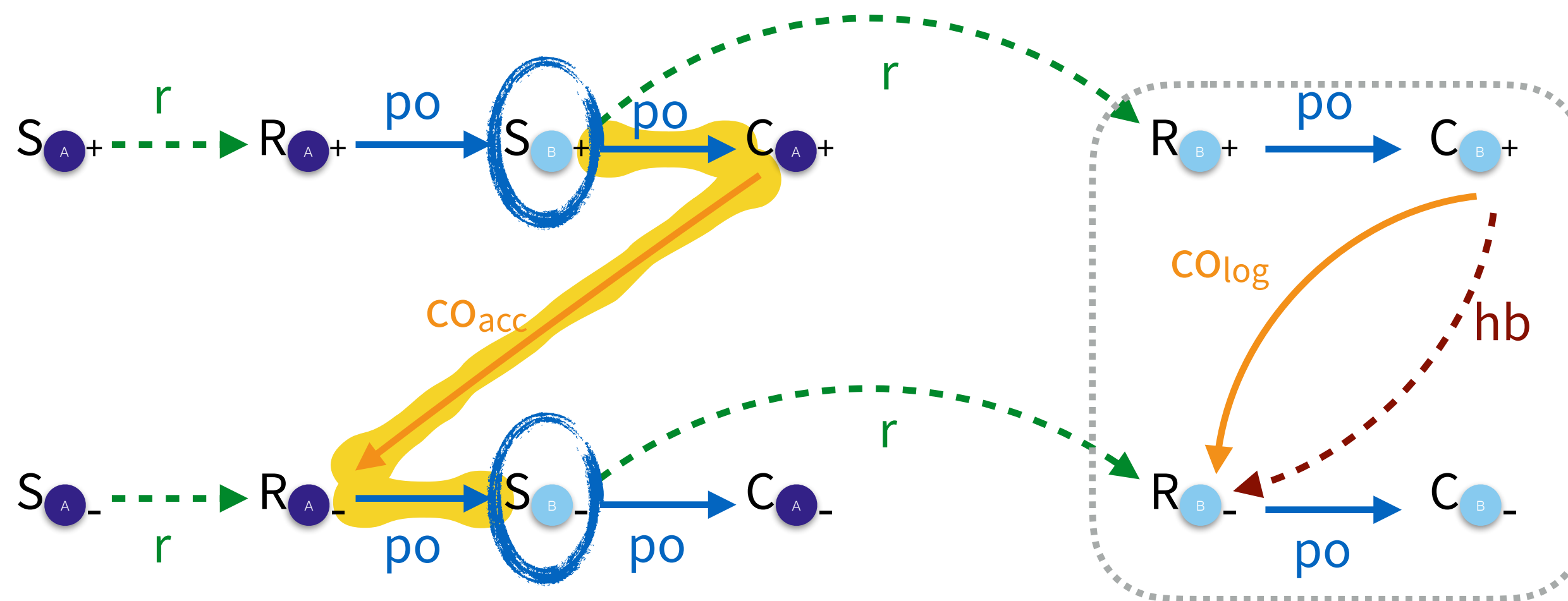


Validating Causality

```
def update(acc: cown[Account], amount: int) {  
  when(acc) { A  
    acc.balance += amount  
    val id = acc.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```

update(src, +100)

update(src, -100)



X Intentionally disallowed



What does this get us?

What does this get us?

$$\begin{array}{c}
 \overline{\text{when}(\bar{c})\{s_1\}; s_2 \hookrightarrow s_2 \mid (\bar{c}, s_1)}^{\text{STEP}} \\
 \\
 \overline{s_1 \hookrightarrow s_2 \mid (\bar{c}_3, s_3) \quad j \text{ fresh}}^{\text{SPAWN}} \\
 \overline{\langle \bar{b}_r[(i, \bar{c}_1, s_1)], \bar{b}_p \rangle / H \rightsquigarrow \langle \bar{b}_r[(i, \bar{c}_1, s_2)], \bar{b}_p : (j, \bar{c}_3, s_3) \rangle / H[i += S_j]} \\
 \\
 \overline{C = \{c \mid c \in \bar{c}' \wedge (_, \bar{c}', _) \in \bar{b}_r \cup \bar{b}_{p1}\} \quad C \cap \bar{c} = \emptyset}^{\text{RUN}} \\
 \overline{\langle \bar{b}_r, \bar{b}_{p1} : (i, \bar{c}, s) : \bar{b}_{p2} \rangle / H \rightsquigarrow \langle \bar{b}_r : (i, \bar{c}, s), \bar{b}_{p1} : \bar{b}_{p2} \rangle / H[i += R_i][\bar{c} += R_i]} \\
 \\
 \overline{\langle \bar{b}_r[(i, \bar{c}, \text{done})], \bar{b}_p \rangle / H \rightsquigarrow \langle \bar{b}_r[\epsilon], \bar{b}_p \rangle / H[i += C_i][\bar{c} += C_i]}^{\text{COMPLETE}}
 \end{array}$$

■ **Figure 9** Semantics of the core BoC calculus with histories.

$$\begin{array}{c}
 \overline{i \vdash_b \epsilon}^{\text{WFB-EMPTY}} \quad \overline{i \vdash_b R_i : \bar{S} : (C_i?)}^{\text{WFB-NEMPTY}} \\
 \\
 \overline{\vdash_c \epsilon}^{\text{WFC-EMPTY}} \quad \overline{\vdash_c R_i}^{\text{WFC-SINGLE}} \quad \overline{\vdash_c \bar{E}}^{\text{WFC-PAIR}} \\
 \overline{\vdash_c R_i : C_i : \bar{e}}
 \end{array}$$

■ **Figure 10** Well-formedness rules for behaviour and cown histories

4.3 Soundness of Core Calculus According to the Axiomatic Model

In the previous section, we showed that a well-formed history translates to a valid execution in the axiomatic model. In this section we show that the semantics is sound according the axiomatic model by proving that it always produces a well-formed history.

In addition to general well-formedness, the history of an execution is going to match the specific configuration that it was produced together with.

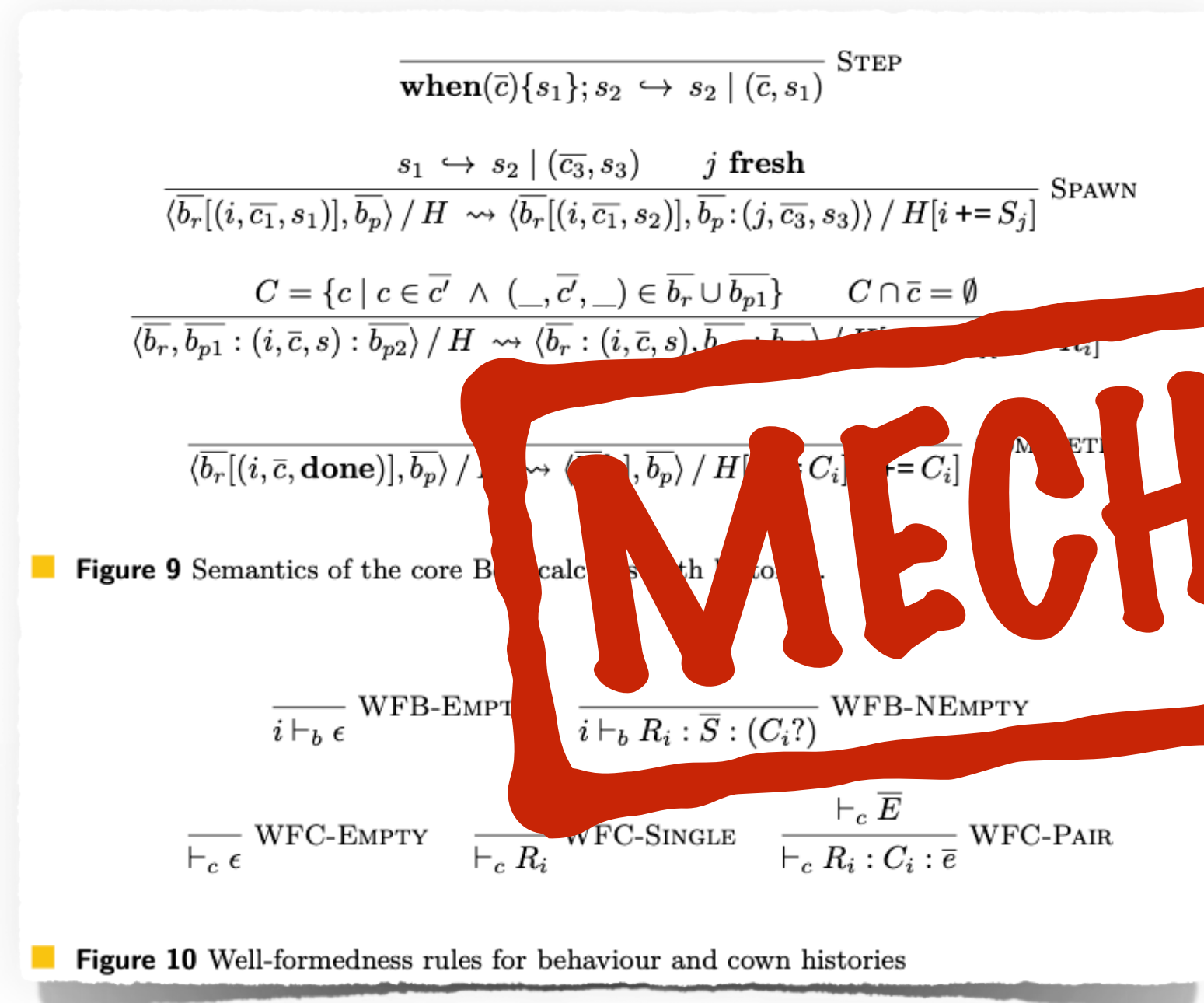
► **Theorem 15** (Preservation of well-formed and matching histories). *Starting from a well-formed and matching history, evaluation produces another well-formed and matching history:*

$$\tau \vdash H \wedge H; \tau \vdash c\bar{d}g \wedge c\bar{d}g/H \rightsquigarrow c\bar{d}g/H' \implies \exists \tau'. \tau' \vdash H' \wedge H'; \tau' \vdash c\bar{d}g$$

Proof. By cases on the evaluation relation. For each case we select a τ' that is τ extended so that the new event produced by that evaluation step is given a timestamp larger than any previous timestamp. Preservation of each of the two relations can then be proved separately. We refer to the mechanisation for details [7]. ◀

The operational semantics, and its implementations, are sound with respect to the axiomatic model

What does this get us?



4.3 Soundness of Core Calculus According to the Axiomatic Model

In the previous section, we showed that the operational semantics translates to a valid execution in the axiomatic model. In this section we show that the axiomatic semantics is sound according to the operational model by proving that it also produces a well-formed history.

In addition to this, we show that, the state of the execution is going to match the specific configuration that it is associated together with.

Theorem 15 (Preservation of well-formed and matching histories). *Starting from a well-formed and matching history, evaluation produces a well-formed and matching history:*

$$i \vdash_b H \vdash_c E \vdash_c R_i : C_i : \bar{e} \vdash_c R_i : C_i : \bar{e} \implies \exists \tau'. \tau' \vdash H' \wedge H'; \tau' \vdash E'$$

Proof. By cases on the evaluation relation. For each case we select a τ' that is τ extended so that the new event produced by that evaluation step is given a timestamp larger than any previous timestamp. Preservation of each of the two relations can then be proved separately. We refer to the mechanisation for details [7].

The operational semantics, and its implementations, are sound with respect to the axiomatic model

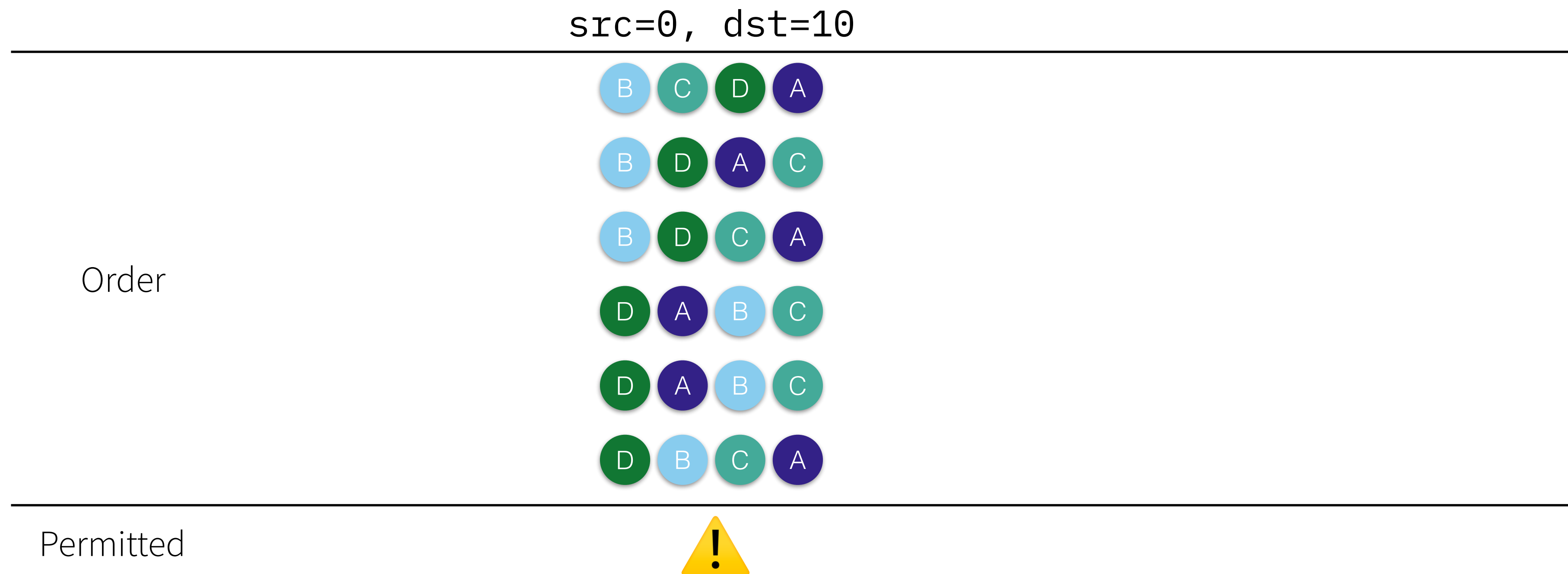
What does this get us?

```
when(src) { A src.balance += 10; }  
when(dst) { B dst.balance += 10; }
```

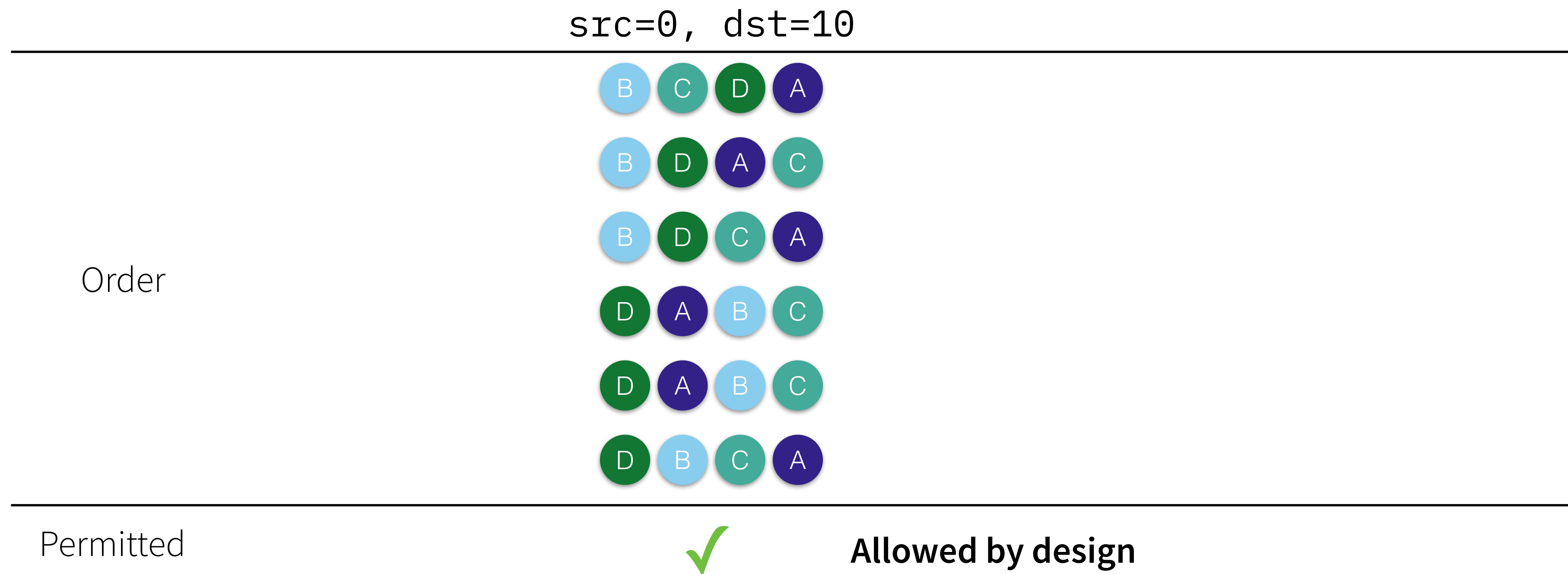
```
when(dst) { C print(dst.balance) }  
when(src) { D print(src.balance) }
```

	src=0, dst=0	src=0, dst=10	src=10, dst=0	src=10, dst=10
Order	C B D A C D A B C D B A D A C B D C A B D C B A	B C D A B D A C B D C A D A B C D A B C D B C A	A C B D A C D B A D C B C A B D C A D B C B A D	A B C D A B D C A D B C B A C D B A D C B C A D
Permitted	✓	⚠	✓	✓

What does this get us?



What does this get us?

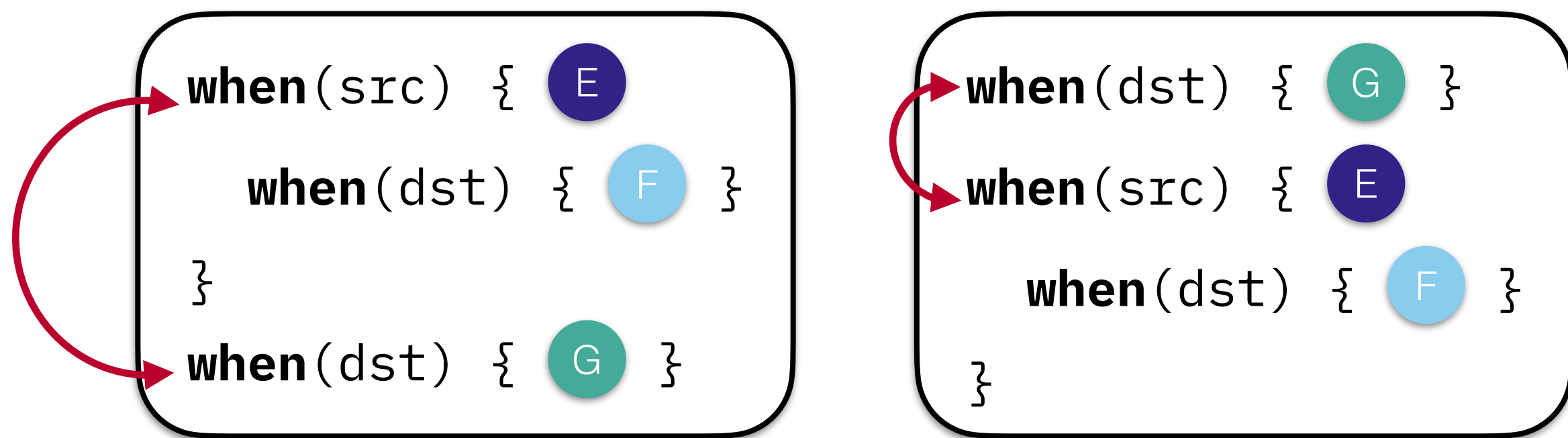




What does this get us?

We can reason about program transformations

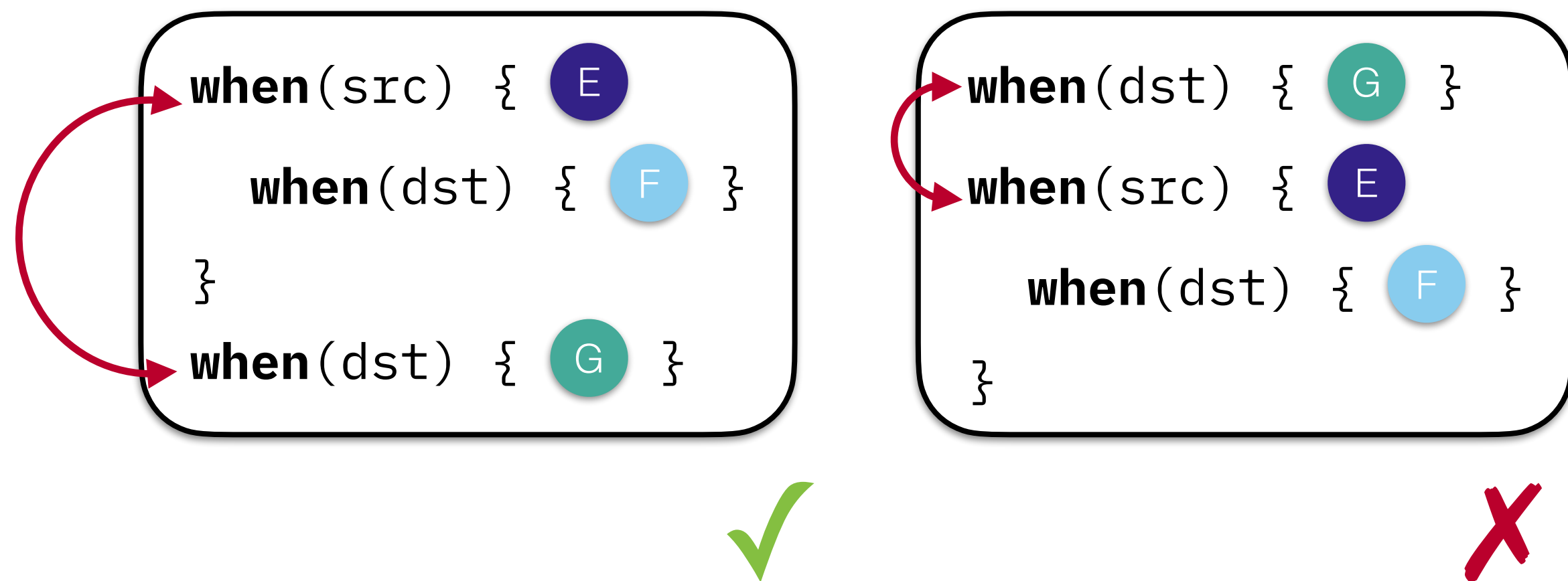
What does this get us?



Can we reorder behaviours?

We can reason about program transformations

What does this get us?

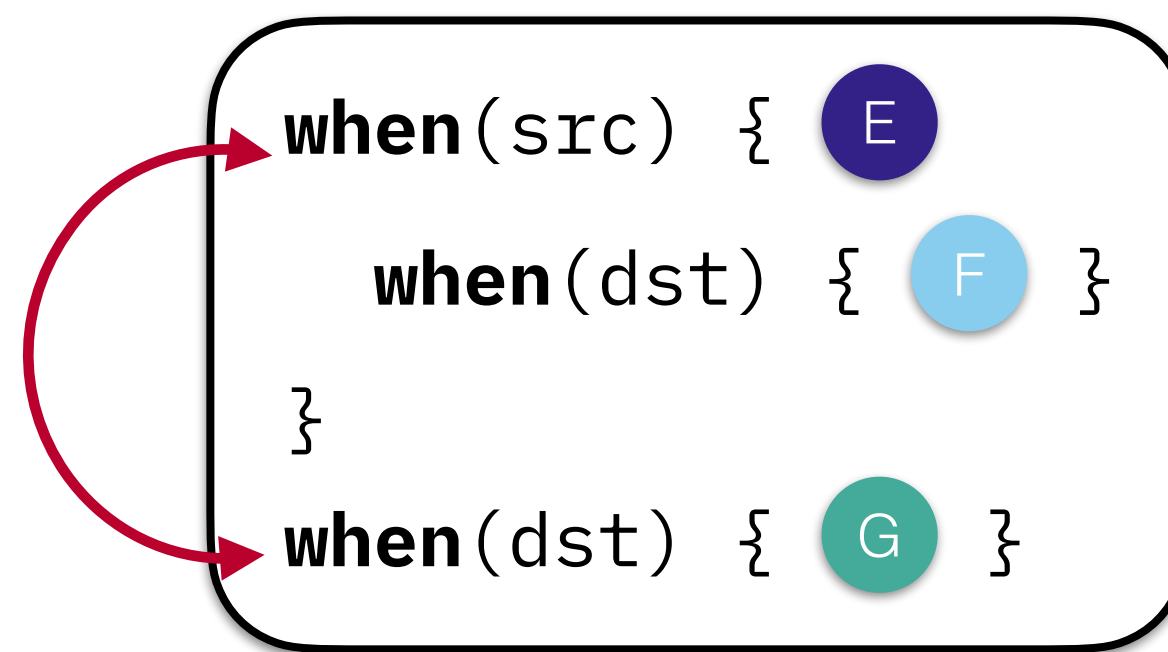


Can we reorder behaviours?

As long as we aren't violating causality

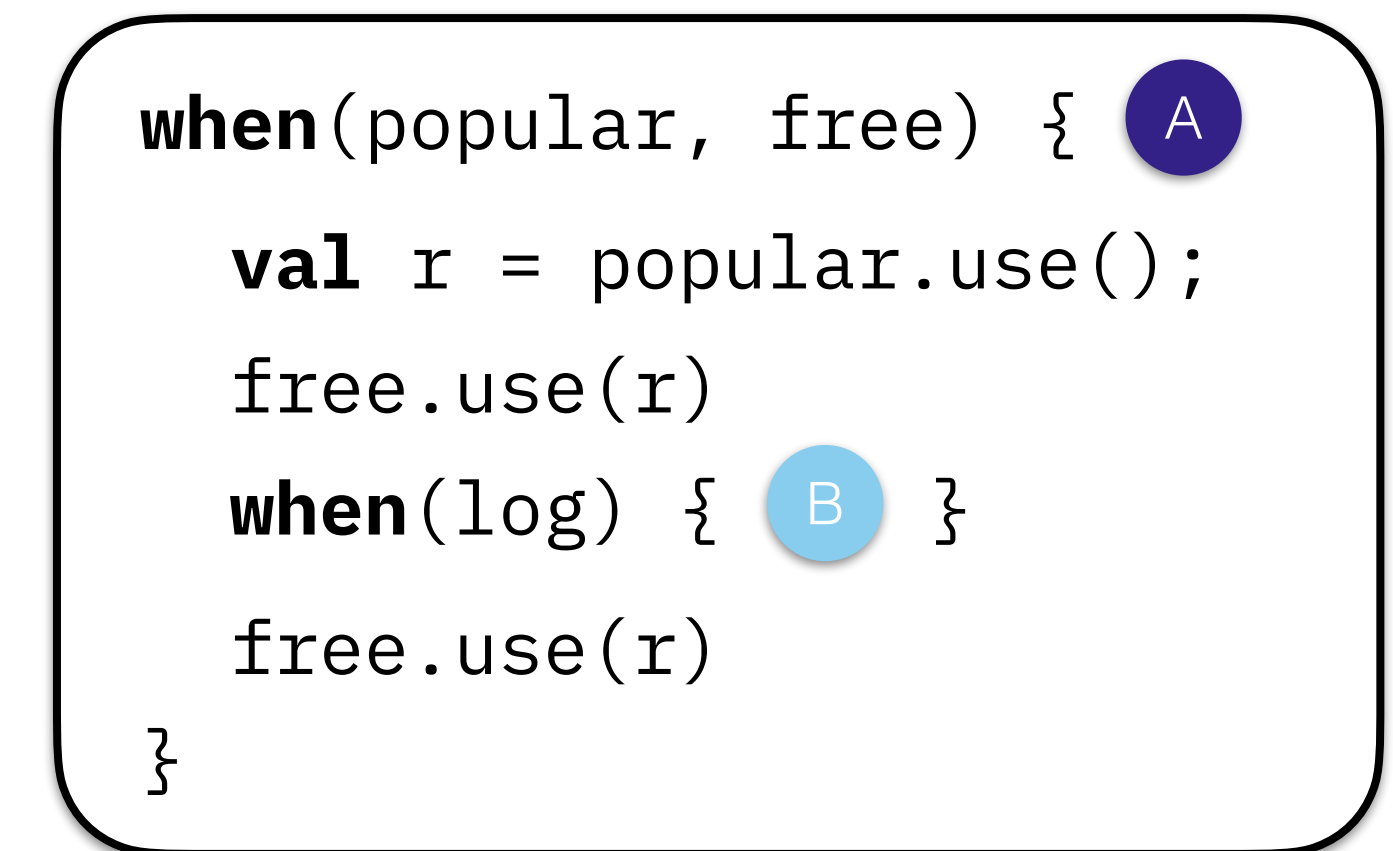
We can reason about program transformations

What does this get us?



Can we reorder behaviours?

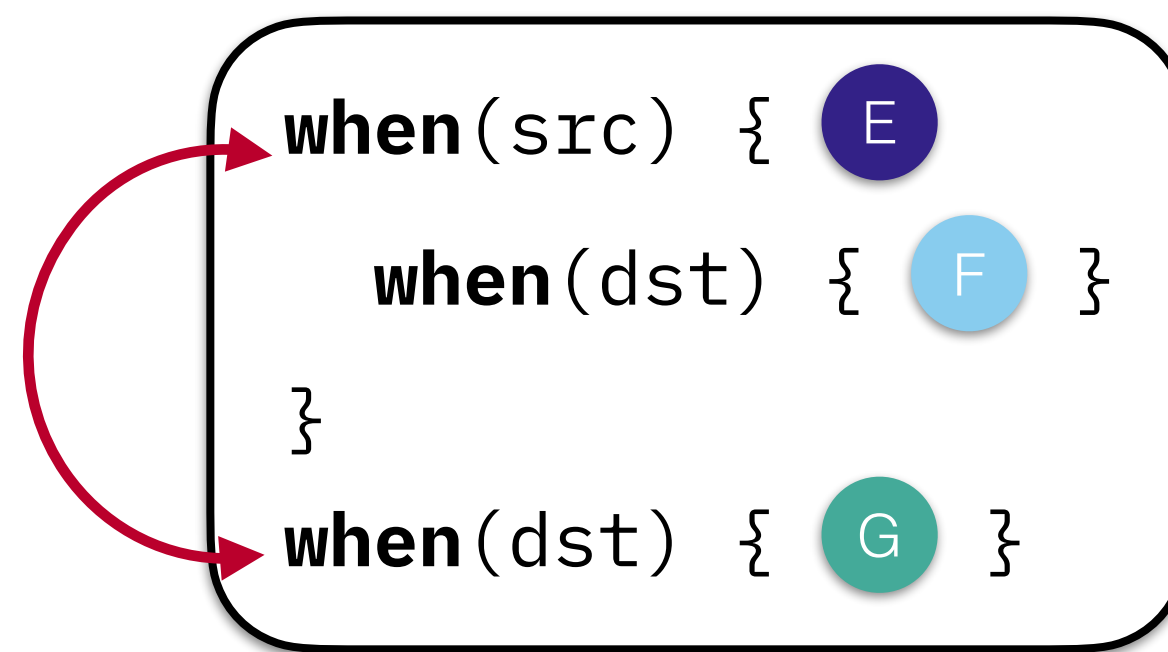
As long as we aren't violating causality



Can another behaviour start using a cow
before a behaviour finishes?

We can reason about program transformations

What does this get us?

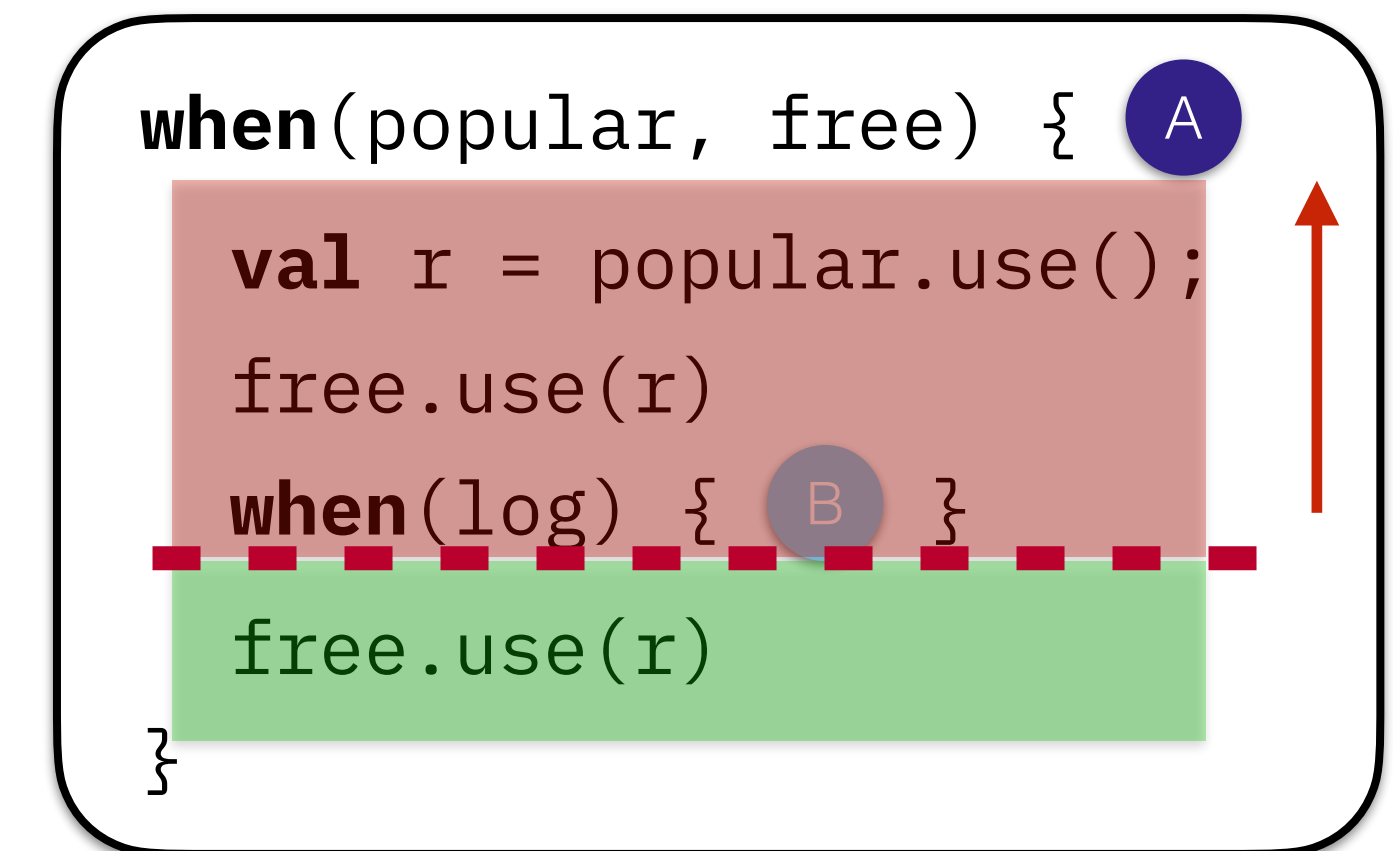


Can we reorder behaviours?

As long as we aren't violating causality



We can reason about program transformations



Can another behaviour start using a cow
before a behaviour finishes?

After the last behaviour spawn



Summary and Future

Summary and Future

- In this paper, we have...

Constructed an axiomatic model for Behaviour-oriented Concurrency

Captured the intended causal order between behaviours

Proven that our earlier work, and its derivatives, are sound with respect to this model

Provided an implementation-independent foundation for reasoning about BoC programs

when Behaviours Have to Happen

An Axiomatic Model of Causality in Behaviour-Oriented Concurrency

Luke Cheeseman ✉ 
Uppsala University, Sweden

Elias Castegren ✉ 
Uppsala University, Sweden

Tobias Wrigstad ✉ 
Uppsala University, Sweden

Sophia Drossopoulou ✉ 
Imperial College London, UK

Matthew J. Parkinson ✉ 
Azure Research, London, UK

Abstract

Behaviour-oriented concurrency (BoC) is a recently established programming model in which programmers define concurrent operations that execute atomically across multiple isolated resources. This allows for expressive interactions but introduces complex causal dependencies determined by dynamic resource overlap. Previous work defines the causal guarantees of BoC operationally, but mixes intended design constraints with incidental implementation details, leading to unintended causal orders. BoC is now being implemented across multiple languages and runtimes, all relying on the operational descriptions of causality. This paper develops an axiomatic model of BoC executions that makes the intrinsic orders explicit and derives the *intended* causal relation from their interaction. Using a set of representative programs and candidate executions, we motivate the design of this causal relation. We then prove that a representative minimal core calculus for BoC is sound with respect to this axiomatic model. Together, these results provide an implementation-independent foundation for reasoning about BoC causality across runtimes, schedulers and optimisation decisions.

2012 ACM Subject Classification Theory of computation → Concurrency

Keywords and phrases Concurrency, Parallelism, Language Design, Causality

Digital Object Identifier 10.4230/LIPIcs.CONCUR.2026.23

Supplementary Material *Software (Proofs)*: https://github.com/fxpl/axiomatic_boc [7]
archived at [swb:1:dir:24350cf7167ab1b9d00f8a2d64cc3fe32dfb4560](https://arxiv.org/abs/24350cf7167ab1b9d00f8a2d64cc3fe32dfb4560)

Acknowledgements We thank David Black-Schaffer and the reviewers for helpful discussions and feedback on the paper.

1 Introduction

Modern concurrent programming must balance the efficiency benefits of parallelism against the complexity of reasoning about program execution. Behaviour-oriented concurrency (BoC) is a programming model in which tasks declare required isolated resources; tasks with disjoint requirements may execute concurrently, while tasks with overlapping requirements are subject to explicit ordering guarantees, simplifying reasoning about program execution [10]. Previous work on BoC defines these ordering guarantees operationally, but mixes intended design constraints with incidental implementation details, inducing unintended causal orders [10]. BoC is now being developed in multiple languages and runtimes, including C++ and Python runtimes, and research languages such as Verona [22, 12, 13, 4]. As these efforts evolve, they rely on the informal and overly restrictive descriptions of causality inherent in the operational

 © Luke Cheeseman, Elias Castegren, Tobias Wrigstad, Sophia Drossopoulou, and Matthew J. Parkinson;
licensed under Creative Commons License CC-BY 4.0
37th International Conference on Concurrency Theory (CONCUR 2026).
Editors: Ana Sokolova and Patrick Totzke; Article No. 23; pp. 23:1–23:17
Leibniz International Proceedings in Informatics
LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

Summary and Future

- In this paper, we have...

Constructed an axiomatic model for Behaviour-oriented Concurrency

Captured the intended causal order between behaviours

Proven that our earlier work, and its derivatives, are sound with respect to this model

Provided an implementation-independent foundation for reasoning about BoC programs

- This enables us to...

Develop and verify optimisations for languages using BoC

Develop a memory model for languages that use BoC

Further refine BoC, e.g. through concurrent execution of behaviours that only read data



Summary and Future

- In this paper, we have...

Constructed an axiomatic model for Behaviour-oriented Concurrency

Captured the intended causal order between behaviours

Proven that our earlier work, and its derivatives, are sound with respect to this model

Provided an implementation-independent foundation for reasoning about BoC programs

- This enables us to...

Develop and verify optimisations for languages using BoC

Develop a memory model for languages that use BoC

Further refine BoC, e.g. through concurrent execution of behaviours that only read data

Thank you for listening

